University of the Philippines Cebu

Data Structures

Second Long Examination

Lecture Notes, Practice Examination and Worked Solutions

Arrays and Array Lists – Vectors – Singly and Doubly Linked Lists
Stacks and Queues – Binary Trees and Binary Search Trees

Compiled from the course lecture notes, slides and sample programs of the Second Long Examination coverage

D. Juevesano

November 2026

All program listings in this volume were compiled and executed; every printed output is the output of the program shown.

Preface

This volume prepares the reader for the Second Long Examination of the course on data structures. It has three parts.

Part I is a set of consolidated lecture notes. It merges the three lecture handouts (abstract data types and array-based lists; linked lists; binary trees), the slide decks on the List and Tree abstract data types, the sample programs `arraylist.c`, `binTree.cpp` and `bst.cpp`, and a linked-list reviewer, into one continuous exposition with definitions, theorems and proofs. The coverage is: arrays and array lists, vectors, singly and doubly linked lists, stacks, queues, and trees *up to binary trees and binary search trees*. Self-balancing trees (AVL and beyond) are *deliberately excluded*.

Part II is a practice examination written in the manner of the departmental examination: short conceptual questions, *tracing problems* that carry a twist, and *problem-solving* items in the style of an online judge, each with test cases, a required time complexity, and, where it applies, a bonus for a faster solution. Part III contains complete solutions with explanations.

The rule of abstraction. Every problem states the *permitted operations*. A solution that reaches into the representation of an ADT — for instance, indexing the array inside a stack when only `push` and `pop` are permitted — forfeits the point of the exercise and, in an examination, usually the credit. Where a problem asks for a new operation, the solution implements it *on top of* the given interface or, when the problem says so, directly on the representation (for example, relinking nodes).

Verification. Each solution, and each tracing program, was compiled with `gcc/g++` and run. The algorithmic solutions were additionally checked against brute-force models on thousands of random inputs, and the pointer-heavy ones under the address and undefined-behaviour sanitizers. The outputs printed in Part III are the real outputs of these programs.

Conventions. Positions in the *lecture-slide* array list are 0-based; the sample program `arraylist.c` uses *1-based* positions in `itemAt` and `setItem`. Always read the problem statement for the convention in force. The *height* of a tree is the number of edges on a longest downward path, and the empty tree has height -1. Throughout, n denotes the number of items.

Sources. The chapter structure of the lecture handouts follows P. Morin, *Open Data Structures* (`ArrayStack`, `ArrayQueue`, `ArrayDeque`, `SLList`, `DLList`, `BinaryTree`, `BinarySearchTree`), which the handouts adapt to C++; all errors in this volume are the compiler's, or mine.

 Juevesano D. 2026 All rights reserved.

Contents

Preface ii

I Lecture Notes 1

1 Foundations 2
1.1 Abstract data types . . . 2
1.2 Asymptotic analysis . . . 2
1.3 Pointers, memory and parameter passing . . . 3
1.4 Reasoning about correctness . . . 4

2 Array-Based Lists 5
2.1 Arrays and their shortcomings . . . 5
2.2 The array list . . . 5
2.3 Dynamic arrays and the vector . . . 7
2.4 Sorted array lists and binary search . . . 8
2.5 Circular arrays: queues and deques . . . 9

3 Linked Lists 11
3.1 The singly linked list . . . 11
3.1.1 Insertion and removal at the ends . . . 12
3.1.2 Case analysis: draw before you code . . . 14
3.2 The doubly linked list . . . 14
3.2.1 Sentinel (dummy) nodes . . . 16
3.3 Programming patterns . . . 17
3.4 Arrays versus linked lists . . . 18

4 Stacks and Queues 19
4.1 The abstract data types . . . 19
4.2 Implementations . . . 19
4.2.1 Array-based . . . 19
4.2.2 Linked . . . 21
4.3 Applications of stacks . . . 21
4.3.1 Balanced brackets . . . 21
4.3.2 Evaluation of expressions . . . 22
4.3.3 Other recurring uses . . . 24
4.4 Applications of queues . . . 25
4.5 Converting between the two . . . 25
4.6 Deque . . . 27

Juevesano D. 2026 All rights reserved.

5 Binary Trees and Binary Search Trees **29**
5.1 Terminology . . . 29
5.2 Counting facts . . . 29
5.3 Representation and recursion . . . 30
5.3.1 The recursive pattern . . . 33
5.4 Traversals . . . 34
5.4.1 Expression trees . . . 34
5.4.2 Iterative inorder . . . 35
5.4.3 Reconstruction . . . 35
5.5 Binary search trees . . . 35
5.5.1 Search and insertion . . . 36
5.5.2 Removal . . . 36
5.5.3 Order-based queries . . . 38

II Practice Examination **39**

6 Instructions and Marking Scheme **40**
6.1 How to use this part . . . 40
6.2 Conventions used in the problem statements . . . 40
6.3 A sample marking scheme . . . 41

7 Section A: Concept Questions **42**

8 Section B: Tracing Problems **45**
Tracing 1. Array list, return values and stale cells . . . 45
Tracing 2. What does the callee really change? . . . 47
Tracing 3. Counting loop iterations . . . 47
Tracing 4. The raw array after erase and insert . . . 48
Tracing 5. Circular queue: wrap-around . . . 49
Tracing 6. SLL with head, tail and size . . . 50
Tracing 7. One star or two? . . . 51
Tracing 8. Erasing from a doubly linked list . . . 52
Tracing 9. Recursion on a list . . . 52
Tracing 10. Stacks and queues in the same function . . . 53
Tracing 11. Infix to postfix . . . 54
Tracing 12. A queue from two stacks . . . 55
Tracing 13. Level-order insertion is not BST insertion . . . 56
Tracing 14. BST with duplicates and comparison counts . . . 56
Tracing 15. BST removals . . . 57
Tracing 16. A recursive tree mystery . . . 57
Tracing 17. The forgotten parent . . . 58
Tracing 18. Iterative inorder with a depth counter . . . 59
Tracing 19. Successor and predecessor . . . 59
Tracing 20. The mid-pointer list . . . 60
Tracing 21. Binary search: counting the steps . . . 60
Tracing 22. Josephus with a twist . . . 61

 Juevesano D. 2026 All rights reserved.

Tracing 23. Splicing lists . . . 61
Tracing 24. Shallow and deep copies . . . 62
Tracing 25. Expression tree . . . 63

9 Section C: Problem Solving 64
9.1 Arrays and Array Lists . . . 64
Problem 1. String compression (run-length encoding) . . . 64
Problem 2. Matrix transpose . . . 65
Problem 3. The missing number . . . 65
Problem 4. A sorted array list . . . 65
Problem 5. Rotate an array . . . 66
Problem 6. Merge two sorted arrays . . . 67
Problem 7. Remove duplicates from a sorted array . . . 67
Problem 8. Two-sum in a sorted array . . . 67
Problem 9. Maximum subarray sum . . . 68
Problem 10. Move zeroes . . . 68
Problem 11. Majority element . . . 68
Problem 12. Rotate a matrix by 90 degrees . . . 69
9.2 Linked Lists . . . 69
Problem 13. Implementation: eraseItems (DLL) . . . 69
Problem 14. Implementation: appendList (copying a list) . . . 70
Problem 15. Implementation: recursive append with a double pointer . . . 71
Problem 16. Reverse a singly linked list . . . 72
Problem 17. Middle node and k-th node from the end . . . 72
Problem 18. Splicing singly linked lists . . . 73
Problem 19. Merge two sorted doubly linked lists . . . 73
Problem 20. Palindromic doubly linked list . . . 74
Problem 21. Reverse a doubly linked list . . . 74
Problem 22. Remove duplicates from a sorted doubly linked list . . . 75
Problem 23. Insert into a sorted doubly linked list . . . 75
Problem 24. An SLL with a middle pointer . . . 75
Problem 25. Rotate a linked list . . . 76
Problem 26. Partition a linked list around a value . . . 77
Problem 27. Remove the n-th node from the end . . . 77
Problem 28. Swap nodes in pairs . . . 77
Problem 29. Cycle detection . . . 78
Problem 30. Group odd and even positions . . . 78
Problem 31. Add two numbers stored in linked lists . . . 79
9.3 Stacks and Queues . . . 79
Problem 32. Valid brackets . . . 79
Problem 33. Infix to postfix and postfix evaluation . . . 80
Problem 34. A queue from two stacks . . . 80
Problem 35. A stack with constant-time minimum . . . 81
Problem 36. Next greater element . . . 81
Problem 37. Josephus problem . . . 82
Problem 38. Reverse the first k items of a queue . . . 82

Juevesano D. 2026 All rights reserved.

Problem 39. Sort a stack . . . 82
Problem 40. Remove adjacent duplicates . . . 83
Problem 41. A stack from a queue . . . 83
9.4 Binary Trees and Binary Search Trees . . . 84
Problem 42. Tree measures . . . 84
Problem 43. Level order printing . . . 84
Problem 44. Validate a binary search tree . . . 85
Problem 45. Inorder successor and predecessor . . . 85
Problem 46. k-th smallest item of a BST . . . 85
Problem 47. Lowest common ancestor in a BST . . . 86
Problem 48. Mirror, same tree, symmetric, path sum . . . 86
Problem 49. Sorted array to balanced BST . . . 87
Problem 50. Range count, floor and ceiling . . . 87
Problem 51. Inorder traversal without recursion . . . 88
Problem 52. Remove a key from a BST . . . 88

III Solutions and Explanations **89**

10 Solutions to Section A **90**

11 Solutions to Section B **94**
Solution to Tracing 1 . . . 94
Solution to Tracing 2 . . . 94
Solution to Tracing 3 . . . 95
Solution to Tracing 4 . . . 95
Solution to Tracing 5 . . . 96
Solution to Tracing 6 . . . 96
Solution to Tracing 7 . . . 96
Solution to Tracing 8 . . . 97
Solution to Tracing 9 . . . 97
Solution to Tracing 10 . . . 97
Solution to Tracing 11 . . . 98
Solution to Tracing 12 . . . 98
Solution to Tracing 13 . . . 98
Solution to Tracing 14 . . . 99
Solution to Tracing 15 . . . 99
Solution to Tracing 16 . . . 99
Solution to Tracing 17 . . . 100
Solution to Tracing 18 . . . 100
Solution to Tracing 19 . . . 100
Solution to Tracing 20 . . . 100
Solution to Tracing 21 . . . 101
Solution to Tracing 22 . . . 101
Solution to Tracing 23 . . . 102
Solution to Tracing 24 . . . 102

 Juevesano D. 2026 All rights reserved.

Solution to Tracing 25 . . . 102

12 Solutions to Section C **103**
12.1 Arrays and Array Lists . . . 103
Solution to Problem 1 . . . 103
Solution to Problem 2 . . . 105
Solution to Problem 3 . . . 106
Solution to Problem 4 . . . 107
Solution to Problem 5 . . . 109
Solution to Problem 6 . . . 110
Solution to Problem 7 . . . 111
Solution to Problem 8 . . . 111
Solution to Problem 9 . . . 112
Solution to Problem 10 . . . 113
Solution to Problem 11 . . . 114
Solution to Problem 12 . . . 115
12.2 Linked Lists . . . 116
Solution to Problem 13 . . . 116
Solution to Problem 14 . . . 118
Solution to Problem 15 . . . 119
Solution to Problem 16 . . . 120
Solution to Problem 17 . . . 121
Solution to Problem 18 . . . 122
Solution to Problem 19 . . . 124
Solution to Problem 20 . . . 125
Solution to Problem 21 . . . 126
Solution to Problem 22 . . . 127
Solution to Problem 23 . . . 127
Solution to Problem 24 . . . 128
Solution to Problem 25 . . . 131
Solution to Problem 26 . . . 132
Solution to Problem 27 . . . 134
Solution to Problem 28 . . . 135
Solution to Problem 29 . . . 136
Solution to Problem 30 . . . 137
Solution to Problem 31 . . . 138
12.3 Stacks and Queues . . . 139
Solution to Problem 32 . . . 139
Solution to Problem 33 . . . 140
Solution to Problem 34 . . . 141
Solution to Problem 35 . . . 143
Solution to Problem 36 . . . 144
Solution to Problem 37 . . . 145
Solution to Problem 38 . . . 146
Solution to Problem 39 . . . 147
Solution to Problem 40 . . . 148

Juevesano D. 2026 All rights reserved.

Solution to Problem 41 . . . 149
12.4 Binary Trees and Binary Search Trees . . . 150
Solution to Problem 42 . . . 150
Solution to Problem 43 . . . 151
Solution to Problem 44 . . . 152
Solution to Problem 45 . . . 153
Solution to Problem 46 . . . 154
Solution to Problem 47 . . . 155
Solution to Problem 48 . . . 156
Solution to Problem 49 . . . 158
Solution to Problem 50 . . . 159
Solution to Problem 51 . . . 160
Solution to Problem 52 . . . 161

A Complexity Summary **163**

B Pitfall Checklist **164**

C Discussion Notes: Binary Trees in C++ **165**
C.1 The node class and the three traversals . . . 165
C.2 A queue of nodes and level order . . . 166
C.3 Insertion from left to right . . . 168
C.3.1 Mistakes in a first attempt . . . 171
C.4 Changing the root: `*r`, `*&r`, `**r` . . . 171
C.5 Two printing bugs . . . 172
C.6 Counting nodes, leaves and internal nodes . . . 172
C.6.1 Putting it together . . . 173
C.7 Recursive left-to-right insertion . . . 174
C.8 Expression trees . . . 176
C.9 Binary search trees . . . 179
C.10 Insertion without a double pointer . . . 185
C.11 Time complexity . . . 188

 Juevesano D. 2026 All rights reserved.

Part I

Lecture Notes

Juevesano D. 2026 All rights reserved.

Chapter 1

Foundations

1.1 Abstract data types

Definition 1.1 (Abstract data type)**.** An *abstract data type* (ADT) is a mathematical model consisting of a set of values together with a set of operations that access or manipulate those values. The ADT specifies *what* the operations do, never *how* they are carried out.

An ADT is therefore a contract. The *interface* lists the operations and their meaning; an *implementation* (the lecture notes call it simply that) chooses a concrete representation and supplies the algorithms. One interface may have many implementations with different costs, and the costs — not the results — are what distinguish them.

Example 1.2. The *List* ADT is a sequence $x_0, x_1, \ldots, x_{n-1}$ with the operations `size()`, `get(i)`, `set(i,x)`, `add(i,x)` (insert x at position i, displacing $x_i, \ldots, x_{n-1}$ by $+1$) and `remove(i)` (delete x_i, displacing $x_{i+1}, \ldots, x_{n-1}$ by -1). Two implementations are studied here: the *array list*, which stores x_i in `a[i]`, and the *linked list*, which stores the items in nodes joined by pointers.

ADT	Operations (course names)	Discipline
Queue	`add(x)`, `remove()`	first in, first out
Stack	`push(x)`, `pop()`	last in, first out
Deque	`addFirst`, `removeFirst`, `addLast`, `removeLast`	both ends
List	`size`, `get(i)`, `set(i,x)`, `add(i,x)`, `remove(i)`	indexed sequence

Table 1.1: The sequence ADTs of the lecture notes. A Stack is a Deque restricted to `addFirst/removeFirst`; a FIFO Queue is a Deque restricted to `addLast/removeFirst`; a Deque is a List restricted to `add(0,x)`, `remove(0)`, `add(size(),x)`, `remove(size()-1)`.

> The rule of abstraction. When a problem lists the permitted operations, those and only those may be applied to the structure. A client of a `stack` writes `push`, `pop`, `top`, `empty`; it never touches the array inside. New operations are allowed *if you implement them* from the permitted ones (or, when a problem explicitly says so, from the representation).

1.2 Asymptotic analysis

Definition 1.3 (Big-Oh, Big-Omega, Big-Theta)**.** Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. Then $f(n) = O(g(n))$ if there are constants $c > 0$ and n_0 such that $f(n) \leq c\, g(n)$ for all $n \geq n_0$; $f(n) = \Omega(g(n))$ if $f(n) \geq c\, g(n)$ for all $n \geq n_0$ (some $c > 0$); $f(n) = \Theta(g(n))$ if both hold.

Example 1.4 (The thought exercise of the slides)**.** (i) $23 = O(\log n)$ *is true.* For $n \geq 2$ we have $\lg n \geq 1$, so $23 \leq 23 \lg n$; take $c = 23$, $n_0 = 2$. (Constants are $O(1) \subseteq O(\log n)$: O is an *upper* bound only.)

 Juevesano D. 2026 All rights reserved.

(ii) $5 \log n = O(1)$ *is false.* Suppose $5 \lg n \leq c$ for all $n \geq n_0$. Choosing $n > \max(n_0, 2^{c/5})$ gives $5 \lg n > c$, a contradiction.

(iii) $5n - 7 = O(n)$ *is true.* For $n \geq 2$, $5n - 7 \leq 5n$; take $c = 5$, $n_0 = 2$ (indeed $n_0 = 2$ is needed only to keep the left side non-negative).

Calculating running time.. The running time of a program is estimated by counting how often its dominant statement executes.

- A loop that runs n times around a constant-time body costs $\Theta(n)$; two nested such loops cost $\Theta(n^2)$.
- *Triangular loops* `for(i<n) for(j<=i)` execute the body $1 + 2 + \cdots + n = n(n+1)/2 = \Theta(n^2)$ times.
- A loop whose index is *multiplied or divided* by a constant each round (`i*=2`, `i/=2`) runs $\Theta(\log n)$ times: starting at 1, the values $1, 2, 4, \ldots$ stay below n for $\lceil \lg n \rceil$ rounds.
- Sequential blocks add; the larger term wins: $n^2 + n \log n = \Theta(n^2)$.
- A recursive procedure that does $\Theta(1)$ work per call and makes one call on $n - 1$ items costs $\Theta(n)$; one that makes two calls on halves and combines in $\Theta(n)$ costs $\Theta(n \log n)$.

Class	Name	Typical source
$\Theta(1)$	constant	`a[i]`, push on an array stack, pointer update
$\Theta(\log n)$	logarithmic	binary search, height of a balanced tree
$\Theta(n)$	linear	traversal, `search` in an unsorted list
$\Theta(n \log n)$	linearithmic	merge sort, sorting before binary search
$\Theta(n^2)$	quadratic	nested loops, insertion at the front n times

Table 1.2: The growth classes met in this course, in increasing order.

Remark 1.5 (Worst, best, amortised). *Worst case* bounds the cost of a single operation over all inputs of size n. *Amortised* cost bounds the average over a *sequence* of operations, even when an individual operation is expensive (Theorem 2.3). Neither is the "average over random inputs".

1.3 Pointers, memory and parameter passing

Every local variable lives on the *call stack*; objects obtained with `malloc` (C) or `new` (C++) live on the *heap* until released with `free` or `delete`. A pointer is a variable holding an address; `p->f` abbreviates `(*p).f`; and `NULL` is the address that refers to nothing.

Memory leak. A heap block that is no longer reachable but was never freed. Every `malloc/new` on the path of a removal must be matched by a `free/delete` of the removed node.

Dangling pointer. A pointer to a block that has been freed. After `free(cur)`, reading `cur->next` is undefined behaviour — *save the successor first*.

Mixing. Memory from `malloc` must be released with `free`, memory from `new` with `delete`; never cross them.

Pass by value, by pointer, by reference.. A C function always receives *copies* of its arguments. Hence:

Juevesano D. 2026 All rights reserved.

(i) A `struct` passed *by value* is copied *member by member*. If the struct contains an *array* (the course's array list `int items[MAX]`), the whole array is copied, and changes made in the callee are lost. If it contains a *pointer* (a linked list's `head`), only the pointer is copied: callee and caller share the *same nodes*. Modifying the shared nodes is visible to the caller; reassigning the copied `head` or `size` is not (Tracing 2).

(ii) A raw array parameter `int a[]` is really a pointer to the first element, so the callee modifies the caller's array.

(iii) To change a variable of the caller, pass its address: `void f(list *l)`, called as `f(&l)`. To change a *pointer* variable — for example the `head` of a list — pass the address of the pointer, a *pointer to a pointer* `node **h`, called as `f(&l.head)` (Section 3.3; Tracing 7).

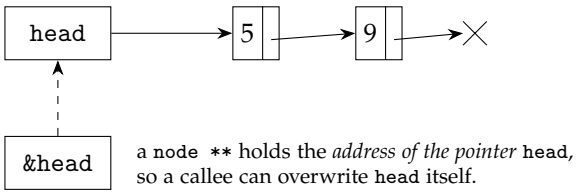


Figure 1.1: A pointer to a pointer.

> EVALUATION ORDER. In C and C++ the order in which function *arguments* (and the operands of `+`) are evaluated is *unspecified*. The statement `printf("%d %d", pop(&s), pop(&s));` may print the two pops in either order. A tracing question that is well posed never relies on it; write each call as its own statement.

1.4 Reasoning about correctness

Definition 1.6 (Invariant)**.** A *loop invariant* is a statement that is true before the loop, is preserved by each iteration, and, together with the exit condition, implies the result. A *class invariant* is a statement about a data structure that every operation assumes on entry and re-establishes on exit.

Example 1.7 (Invariant of the array list)**. (I)** $0 \leq$ `size` $\leq$ `MAX`, and the list is exactly `items[0..size-1]`. Cells with index $\geq$ `size` are *not part of the list*: their content is irrelevant, which is why `removeLast` only decrements `size` (the slides' question "should we worry about having two items equal to 10?" has the answer *no*).

Example 1.8 (Invariant of the singly linked list with tail)**. (I)** `size` equals the number of nodes reachable from `head`; `head==NULL` iff `tail==NULL` iff `size==0`; if non-empty, `tail` is the node whose `next` is `NULL`.

The edge-case checklist.. Before submitting any list or tree routine, run it by hand on: (a) the empty structure; (b) one element; (c) two elements; (d) the operation applied at the first position; (e) at the last position; (f) in the middle; (g) every element equal to the key; (h) no element equal to the key. Most lost marks in this course come from (a), (b), (d) and (e): forgetting to update `head`, `tail` or `size`.

 Juevesano D. 2026 All rights reserved.

Chapter 2

Array-Based Lists

2.1 Arrays and their shortcomings

An array stores m items of one type in m consecutive memory cells, so the address of cell i is computed in one step: `a[i]` costs $\Theta(1)$. The *primitive* C/C++ array has well-known shortcomings, which motivate every container in this chapter (lecture slides):

- its size is not stored — a function receiving `int a[]` cannot ask how long it is;
- bounds are not checked: `a[m]` silently reads or corrupts a neighbouring cell;
- inserting or deleting in the middle requires manual shifting;
- an array parameter is passed by address, so the callee changes the caller's array.

The remedy is to *wrap* the array together with its bookkeeping, so that the pair behaves as a list.

2.2 The array list

Definition 2.1 (Array list)**.** An *array list* keeps a fixed-capacity array `items[MAX]` and an integer `size`. The list is `items[0..size-1]`; invariant (I) of Example 1.7 holds.

The sample program `arraylist.c` defines the structure and the operations of Table 2.1.

Operation	Effect	Position convention	Cost
`init(l)`	`size=0`	—	$\Theta(1)$
`append(l,x)` / `push_back`	store at `items[size]`	—	$\Theta(1)$
`insertFront(l,x)`	shift all right, store at `items[0]`	—	$\Theta(n)$
`insert(pos,x)`	shift `items[pos..size-1]` right	0-based (slides)	$\Theta(n - \texttt{pos})$
`removeLast(l)`	`size-`	—	$\Theta(1)$
`removeFirst(l)`	shift all left, `size-`	—	$\Theta(n)$
`erase(pos)`	shift `items[pos+1..]` left	0-based (slides)	$\Theta(n - \texttt{pos})$
`itemAt(l,p)` / `setItem(l,x,p)`	read / write	**1-based** (`arraylist.c`)	$\Theta(1)$
`isFull(l)`	`size==MAX`	—	$\Theta(1)$

Table 2.1: Operations of the array list. Check the statement for the position convention.

```
1 #define MAX 10
2 typedef struct { int items[MAX]; int size; } list;
3
4 int append(list *l, int x) {              /* O(1) */
5     if (l->size == MAX) return 0;         /* full: report failure, change nothing
      */
```

```
6       l->items[l->size++] = x;              /* store, THEN increment
        */
7       return 1;
8   }
9   int insertFront(list *l, int x) {         /* O(n) */
10      int i;
11      if (l->size == MAX) return 0;
12      for (i = l->size; i > 0; i--)         /* move the LAST item first, so that
        */
13          l->items[i] = l->items[i - 1];    /* nothing is overwritten before it
        */
14      l->items[0] = x;                      /* is copied
        */
15      l->size++;
16      return 1;
17  }
18  int removeFirst(list *l) {                /* O(n) */
19      int i;
20      if (l->size == 0) return 0;
21      for (i = 0; i < l->size - 1; i++)     /* move the FIRST item first
        */
22          l->items[i] = l->items[i + 1];
23      l->size--;
24      return 1;
25  }
```

Listing 2.1: The array list of the sample program (excerpt).

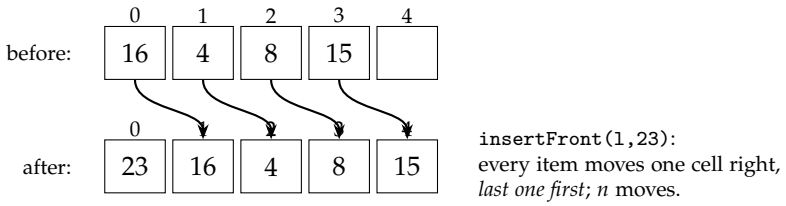


Figure 2.1: Insertion at the front shifts the whole list.

Theorem 2.2 (Cost of positional insertion and deletion)**.** *Inserting at (0-based) position p into an array list of size n performs exactly $n - p$ element moves; deleting the item at position p performs exactly $n - 1 - p$ moves. Hence both are $\Theta(n)$ in the worst case ($p = 0$) and $\Theta(1)$ at the rear ($p = n$ or $p = n - 1$).*

Proof. Insertion moves `items[p..n-1]`, which are $n - p$ cells, each once. Deletion moves `items[p+1..n-1]`, which are $n - 1 - p$ cells. ■

> Direction of the shift. Shift to the *right* (insertion) from the *end* towards the position; shift to the *left* (deletion) from the position towards the end. Doing it the other way overwrites items before they are copied and turns the whole tail into copies of one value — a classical tracing question.

 Juevesano D. 2026 All rights reserved.

2.3 Dynamic arrays and the vector

A fixed `MAX` fails when the list outgrows it. A *dynamic array* (the `ArrayStack` of the notes, the `std::vector` of C++) keeps a backing array of length `cap` $\geq n$ and replaces it when necessary.

- **Grow.** If an insertion finds `n+1 > cap`, allocate a new array of length $\max(1, 2n)$, copy the n items, release the old array.
- **Shrink.** After a removal, if `cap` $\geq 3n$, resize to $\max(1, 2n)$ so that memory is not wasted.

```
// A growable array of int ("vector list"), after the course notes.
// Invariant: 0 <= n <= cap, the list is a[0..n-1], and cap == 0 or cap >= 1.
class IntVector {
    int *a;      // the backing array (heap)
    int  cap;    // its length
    int  n;      // number of list items
    // resize: allocate a new array of length max(1, 2n) and copy the n items.
    void resize() {
        int newCap = (2 * n > 1) ? 2 * n : 1;
        int *b = (int *)malloc(newCap * sizeof(int));
        for (int i = 0; i < n; i++) b[i] = a[i];     // O(n) copy
        free(a);
        a = b;
        cap = newCap;
    }
public:
    IntVector()  { a = NULL; cap = 0; n = 0; }
    ~IntVector() { free(a); }
    int  size()     { return n; }
    int  capacity() { return cap; }
    int  get(int i) { return a[i]; }     // O(1); caller checks 0 <= i < n
    void set(int i, int x) { a[i] = x; }             // O(1)

    // add(i, x): insert x at position i (0 <= i <= n); shift a[i..n-1] right.
    void add(int i, int x) {
        if (n + 1 > cap) resize();               // full: double the array
        for (int j = n; j > i; j--) a[j] = a[j - 1];   // shift, from the END
        a[i] = x;
        n++;
    }
    void push_back(int x) { add(n, x); }             // amortised O(1)

    // remove(i): delete position i, shifting a[i+1..n-1] left.
    int remove(int i) {
        int x = a[i];
        for (int j = i; j < n - 1; j++) a[j] = a[j + 1];
        n--;
        if (cap >= 3 * n) resize();              // too empty: shrink (3n!)
        return x;
    }
    int pop_back() { return remove(n - 1); }         // amortised O(1)
```

Juevesano D. 2026 All rights reserved.

```
42 };
```

Listing 2.2: A growable array of integers (verified by randomised testing).

Starting from empty, twenty consecutive `push_back` calls produce the (size/capacity) pairs `1/1 2/2 3/4 4/4 5/8 ... 8/8 9/16 ... 16/16 17/32`: the capacity doubles exactly when the array is full.

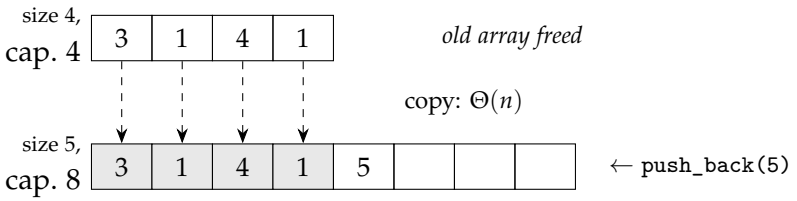
Theorem 2.3 (Amortised cost of `push_back`)**.** *Starting from an empty dynamic array that doubles when full, any sequence of m calls to `push_back` costs $O(m)$ in total, i.e. $O(1)$ amortised per call.*

Proof. A call that does not resize costs 1. Resizes occur when the size is $1, 2, 4, \ldots, 2^k$ with $2^k < m$, and the resize at size 2^j copies 2^j items. The total copying cost is $\sum_{j=0}^{k} 2^j = 2^{k+1} - 1 < 2m$. Adding the m unit costs gives a total below $3m$. ■

Remark 2.4 (Why shrink at $3n$ and not $2n$)*.* If the array shrank as soon as `cap` $\geq 2n$, then right after a doubling (`cap` $= 2n$) one removal would trigger a shrink and one insertion a grow: every operation would cost $\Theta(n)$. Waiting until `cap` $\geq 3n$ leaves a gap of $\Omega(n)$ operations between consecutive resizes, which restores $O(1)$ amortised cost.

`std::vector<T>`	Meaning	Cost
`v[i]`, `v.at(i)`	access (`at` checks bounds)	$\Theta(1)$
`v.size()`, `v.capacity()`, `v.empty()`	bookkeeping	$\Theta(1)$
`v.push_back(x)`, `v.pop_back()`	append / delete the last	$\Theta(1)$ amortised
`v.insert(it,x)`, `v.erase(it)`	positional (shifts)	$\Theta(n - \text{pos})$
`v.front()`, `v.back()`	ends	$\Theta(1)$
`v.reserve(k)`	pre-allocate capacity $\geq k$	$\Theta(n)$
`v.begin()`, `v.end()`	iterators	$\Theta(1)$

Table 2.2: The vector interface. A resize invalidates all pointers, references and iterators into the old array.



`push_back` on a full vector: a new array of double capacity is allocated, the items are copied, the old array is freed, then 5 is stored. All pointers and iterators into the old array are invalid afterwards.

2.4 Sorted array lists and binary search

If the list is kept in non-decreasing order, searching need not be linear. Let ℓ be the array `items[0..n-1]`.

Definition 2.5 (Lower and upper bound)**.** $\mathrm{lb}(x)$ is the smallest index i with `items`$[i] \geq x$, or n if none; $\mathrm{ub}(x)$ is the smallest index i with `items`$[i] > x$, or n if none.

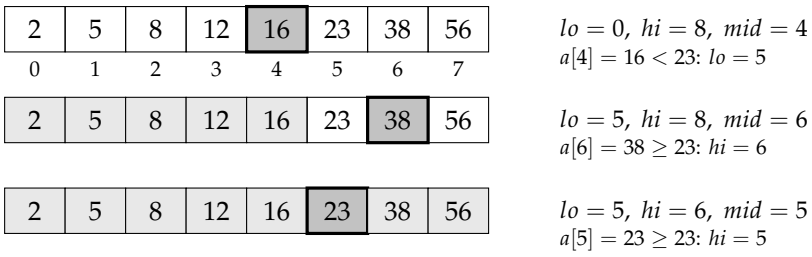
 Juevesano D. 2026 All rights reserved.

Then x occurs in the list iff $\mathrm{lb}(x) < n$ and $\texttt{items}[\mathrm{lb}(x)] = x$; the number of occurrences is $\mathrm{ub}(x) - \mathrm{lb}(x)$; and the sorted position for inserting x *after* equal items is $\mathrm{ub}(x)$.

```
int lowerBound(int a[], int n, int x) {
    int lo = 0, hi = n, mid;
    /* Invariant: a[i] <  x for every i < lo,
                  a[i] >= x for every i >= hi.   */
    while (lo < hi) {
        mid = lo + (hi - lo) / 2;      /* never (lo+hi)/2: avoids overflow       */
        if (a[mid] < x) lo = mid + 1;  /* mid and everything left is too small   */
        else            hi = mid;      /* mid could be the answer: keep it       */
    }
    return lo;                          /* lo == hi: the boundary
    */
}
```

Listing 2.3: Lower bound by bisection.

Theorem 2.6 (Correctness and cost of bisection)**.** *The invariant holds initially (the ranges are empty), is preserved by both branches, and when $lo = hi$ it says exactly that $lo = \mathrm{lb}(x)$. Each round halves $hi - lo$, so the loop runs at most $\lfloor \lg n \rfloor + 1$ times: search is $\Theta(\log n)$.*



`lowerBound(a, 8, 23)`. Shaded cells are already excluded; the highlighted cell is `mid`. The range $[lo, hi)$ halves each round.

Proposition 2.7 (Cost of the sorted array list)**.** *`search`, `count`, `lowerBound`, `upperBound`:* $\Theta(\log n)$. *`insert` and `erase`:* $\Theta(\log n)$ *to locate plus* $\Theta(n)$ *to shift, i.e.* $\Theta(n)$ *overall. Removing* all *copies of a value with a single shift costs* $\Theta(\log n + n)$ *rather than* $\Theta(kn)$ *for k repeated `erase` calls.*

Complete code appears in Problem 4. Note that sorting does not make *insertion* cheaper; it makes *searching* cheaper.

2.5 Circular arrays: queues and deques

An array queue that always keeps its front at `a[0]` pays $\Theta(n)$ per dequeue. The remedy is to remember the index j of the front, and to store item i of the queue at `a[(j+i) % a.length]`, so that indices wrap around (7:00 + 10h = 5:00 because $17 \equiv 5 \pmod{12}$).

- `add(x)`: if full, resize; store at `a[(j+n)%len]`; $n{+}{+}$.

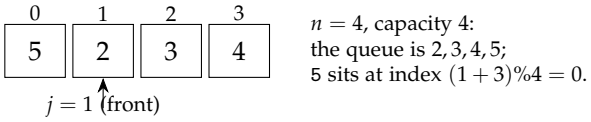


Figure 2.2: A full circular queue (the state of Tracing 5).

- `remove()`: take `a[j]`; $j = (j + 1)\%$`len`; $n--$; resize if `len` $\geq 3n$.
- `resize()`: allocate b of length $\max(1, 2n)$; copy `a[(j+i)%len]` into `b[i]`; reset $j = 0$.

Both operations are $\Theta(1)$ amortised (Theorem 2.3's argument applies verbatim).

ArrayDeque.. A deque allows insertion and removal at either end, and the List operations at arbitrary positions. `get(i)` and `set(i,x)` access `a[(j+i)%len]`. For `add(i,x)` the implementation shifts whichever half is *shorter*: if $i < n/2$ it moves `a[j..j+i]` one cell to the *left* (and decrements j), otherwise it moves `a[j+i..j+n-1]` one cell to the *right*. Thus at most $\min(i, n - i)$ items move.

Operation	ArrayStack	ArrayQueue	ArrayDeque
`get(i)`, `set(i,x)`	$\Theta(1)$	—	$\Theta(1)$
`add(i,x)`, `remove(i)`	$\Theta(1 + n - i)$	—	$\Theta(1 + \min(i, n - i))$
add / remove at the rear	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
add / remove at the front	$\Theta(n)$	remove: $\Theta(1)$	$\Theta(1)$

Table 2.3: Array-based implementations (resizing amortised).

 Juevesano D. 2026 All rights reserved.

Chapter 3

Linked Lists

An array list is rigid: insertion in the middle moves everything behind it, and growth copies the whole array. A *linked list* gives up random access in exchange for cheap structural change. Instead of consecutive cells it uses *nodes*, each holding an item and pointers to its neighbours.

3.1 The singly linked list

Definition 3.1 (Singly linked list)**.** A node is a pair $(x, next)$. A *singly linked list* (SLL) is a pointer `head` to the first node, optionally a pointer `tail` to the last, and a counter `size`. The last node's `next` is `NULL`; the empty list has `head==NULL`.

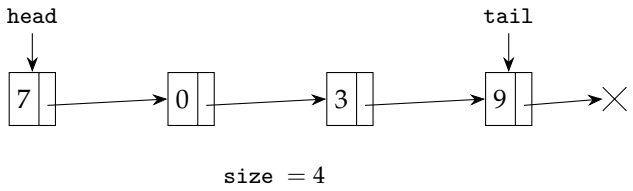


Figure 3.1: A singly linked list with head and tail pointers.

```c
typedef struct snode {
    int item;
    struct snode *next;
} snode;

typedef struct {
    snode *head;
    snode *tail;
    int size;
} sll;

void sinit(sll *l) {
    l->head = NULL;
    l->tail = NULL;
    l->size = 0;
}

// Adds x at the back. Returns 1 on success, 0 if malloc fails.
int pushBack(sll *l, int x) {
    snode *n = (snode *)malloc(sizeof(snode));
    if (n == NULL) return 0;
```

```
    n->item = x;
    n->next = NULL;
    if (l->tail == NULL) l->head = n;
    else l->tail->next = n;
    l->tail = n;
    l->size++;
    return 1;
}

// Adds x at the front. Returns 1 on success, 0 if malloc fails.
int pushFront(sll *l, int x) {
    snode *n = (snode *)malloc(sizeof(snode));
    if (n == NULL) return 0;
    n->item = x;
    n->next = l->head;
    l->head = n;
    if (l->tail == NULL) l->tail = n;
    l->size++;
    return 1;
}

// Prints the list as [1 2 3].
void sdisplay(sll *l) {
    snode *c;
    printf("[");
    for (c = l->head; c != NULL; c = c->next) {
        printf("%d", c->item);
        if (c->next != NULL) printf(" ");
    }
    printf("]\n");
}

void sdestroy(sll *l) {
    snode *c = l->head;
    while (c != NULL) {
        snode *nxt = c->next;
        free(c);
        c = nxt;
    }
    sinit(l);
}
```

Listing 3.1: An SLL with head, tail and size; `pushBack` and `pushFront` are $\Theta(1)$.

3.1.1 Insertion and removal at the ends

push_front (prepend).. Allocate u; set $u.next =$ `head`; set `head` $= u$; if the list was empty also set `tail` $= u$; increment `size`. $\Theta(1)$.

push_back (append).. Allocate u; if the list is empty set `head`=`tail`= u, otherwise `tail->next`= u and `tail`= u. $\Theta(1)$ *because the tail pointer exists*; without it, appending must walk the whole list, $\Theta(n)$.

 Juevesano D. 2026 All rights reserved.

pop_front.. Save the old head; `head = head->next`; if the list became empty set `tail=NULL`; free the old node. $\Theta(1)$.

pop_back.. The new tail is the node *before* the old tail, and a singly linked node cannot reach its predecessor: one must walk from `head` to the second-to-last node, $n-2$ steps. $\Theta(n)$.

```c
/* popFront and popBack for the SLL of Listing "sll_template" */
int popFront(sll *l, int *out) {
    snode *old;
    if (l->head == NULL) return 0;          /* underflow                    */
    old = l->head;
    *out = old->item;
    l->head = old->next;                    /* unlink                       */
    if (l->head == NULL) l->tail = NULL;    /* list became empty            */
    free(old);
    l->size--;
    return 1;
}
int popBack(sll *l, int *out) {
    snode *cur;
    if (l->head == NULL) return 0;
    *out = l->tail->item;
    if (l->head == l->tail) {               /* exactly one node             */
        free(l->head);
        l->head = l->tail = NULL;
    } else {
        cur = l->head;                      /* walk to the SECOND-TO-LAST   */
        while (cur->next != l->tail) cur = cur->next;
        free(l->tail);
        cur->next = NULL;
        l->tail = cur;
    }
    l->size--;
    return 1;
}
```

Listing 3.2: `popFront` is $\Theta(1)$; `popBack` is $\Theta(n)$.

Operation	SLL (head+tail)	Why
push_front, pop_front	$\Theta(1)$	direct access to `head`
push_back	$\Theta(1)$	direct access to `tail`
pop_back	$\Theta(n)$	predecessor of `tail` is unreachable
insert / erase at position p	$\Theta(p)$	walk to the node at $p-1$
`get(i)`, `set(i,x)`, search	$\Theta(i)$, $\Theta(n)$	no random access

Table 3.1: Costs for the singly linked list. A stack (push, pop at the front) and a FIFO queue (push at the back, pop at the front) are therefore both $\Theta(1)$.

Juevesano D. 2026 All rights reserved.

3.1.2 Case analysis: draw before you code

Almost every linked-list bug is a missed special case. When removing the node t with predecessor p (if any), the cases are:

Case	Pointer repair	Also update
t is the only node	`head=tail=NULL`	`size`
t is the head (not only)	`head=t->next`	`size`
t is the tail (not only)	`p->next=NULL`	`tail=p`, `size`
t in the middle	`p->next=t->next`	`size`

Insertion has the analogous cases: into an empty list, at the front, at the end, in the middle. A *dummy head node* (Section 3.3) removes the "head" case from the code.

3.2 The doubly linked list

Definition 3.2 (Doubly linked list)**.** Each node holds $(prev, x, next)$. A *doubly linked list* (DLL) keeps `head`, `tail` and `size`. Invariant: for every node u with $u.next \neq$ `NULL` we have $u.next.prev = u$; `head->prev` and `tail->next` are `NULL`.

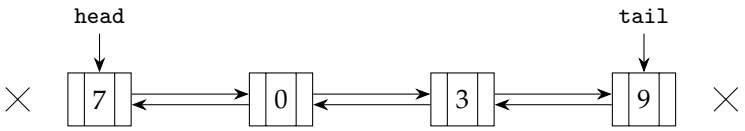


Figure 3.2: A doubly linked list.

What the extra pointer buys..

- `pop_back` becomes $\Theta(1)$: `tail = tail->prev; tail->next = NULL`.
- Given a pointer to *any* node, that node can be unlinked in $\Theta(1)$ (no walk to find its predecessor): `t->prev->next = t->next; t->next->prev = t->prev`, plus the end cases.
- Traversal works in both directions, so a node at index i can be reached from the nearer end in $\Theta(1 + \min(i, n - i))$.

The price is an extra pointer per node and *twice as many pointer updates* per operation — forgetting the backward link is the most common DLL bug.

```
typedef struct node {
    int item;
    struct node *next;
    struct node *prev;
} node;

typedef struct {
    node *head;
    node *tail;
    int size;
} list;
```

 Juevesano D. 2026 All rights reserved.

```
12
13 void init(list *l) {
14     l->head = NULL;
15     l->tail = NULL;
16     l->size = 0;
17 }
18
19 // Adds x at the tail. Returns 1 on success, 0 if malloc fails.
20 int append(list *l, int x) {
21     node *n = (node *)malloc(sizeof(node));
22     if (n == NULL) return 0;
23     n->item = x;
24     n->next = NULL;
25     n->prev = l->tail;
26     if (l->tail == NULL) l->head = n;
27     else l->tail->next = n;
28     l->tail = n;
29     l->size++;
30     return 1;
31 }
32
33 // Prints head to tail, separated by spaces.
34 void display(list *l) {
35     node *c;
36     for (c = l->head; c != NULL; c = c->next) {
37         printf("%d", c->item);
38         if (c->next != NULL) printf(" ");
39     }
40     printf("\n");
41 }
42
43 // Prints tail to head, separated by spaces.
44 void displayBackward(list *l) {
45     node *c;
46     for (c = l->tail; c != NULL; c = c->prev) {
47         printf("%d", c->item);
48         if (c->prev != NULL) printf(" ");
49     }
50     printf("\n");
51 }
52
53 void destroy(list *l) {
54     node *c = l->head;
55     while (c != NULL) {
56         node *nxt = c->next;
57         free(c);
58         c = nxt;
59     }
60     init(l);
61 }
```

Juevesano D. 2026 All rights reserved.

Listing 3.3: The doubly linked list used by the departmental exercises (`list`, `node`, `append`, `display`).

Removing a node (the four updates).. For a node `cur` in a list `l`:

```
if (cur->prev != NULL) cur->prev->next = cur->next; else l->head = cur->next;
if (cur->next != NULL) cur->next->prev = cur->prev; else l->tail = cur->prev;
free(cur);  l->size--;
```

Each `else` handles the case that `cur` is at an end. See Problem 13.

3.2.1 Sentinel (dummy) nodes

Every special case above exists because `NULL` terminates the list. If a *dummy* node that carries no item is linked into a cycle — `dummy.next` is the head, `dummy.prev` is the tail, and the empty list has `dummy.next=dummy.prev=&dummy` — then no pointer is ever `NULL` and insertion before a node w and removal of w are uniform. The notes' `DLList` is built this way, with `getNode(i)` walking from the nearer end:

```
1  // DLList with a DUMMY node (a circular list), after the course notes.
2  // dummy.next is the head, dummy.prev is the tail; an empty list points to
3  // itself, so no pointer is ever NULL and the special cases disappear.
4  struct dnode {
5      int x;
6      dnode *next, *prev;
7  };
8  class DLList {
9      dnode dummy;
10     int n;
11 public:
12     DLList() { dummy.next = &dummy; dummy.prev = &dummy; n = 0; }
13     ~DLList() { while (n > 0) remove(0); }
14     int size() { return n; }
15
16     // getNode(i): start from the nearer end.  Cost O(1 + min(i, n-i)).
17     dnode *getNode(int i) {
18         dnode *p;
19         int j;
20         if (i < n / 2) {
21             p = dummy.next;
22             for (j = 0; j < i; j++) p = p->next;
23         } else {
24             p = &dummy;
25             for (j = n; j > i; j--) p = p->prev;
26         }
27         return p;
28     }
29     int get(int i)         { return getNode(i)->x; }
30     int set(int i, int x) {
31         dnode *u = getNode(i);
32         int y = u->x;
33         u->x = x;
```

 Juevesano D. 2026 All rights reserved.

```
34          return y;
35      }
36
37      // addBefore(w, x): link a new node u between w->prev and w.  O(1).
38      dnode *addBefore(dnode *w, int x) {
39          dnode *u = (dnode *)malloc(sizeof(dnode));
40          u->x = x;
41          u->prev = w->prev;
42          u->next = w;
43          u->next->prev = u;
44          u->prev->next = u;
45          n++;
46          return u;
47      }
48      void add(int i, int x) { addBefore(getNode(i), x); }     // i in 0..n
49
50      // remove(w): make the neighbours skip w.  O(1) once w is known.
51      void remove(dnode *w) {
52          w->prev->next = w->next;
53          w->next->prev = w->prev;
54          free(w);
55          n--;
56      }
57      int remove(int i) {
58          dnode *w = getNode(i);
59          int x = w->x;
60          remove(w);
61          return x;
62      }
63  };
```

Listing 3.4: `DLList`: a doubly linked list with a dummy node.

All of `get`, `set`, `add` and `remove` cost $\Theta(1 + \min(i, n - i))$; the pointer surgery itself is $\Theta(1)$ once the node is found.

3.3 Programming patterns

Erase while iterating.. Never advance with `cur = cur->next` *after* freeing `cur`. Save `nxt = cur->next` *first*, then free, then `cur = nxt`.

Dummy head.. For a singly linked list, create a local node `dummy` with `dummy.next = head`. The code that reaches "the node before the target" then works even when the target is the head. At the end set `head = dummy.next` (and repair `tail` if the removal could have reached the end).

Two pointers at different speeds or gaps..

- *Middle*: `slow` moves one node, `fast` two, per step; when `fast` falls off the end, `slow` is at the middle. One pass.
- *k-th node from the end*: move `lead` k nodes ahead of `lag`, then advance both until `lead` is `NULL`.

Juevesano D. 2026 All rights reserved.

- *Cycle detection (Floyd)*: if the list has a cycle `fast` laps `slow`; they meet. Restarting one pointer at `head` and advancing both by one meets at the first cycle node.

Reversal.. The invariant of the three-pointer loop is: the nodes before `cur` already form a reversed list headed by `prev`.

Recursion on lists.. A list is either empty or a node followed by a list, so recursive procedures mirror the definition. To *change* the list structurally with recursion, pass the address of the pointer that must change: `void append(node **h, int x)`. Then `*h` is "the pointer that currently points at the rest of the list"; the base case `*h==NULL` stores the new node *into that pointer*, and the recursive case passes `&((*h)->next)`. This is exactly why the double pointer is needed — a single `node *h` would be a copy (Tracing 7).

Copying: shallow versus deep.. Assigning one list *struct* to another copies `head`, `tail`, `size` but not the nodes: both variables then share every node, and freeing one invalidates the other. A *deep copy* allocates a new node for every item. A routine such as `appendList(l, m)` that must survive the later destruction of `m` has to deep copy (Problem 14).

Moving versus copying; splicing.. If the donor list is allowed to give up its nodes, the nodes can simply be *relinked*: with a tail pointer, appending a whole list to another costs $\Theta(1)$ (four pointer assignments), and inserting it after a known node is $\Theta(1)$. This is the list *splice* (Problem 18); it is the operation that arrays can never do in constant time.

Augmenting the list.. Extra bookkeeping pointers (such as a pointer to the middle node) make some queries $\Theta(1)$ at the price of keeping the extra pointer correct in *every* update. The design question is always: how does this operation change the index of the node the extra pointer must designate? Problem 24 works this out in full.

3.4 Arrays versus linked lists

Operation	Array list	SLL (head+tail)	DLL
`get(i)`	$\Theta(1)$	$\Theta(i)$	$\Theta(\min(i, n-i))$
insert / erase at front	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
insert / erase at rear	$\Theta(1)$	$\Theta(1)$ / $\Theta(n)$	$\Theta(1)$
insert / erase after a known node	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
erase a known node	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
splice one list into another	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
memory per item	item	item + 1 pointer	item + 2 pointers
cache behaviour	contiguous	scattered	scattered

Table 3.2: When position matters, choose the array; when structure changes matter, choose the linked list.

 Juevesano D. 2026 All rights reserved.

Chapter 4

Stacks and Queues

Stacks and queues are lists with *restricted access*. The restriction is the point: a smaller interface is easier to reason about, easier to implement in $\Theta(1)$ time, and it states the programmer's intent.

4.1 The abstract data types

Definition 4.1 (Stack)**.** A *stack* is a sequence of items accessed at one end only, the *top*. Operations: `push(x)` adds x on top; `pop()` removes and returns the top; `top()` (or `peek`) returns the top without removing it; `empty()`, `size()`. The discipline is *last in, first out* (LIFO).

Definition 4.2 (Queue)**.** A *queue* is a sequence in which items are added at the *back* and removed from the *front*. Operations: `enqueue(x)` (`push`), `dequeue()` (`pop`), `front()`, `empty()`, `size()`. The discipline is *first in, first out* (FIFO).

Definition 4.3 (Deque)**.** A *double-ended queue* supports insertion and removal at both ends. It subsumes both the stack and the queue.

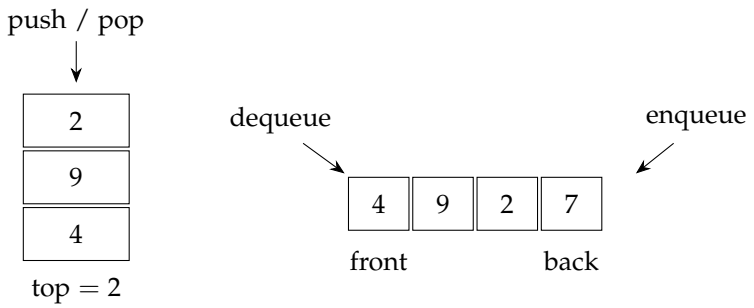


Figure 4.1: A stack (left) and a queue (right).

> Abstraction. When a problem says "use only `push`, `pop`, `top`, `empty`" you may *not* look inside the stack, index into it, or iterate over it. To inspect elements below the top you must pop them and (if they must survive) push them onto another stack. The same holds for queues: you see only the front.

4.2 Implementations

4.2.1 Array-based

A stack is trivial on an array: the items are `a[0..n-1]`, the top is `a[n-1]`, and both `push` and `pop` touch only the end, so both are $\Theta(1)$ (amortised $\Theta(1)$ if the array doubles; see Theorem 2.3).

Juevesano D. 2026 All rights reserved.

A queue on an array needs care. If the front is always `a[0]`, each `dequeue` shifts $n-1$ items: $\Theta(n)$. The cure is a *circular buffer*: keep the index `j` of the front and the count `n`; item i lives at `a[(j+i) % CAP]`. Then

$$\begin{aligned}\text{enqueue}: &\ a[(j+n) \bmod CAP] \leftarrow x, \quad n \leftarrow n+1;\\ \text{dequeue}: &\ x \leftarrow a[j], \quad j \leftarrow (j+1) \bmod CAP, \quad n \leftarrow n-1,\end{aligned}$$

both $\Theta(1)$.

```c
typedef struct {
    int items[100];
    int size;           // the top is items[size - 1]
} stack;

void stInit(stack *s) { s->size = 0; }
int stEmpty(stack *s) { return s->size == 0; }
int stSize(stack *s) { return s->size; }

// Returns 1 on success, 0 if the stack is full.
int stPush(stack *s, int x) {
    if (s->size == 100) return 0;
    s->items[s->size] = x;
    s->size++;
    return 1;
}

// Removes the top into *x. Returns 1 on success, 0 if the stack is empty.
int stPop(stack *s, int *x) {
    if (s->size == 0) return 0;
    s->size--;
    *x = s->items[s->size];
    return 1;
}

// Copies the top into *x without removing it. Returns 0 if empty.
int stTop(stack *s, int *x) {
    if (s->size == 0) return 0;
    *x = s->items[s->size - 1];
    return 1;
}

typedef struct {
    int items[100];
    int front;          // index of the first item
    int size;
} queue;                // circular: item i is at items[(front + i) % 100]

void qInit(queue *q) { q->front = 0; q->size = 0; }
int qEmpty(queue *q) { return q->size == 0; }
int qSize(queue *q) { return q->size; }

// Returns 1 on success, 0 if the queue is full.
```

 Juevesano D. 2026 All rights reserved.

```c
int qEnq(queue *q, int x) {
    if (q->size == 100) return 0;
    q->items[(q->front + q->size) % 100] = x;
    q->size++;
    return 1;
}

// Removes the front into *x. Returns 1 on success, 0 if the queue is empty.
int qDeq(queue *q, int *x) {
    if (q->size == 0) return 0;
    *x = q->items[q->front];
    q->front = (q->front + 1) % 100;
    q->size--;
    return 1;
}

// Copies the front into *x without removing it. Returns 0 if empty.
int qFront(queue *q, int *x) {
    if (q->size == 0) return 0;
    *x = q->items[q->front];
    return 1;
}
```

Listing 4.1: Array-backed stack and circular queue (capacity 100).

Remark 4.4. The pair (j, n) rather than $(\text{front}, \text{rear})$ avoids the classical ambiguity "is *front* = *rear* empty or full?". With a count the answers are $n = 0$ and $n = CAP$. Tracing 5 exercises the wrap-around.

4.2.2 Linked

A *stack* on a singly linked list pushes and pops at the `head`: both $\Theta(1)$ and there is no capacity. A *queue* enqueues at the `tail` and dequeues from the `head`: both $\Theta(1)$ *provided a tail pointer is kept*. The reverse choice (enqueue at the head, dequeue at the tail) makes `dequeue` $\Theta(n)$ in a singly linked list, as seen in Chapter 3.

	Array (fixed)	Array (doubling)	Linked list
`push/enqueue`	$\Theta(1)$	$\Theta(1)$ amortised	$\Theta(1)$
`pop/dequeue`	$\Theta(1)$	$\Theta(1)$ amortised	$\Theta(1)$
overflow	possible	no	no
memory	CAP cells	between n and $3n$ cells	n nodes + n pointers

Table 4.1: Stack and queue implementations all give constant-time operations; they differ in the constants and in memory.

4.3 Applications of stacks

4.3.1 Balanced brackets

The nesting discipline of brackets is LIFO: the most recently opened bracket must be the next to close. Scan the string; push each opener; on a closer, pop and compare. The string is balanced iff no mismatch occurs and the stack is empty at the end.

Juevesano D. 2026 All rights reserved.

Proposition 4.5. *The algorithm is correct and runs in $\Theta(n)$ time and $O(n)$ space.*

Proof. Invariant: after reading a prefix, the stack holds exactly the openers of the prefix that are still unmatched, in order of appearance. A closer is legal iff the top of that list is its partner; if the stack is empty the closer has no partner. Each character causes at most one push or one pop. ■

```
// Returns 1 if every bracket in s is closed by the same kind of bracket,
// in the right order.
// s contains only the characters ( ) [ ] { }.
int isBalanced(char s[]) {
    stack st;
    stInit(&st);

    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') {
            stPush(&st, c);                  // remember the opener
        } else {
            int open;
            if (!stPop(&st, &open)) return 0; // closer with nothing to close
            if (c == ')' && open != '(') return 0;
            if (c == ']' && open != '[') return 0;
            if (c == '}' && open != '{') return 0;
        }
    }
    return stEmpty(&st);                     // leftover openers: not balanced
}
```

Listing 4.2: Bracket matching (Problem 32).

4.3.2 *Evaluation of expressions*

Humans write *infix* expressions `a+b*c`, which need precedence and parentheses. *Postfix* (reverse Polish) writes operators after their operands — `abc*+` — and needs neither.

Evaluating postfix.. Scan left to right: push an operand; on an operator pop b then a (the *right* operand is on top), push $a \,\mathrm{op}\, b$. At the end the stack holds the value. $\Theta(n)$.

Infix to postfix (shunting yard).. Operands go straight to the output. For an operator o: while the stack top is an operator of higher precedence, *or* of equal precedence and o is left-associative, pop it to the output; then push o. A `(` is pushed; a `)` pops to the output until the matching `(` is removed. At the end pop everything.

Infix	Postfix	Note
`a+b*c`	`abc*+`	`*` binds tighter
`(a+b)*c`	`ab+c*`	parentheses override
`a-b-c`	`ab-c-`	equal precedence, left-associative
`a+b*(c-d)/e`	`abcd-*e/+`	used in Tracing 11

 Juevesano D. 2026 All rights reserved.

```
// Precedence of an operator: * and / bind tighter than + and -.
int prec(char op) {
    if (op == '*' || op == '/') return 2;
    return 1;
}

// Converts infix to postfix. Operands are single digits; operators are
// + - * / and parentheses; there are no spaces. "2+3*4" -> "234*+".
void toPostfix(char in[], char out[]) {
    stack ops;                      // operators waiting for their operands
    int k = 0;                      // next free place in out
    stInit(&ops);

    for (int i = 0; in[i] != '\0'; i++) {
        char c = in[i];
        if (c >= '0' && c <= '9') {
            out[k++] = c;           // operands go straight to the output
        } else if (c == '(') {
            stPush(&ops, c);
        } else if (c == ')') {
            int top;                // pop until the matching '('
            while (stPop(&ops, &top) && top != '(') out[k++] = top;
        } else {
            int top;                // pop operators that must run first
            while (stTop(&ops, &top) && top != '(' && prec(top) >= prec(c)) {
                stPop(&ops, &top);
                out[k++] = top;
            }
            stPush(&ops, c);
        }
    }
    int top;
    while (stPop(&ops, &top)) out[k++] = top;
    out[k] = '\0';
}

// Evaluates a postfix expression of single digits and + - * /
// (integer division).
// "234*+" -> 14. The input is always valid.
int evalPostfix(char p[]) {
    stack st;
    stInit(&st);

    for (int i = 0; p[i] != '\0'; i++) {
        char c = p[i];
        if (c >= '0' && c <= '9') {
            stPush(&st, c - '0');
        } else {
            int b, a;               // b was pushed last: the right side
            stPop(&st, &b);
            stPop(&st, &a);
            if (c == '+') stPush(&st, a + b);
```

Juevesano D. 2026 All rights reserved.

```
            if (c == '-') stPush(&st, a - b);
            if (c == '*') stPush(&st, a * b);
            if (c == '/') stPush(&st, a / b);
        }
    }
    int result;
    stPop(&st, &result);
    return result;
}
```

Listing 4.3: Shunting-yard conversion and postfix evaluation (Problem 33).

4.3.3 Other recurring uses

- *Reversal*: pushing then popping reverses a sequence.
- *Backtracking and undo*: the stack records the path so far.
- *The call stack*: recursion *is* a stack of activation records; any recursive algorithm can be made iterative with an explicit stack (iterative tree traversal, Chapter 5).
- *Monotonic stack*: keep the stack increasing (or decreasing); popping smaller elements when a larger one arrives answers "next greater element" in $\Theta(n)$ total, because every item is pushed once and popped at most once (Problem 36).
- *Auxiliary information*: a second stack storing the running minimum yields `min()` in $\Theta(1)$ (Problem 35).

```
// out[i] = the first value to the right of a[i] that is greater than a[i],
// or -1 if there is none.  [2 1 5 3] -> [5 5 -1 -1]
// The stack holds indices whose answer is still unknown; their values
// decrease from bottom to top.
void nextGreater(int a[], int n, int out[]) {
    stack waiting;
    stInit(&waiting);

    for (int i = 0; i < n; i++) out[i] = -1;

    for (int i = 0; i < n; i++) {
        int j;
        // a[i] answers every waiting index whose value is smaller
        while (stTop(&waiting, &j) && a[j] < a[i]) {
            out[j] = a[i];
            stPop(&waiting, &j);
        }
        stPush(&waiting, i);
    }
}
```

Listing 4.4: Next greater element with a monotonic stack: each index is pushed and popped at most once.

 Juevesano D. 2026 All rights reserved.

4.4 Applications of queues

Queues model fairness and order of arrival: print jobs, task scheduling, buffering, and *breadth-first* processing (the level-order traversal of a tree, Chapter 5). The *Josephus problem* — n people in a circle, every k-th is eliminated — is a direct simulation: dequeue $k-1$ people and re-enqueue them, then discard the next one. This takes $\Theta(nk)$ operations; a closed recurrence $J(1) = 0$, $J(m) = (J(m-1) + k) \bmod m$ gives $\Theta(n)$ (Problem 37).

```c
// n people stand in a circle, numbered 1..n. Starting at 1, every k-th
// person is removed until one is left. Returns that person's number.
int josephus(int n, int k) {
    queue q;
    int x;
    qInit(&q);
    for (int i = 1; i <= n; i++) qEnq(&q, i);

    while (qSize(&q) > 1) {
        for (int i = 1; i < k; i++) {    // the first k-1 are skipped:
            qDeq(&q, &x);                // they go to the back
            qEnq(&q, x);
        }
        qDeq(&q, &x);                    // the k-th is removed
    }
    qDeq(&q, &x);
    return x;
}

// Bonus, O(n) time and O(1) space. With i people the survivor's 0-based
// position is (survivor with i-1 people + k) % i.
int josephusFast(int n, int k) {
    int pos = 0;                         // one person: position 0
    for (int i = 2; i <= n; i++) pos = (pos + k) % i;
    return pos + 1;
}
```

Listing 4.5: Josephus by queue simulation and by recurrence.

4.5 Converting between the two

A queue from two stacks.. Keep an `in` stack and an `out` stack. `enqueue` pushes onto `in`. `dequeue` pops from `out`; if `out` is empty, first move *all* of `in` to `out` (reversing it, so the oldest item surfaces).

Theorem 4.6. *Starting from empty, any sequence of m operations on the two-stack queue performs $O(m)$ stack operations; each operation is $\Theta(1)$ amortised.*

Proof. Charge three units for an `enqueue`: one for pushing onto `in`, one for the later pop from `in` and one for the later push onto `out`. The transfer then pays for itself, because it only moves items that carry prepaid credit. An item is moved at most once, since after the transfer it is on `out` and leaves from there. A `dequeue` costs one pop from `out` (plus prepaid transfer work). Hence the total is at most $3m$. ■

Juevesano D. 2026 All rights reserved.

A single dequeue can cost $\Theta(n)$, so the bound is amortised, not worst-case (Tracing 12).

```
// A queue built from two stacks. Items enter through "in" and leave through
// "out"; moving everything from in to out reverses the order, which turns
// last-in-first-out into first-in-first-out.
typedef struct {
    stack in;
    stack out;
} twoStackQueue;

void tsqInit(twoStackQueue *q) {
    stInit(&q->in);
    stInit(&q->out);
}

void tsqPush(twoStackQueue *q, int x) {
    stPush(&q->in, x);
}

// Moves items only when out is empty, so each item moves at most once.
void shift(twoStackQueue *q) {
    if (stEmpty(&q->out)) {
        int x;
        while (stPop(&q->in, &x)) stPush(&q->out, x);
    }
}

// Removes the front into *x. Returns 0 if the queue is empty.
int tsqPop(twoStackQueue *q, int *x) {
    shift(q);
    return stPop(&q->out, x);
}

// Copies the front into *x without removing it. Returns 0 if empty.
int tsqPeek(twoStackQueue *q, int *x) {
    shift(q);
    return stTop(&q->out, x);
}

int tsqEmpty(twoStackQueue *q) {
    return stEmpty(&q->in) && stEmpty(&q->out);
}
```

Listing 4.6: A queue built from two stacks (the *only* stack operations used are push, pop, top, empty).

A stack from queue(s).. With one queue: to push x, enqueue it and then rotate the previous n items behind it (dequeue and re-enqueue each); the newest item is then at the front. `push` is $\Theta(n)$, `pop` is $\Theta(1)$ (Problem 41).

Minimum stack.. Pair each pushed value with the minimum of the stack at that moment, or use a second stack of minima. Then `push`, `pop`, `top` and `min` are all $\Theta(1)$.

 Juevesano D. 2026 All rights reserved.

```c
// A stack that also returns its minimum in O(1).
// "mins" holds, for every level of "all", the smallest value at or below it.
typedef struct {
    stack all;
    stack mins;
} minStack;

void msInit(minStack *m) {
    stInit(&m->all);
    stInit(&m->mins);
}

void msPush(minStack *m, int x) {
    int smallest;
    stPush(&m->all, x);
    if (stTop(&m->mins, &smallest) && smallest < x) {
        stPush(&m->mins, smallest);       // the old minimum still wins
    } else {
        stPush(&m->mins, x);              // x is the new minimum
    }
}

// Removes the top into *x. Returns 0 if empty.
int msPop(minStack *m, int *x) {
    int dummy;
    if (!stPop(&m->all, x)) return 0;
    stPop(&m->mins, &dummy);              // keep the two stacks in step
    return 1;
}

// Copies the minimum into *x. Returns 0 if empty.
int msMin(minStack *m, int *x) {
    return stTop(&m->mins, x);
}
```

Listing 4.7: A stack with constant-time minimum.

4.6 Deque

A deque (double-ended queue) can be implemented with a circular array or a doubly linked list; all four end operations (add and remove at either end) are $\Theta(1)$, amortised for the array. It is not needed for the problems of this coverage, but it is the natural structure whenever items must be added or removed at both ends.

Juevesano D. 2026 All rights reserved.

Task	Structure	Time
matching brackets	stack	$\Theta(n)$
evaluate postfix	stack	$\Theta(n)$
infix to postfix	stack	$\Theta(n)$
next greater element	monotonic stack	$\Theta(n)$
queue from stacks	two stacks	$\Theta(1)$ amortised
Josephus	queue / recurrence	$\Theta(nk)$ / $\Theta(n)$
stack from a queue	one queue	push $\Theta(n)$, pop $\Theta(1)$

Table 4.2: Standard tasks and the structure that gives their best time.

 Juevesano D. 2026 All rights reserved.

Chapter 5

Binary Trees and Binary Search Trees

Lists impose a linear order. A *tree* imposes a hierarchy: file systems, expression syntax, organisation charts, decision procedures. Our coverage ends with the binary search tree; self-balancing variants are outside it.

5.1 Terminology

Definition 5.1 (Binary tree)**.** A *binary tree* is either empty, or a *root* node together with an ordered pair of binary trees, its *left subtree* and *right subtree*.

The definition is recursive, and so are nearly all tree algorithms: do the work at the root, then ask the subtrees to do the same.

Term	Meaning
parent, child	v is a child of u if v is the root of a subtree of u; u is then the parent of v
root	the unique node without a parent
leaf	a node with no children; a node with at least one child is *internal*
siblings	nodes with the same parent
ancestor, descendant	u is an ancestor of v if u lies on the path from the root to v (a node is its own ancestor here)
depth of v	number of *edges* from the root to v (root: 0)
height of v	number of edges on a longest downward path from v to a leaf (a leaf: 0)
height of a tree	the height of its root; the empty tree has height -1
level d	the set of all nodes of depth d
subtree at v	v together with all its descendants
degree	the number of children of a node (at most 2 here)

Definition 5.2. A binary tree is *full* (or *proper*) if every node has 0 or 2 children; *perfect* if it is full and all leaves have the same depth; *complete* if every level except possibly the last is completely filled and the last is filled from the left.

5.2 Counting facts

Theorem 5.3. *A binary tree with $n \geq 1$ nodes has exactly $n - 1$ edges.*

Proof. Every node except the root has exactly one parent edge, and every edge is the parent edge of exactly one node. ∎

Theorem 5.4. *Level d of a binary tree has at most 2^d nodes. A binary tree of height h has at most $2^{h+1} - 1$ nodes, and at least $h + 1$.*

Juevesano D. 2026 All rights reserved.

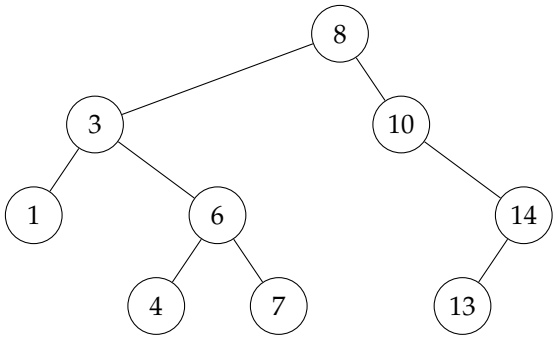


Figure 5.1: The tree S_1 obtained by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST. It has 9 nodes, height 3, 4 leaves $(1, 4, 7, 13)$; node 6 has depth 2 and height 1.

Proof. Induction on d: level 0 has 1 node; each node of level d has at most 2 children, which make up level $d+1$. Summing, $\sum_{d=0}^{h} 2^d = 2^{h+1} - 1$. A tree of height h contains a path of $h+1$ nodes. ■

Corollary 5.5. *For $n \geq 1$ nodes, $\lceil \lg(n+1) \rceil - 1 \leq h \leq n-1$. Both extremes occur: the perfect tree (about $\lg n$) and the "list" where every node has one child ($n-1$).*

Proof. From $n \leq 2^{h+1} - 1$ we get $h \geq \lg(n+1) - 1$, and h is an integer. The upper bound is the path of n nodes. ■

Theorem 5.6. *Let n_0 be the number of leaves and n_2 the number of nodes with exactly two children. Then $n_0 = n_2 + 1$.*

Proof. Let n_1 count nodes with one child. Then $n = n_0 + n_1 + n_2$. Counting edges by their parent: $n - 1 = n_1 + 2n_2$ (Theorem 5.3). Subtract: $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2 \Rightarrow n_0 = n_2 + 1$. ■

5.3 Representation and recursion

The course's node holds an item, links to the left and right children, and (in the sample programs) a link to the parent:

```
using namespace std;

class node {
public:
    int item;
    node *left;
    node *right;
    node *parent;
    node(int x) {
        item = x;
        left = NULL;
        right = NULL;
        parent = NULL;
    }
};

```

 Juevesano D. 2026 All rights reserved.

```
// BST insert: keys smaller than a node go left, all others go right.
void insert(node **root, int x) {
    node *n = new node(x);
    node *cur = *root;
    node *prev = NULL;
    if (cur == NULL) {
        *root = n;
        return;
    }
    while (cur != NULL) {
        prev = cur;
        if (x < cur->item) cur = cur->left;
        else cur = cur->right;
    }
    if (x < prev->item) prev->left = n;
    else prev->right = n;
    n->parent = prev;
}

// Plain queue of node pointers (used for level-order work).
class nqueue {
    node *items[200];
    int front;
    int count;
public:
    nqueue() { front = 0; count = 0; }
    bool empty() { return count == 0; }
    int size() { return count; }
    void push(node *x) {
        items[(front + count) % 200] = x;
        count++;
    }
    node *pop() {
        node *x = items[front];
        front = (front + 1) % 200;
        count--;
        return x;
    }
};

// Plain stack of node pointers (used to avoid recursion).
class nstack {
    node *items[200];
    int count;
public:
    nstack() { count = 0; }
    bool empty() { return count == 0; }
    void push(node *x) { items[count++] = x; }
    node *pop() { return items[--count]; }
    node *top() { return items[count - 1]; }
};

void preOrder(node *r) {
```

Juevesano D. 2026 All rights reserved.

```
70      if (r == NULL) return;
71      cout << r->item << " ";
72      preOrder(r->left);
73      preOrder(r->right);
74  }
75
76  void inOrder(node *r) {
77      if (r == NULL) return;
78      inOrder(r->left);
79      cout << r->item << " ";
80      inOrder(r->right);
81  }
82
83  void postOrder(node *r) {
84      if (r == NULL) return;
85      postOrder(r->left);
86      postOrder(r->right);
87      cout << r->item << " ";
88  }
89
90  // Prints the items level by level on one line (breadth-first).
91  void levelOrder(node *r) {
92      if (r == NULL) return;
93      nqueue q;
94      q.push(r);
95      while (!q.empty()) {
96          node *d = q.pop();
97          cout << d->item << " ";
98          if (d->left != NULL) q.push(d->left);
99          if (d->right != NULL) q.push(d->right);
100     }
101     cout << endl;
102 }
103
104 // Fills the first vacant place in level order (builds a complete tree).
105 void insertLevel(node **root, int x) {
106     node *n = new node(x);
107     if (*root == NULL) {
108         *root = n;
109         return;
110     }
111     nqueue q;
112     q.push(*root);
113     while (!q.empty()) {
114         node *d = q.pop();
115         if (d->left == NULL) {
116             d->left = n;
117             n->parent = d;
118             return;
119         }
120         q.push(d->left);
121         if (d->right == NULL) {
122             d->right = n;
```

 Juevesano D. 2026 All rights reserved.

```
            n->parent = d;
            return;
        }
        q.push(d->right);
    }
}

// BST search: true if x is in the tree.
bool search(node *r, int x) {
    while (r != NULL) {
        if (x == r->item) return true;
        if (x < r->item) r = r->left;
        else r = r->right;
    }
    return false;
}
```

Listing 5.1: The `node` class, the course's traversals, level-order, BST `insert` and level-order `insertLevel`. Small queue and stack classes of node pointers replace the library containers.

> PARENT POINTERS. The parent link makes successor, predecessor and "go up" operations easy, but *every* structural update must also repair it: when a child is attached, set `child->parent`. A forgotten parent link is silent until the tree is later walked upward (Tracing 17).

5.3.1 The recursive pattern

Because a tree is a root plus two trees, a function on trees has the shape

```
T f(node *r) {
    if (r == NULL) return BASE;            // empty tree
    return COMBINE(r->item, f(r->left), f(r->right));
}
```

Examples (the first two are in Listing 5.2): `count`, with BASE $= 0$ and COMBINE $= 1 + \ell + r$; `height`, with BASE $= -1$ and COMBINE $= 1 + \max(\ell, r)$; the number of leaves, the sum of items, the mirror image.

```
// Number of nodes. An empty tree has 0.
int countNodes(node *r) {
    if (r == NULL) return 0;
    return 1 + countNodes(r->left) + countNodes(r->right);
}

// Height in edges: a leaf has height 0 and the empty tree has height -1.
int height(node *r) {
    if (r == NULL) return -1;
    int lh = height(r->left);
    int rh = height(r->right);
    if (lh > rh) return 1 + lh;
    return 1 + rh;
}

```

Juevesano D. 2026 All rights reserved.

```
// Number of leaves (nodes without children).
int countLeaves(node *r) {
    if (r == NULL) return 0;
    if (r->left == NULL && r->right == NULL) return 1;
    return countLeaves(r->left) + countLeaves(r->right);
}

// Sum of all items.
int sumItems(node *r) {
    if (r == NULL) return 0;
    return r->item + sumItems(r->left) + sumItems(r->right);
}
```

Listing 5.2: Number of nodes, height, number of leaves, and sum of the items.

Proposition 5.7. *Each of these recursive functions visits every node once and runs in $\Theta(n)$ time. The recursion depth equals the height of the tree, so the extra space is $\Theta(h)$.*

Proof. The function is called exactly once on every node (as a non-empty call) and once on each missing child, i.e. n plus $n+1$ calls in total, each doing $\Theta(1)$ work. The call stack holds one frame per level of the path being explored. ■

The space bound is why a degenerate tree of 10^5 nodes can overflow the call stack: $h = n - 1$.

5.4 Traversals

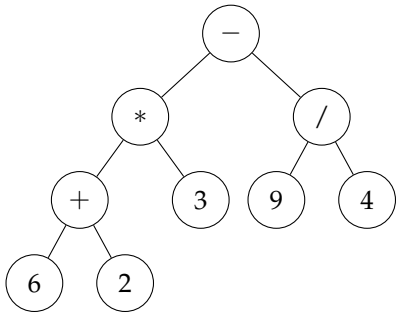
A *traversal* visits every node exactly once. The three depth-first orders differ only in *when* the root is visited:

Order	Rule	Result on S_1
preorder	root, left, right	8 3 1 6 4 7 10 14 13
inorder	left, root, right	1 3 4 6 7 8 10 13 14
postorder	left, right, root	1 4 7 6 3 13 14 10 8
level order	level by level, left to right	8 3 10 1 6 14 4 7 13

Depth-first orders use recursion (a stack); *level order* uses a *queue*: enqueue the root; repeatedly dequeue a node, visit it, and enqueue its children. All four take $\Theta(n)$ time.

5.4.1 Expression trees

An arithmetic expression is a tree: leaves are operands, internal nodes are operators, the children are the operands of the operator.



 Juevesano D. 2026 All rights reserved.

Preorder gives the *prefix* form `-*+623/94`; postorder the *postfix* form `62+3*94/-`; inorder, with parentheses added by the printer, the infix form `(((6+2)*3)-(9/4))`; with integer division this evaluates to $24 - 2 = 22$. Evaluating the tree is a postorder computation: the value of a node needs the values of both children first (Tracing 25).

5.4.2 *Iterative inorder*

Recursion is a hidden stack; an explicit stack removes it. Push the left spine; pop a node, visit it, then walk the left spine of its right child.

```
// Prints the items in inorder without recursion.
void inorderIterative(node *root) {
    nstack st;
    node *cur = root;

    while (cur != NULL || !st.empty()) {
        while (cur != NULL) {          // save the path down the left side
            st.push(cur);
            cur = cur->left;
        }
        cur = st.pop();
        cout << cur->item << " ";      // visit
        cur = cur->right;              // then do the right subtree
    }
}
```

Listing 5.3: Inorder traversal without recursion (Problem 51).

5.4.3 *Reconstruction*

Theorem 5.8. *Preorder together with inorder (of a tree with distinct items) determines the tree uniquely; so do postorder together with inorder. Preorder together with postorder does* not *in general.*

Proof. Preorder begins with the root r. In the inorder sequence the items left of r form the left subtree, those right of r the right subtree; their sizes tell which following block of preorder belongs to which subtree. Recurse. For the negative claim, the trees with a single child on the left or on the right both have preorder (r, c) and postorder (c, r). ■

5.5 Binary search trees

Definition 5.9 (Binary search tree)**.** A *binary search tree* (BST) is a binary tree whose nodes hold keys such that for every node v, every key in the left subtree of v is $<$ the key of v, and every key in the right subtree is $\geq$ the key of v. (In the course code, equal keys go to the right.)

> The property is global. It is *not* enough that each node is larger than its left child and smaller than its right child. The tree with root 5, left child 3 and 3's right child 8 passes every local test but violates the property, since $8 > 5$ lies in the left subtree of 5. A correct test passes lower and upper bounds down the recursion (Problem 44).

Theorem 5.10. *An inorder traversal of a BST lists its keys in non-decreasing order.*

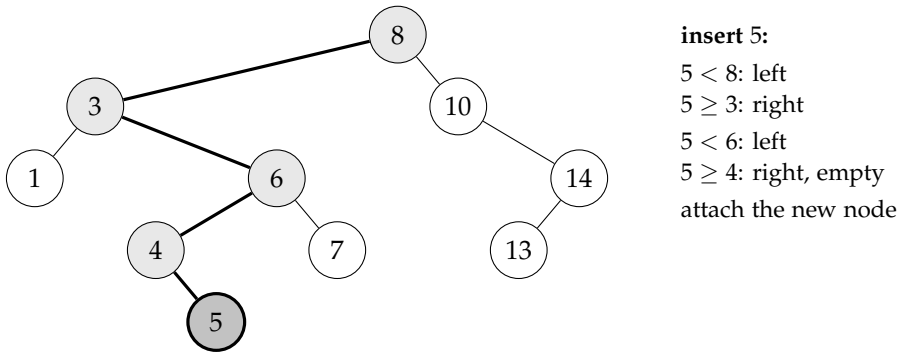
Juevesano D. 2026 All rights reserved.

Proof. Induction on size: the inorder listing is (sorted left subtree), root, (sorted right subtree); by the BST property everything in the first block is $<$ root $\leq$ everything in the last. ■

5.5.1 Search and insertion

Search for x: if the tree is empty, fail; if x equals the root key, succeed; if x is smaller go left, otherwise right. **Insertion** follows the same search path until it falls off the tree, and attaches the new node as a leaf there. In both, the work is proportional to the length of one root-to-node path: $O(h)$.

Operation	Best / typical	Worst case	Worst-case tree
search, insert, remove	$\Theta(\log n)$ (balanced)	$\Theta(n)$	keys inserted in sorted order
min, max	$\Theta(h)$	$\Theta(n)$	
inorder traversal	$\Theta(n)$	$\Theta(n)$	



Insertion into S_1: follow the search path until an empty position is reached and put the new node there. The key 5 *becomes the right child of* 4 *(equal keys would also go right).*

The shape depends on the order of insertion.. The keys $1, 2, 3, 4, 5$ inserted in this order give a path of height 4; the order $3, 1, 5, 2, 4$ gives height 2. Inserting keys in sorted order is the worst case: every insertion goes right (Tracing 14). Keys inserted in a random order give expected height $O(\log n)$; guaranteeing it needs balancing, which lies beyond this coverage.

5.5.2 Removal

Removing a node t (found by search) has three cases:

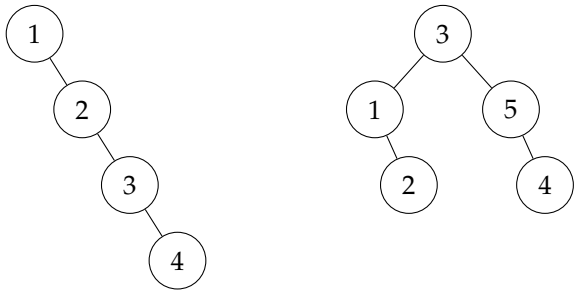
(i) t **is a leaf**: detach it (set the parent's link to `NULL`).

(ii) t **has one child**: replace t by that child (the parent's link now points to the child).

(iii) t **has two children**: find the *inorder successor* s of t — the minimum of t's right subtree, a node with no left child. Copy s's key into t, then remove s from the right subtree; this removal is case 1 or 2 (alternatively use the predecessor, the maximum of the left subtree).

Proposition 5.11. *Removal preserves the BST property.*

 Juevesano D. 2026 All rights reserved.



sorted insertion: $h = n - 1$ balanced: $h \approx \lg n$

Figure 5.2: The same keys, two shapes.

Proof. Cases 1 and 2 only delete a node or lift a whole subtree by one level: relative order is unchanged. In case 3 the successor key is $\geq$ every key of the left subtree and $\leq$ every other key of the right subtree (it is the smallest of the right subtree). Placing it at t keeps both inequalities, and removing it from the right subtree is a case-1/2 removal. ■

```
// Removes one node with item x. Returns 1 if found, 0 otherwise.
int removeKey(node **root, int x) {
    // 1. find the node
    node *cur = *root;
    while (cur != NULL && cur->item != x) {
        if (x < cur->item) cur = cur->left;
        else cur = cur->right;
    }
    if (cur == NULL) return 0;

    // 2. two children: copy the successor's item here, remove the successor
    if (cur->left != NULL && cur->right != NULL) {
        node *s = cur->right;
        while (s->left != NULL) s = s->left;
        cur->item = s->item;
        cur = s;                        // s has no left child
    }

    // 3. cur has at most one child; that child takes its place
    node *child = cur->left;
    if (child == NULL) child = cur->right;
    if (child != NULL) child->parent = cur->parent;

    if (cur->parent == NULL) *root = child;
    else if (cur->parent->left == cur) cur->parent->left = child;
    else cur->parent->right = child;

    delete cur;
    return 1;
}
```

Listing 5.4: BST removal by the three cases.

Juevesano D. 2026 All rights reserved.

Complete C++ code for search, minimum, maximum, the four insertion styles and removal, traced on a 16-node tree, is in Appendix C.

5.5.3 Order-based queries

Because inorder is sorted, many questions reduce to a guided walk:

- the k-th smallest key: inorder with a counter, stopping at k (Problem 46);
- successor and predecessor of a node (Problem 45): the successor is the leftmost node of the right subtree if there is one; otherwise the lowest ancestor whose *left* subtree contains the node (found with parent pointers);
- lowest common ancestor: walk down from the root; the first node with $a \leq v \leq b$ is the LCA (Problem 47);
- range counting, floor and ceiling: prune subtrees that cannot contain answers (Problem 50);
- building a balanced BST from a sorted array: take the middle as root, recurse (Problem 49), giving height $\lfloor \lg n \rfloor$.

Task	Time	Space (extra)
count / height / sum	$\Theta(n)$	$\Theta(h)$
level order	$\Theta(n)$	$\Theta(w)$, w = max width
BST search / insert / remove	$\Theta(h)$	$\Theta(1)$ iterative

Table 5.1: Cost summary for tree algorithms.

TREE CHECKLIST. (i) Always test the empty tree and the one-node tree. (ii) State the convention for height. (iii) Say whether equal keys go left or right. (iv) When inserting or removing, update *both* the child link and the parent link. (v) Pointers to nodes you intend to modify must be passed as `node **` (or returned and reassigned).

 Juevesano D. 2026 All rights reserved.

Part II

Practice Examination

Juevesano D. 2026 All rights reserved.

Chapter 6

Instructions and Marking Scheme

This part simulates the departmental written examination. It has three sections, in the order in which they are usually set.

Section	Chapter	Content
A. Concepts	7	short written answers: definitions, complexities, small proofs
B. Tracing	8	read a program and state its exact output; most items hide a twist
C. Problem solving	9	write code against a specification, with test cases and a required time complexity

6.1 How to use this part

(i) **Attempt each item on paper first.** Tracing is a paper skill; reading the answer teaches little..

(ii) **Time yourself.** A realistic budget is 2–3 minutes per concept item, 5–8 minutes per tracing item and 20–30 minutes per easy-to-medium problem.

(iii) **Check against Part III**, which gives the real program output, the reasoning, and for problems an idea, a complete commented solution, its complexity and the usual mistakes.

(iv) **Run the code.** Every program in the book was compiled and run; retyping the solutions is excellent practice, and altering the inputs shows what each line does.

6.2 Conventions used in the problem statements

- **Permitted operations.** Each problem lists the operations you may call. Anything else must be written by you, from the representation, if the statement allows it. Calling an operation that is not listed (for instance `search` when the problem forbids it) forfeits the credit even if the output is correct.
- **Starter code and constraints.** The box under the statement is the function (or the few functions) you have to write; the types and helper operations it uses are those of the cited listing. The constraints give the sizes the tests may reach.
- **Required.** The time (and sometimes space) complexity needed for full credit. A slower correct solution earns partial credit.
- **Bonus.** Extra credit for beating the required bound, or for an additional property (in place, single pass, $O(1)$ extra space).

 Juevesano D. 2026 All rights reserved.

- **Tests.** Sample inputs and expected outputs. They are illustrations, not the full test set: a real judge adds the empty case, the one-element case, duplicates, the maximum size and adversarial orderings.
- n is the number of items. Positions are 0-based unless the statement says otherwise.
- Unless stated otherwise, you may assume the memory returned by `malloc/new` is available, and you must *not* leak nodes that your code removes.

6.3 A sample marking scheme

The departmental scheme differs from term to term; the following is a plausible shape, useful for self-assessment.

Criterion	Share
Correct on all tests, including edge cases	50%
Meets the required complexity	20%
Uses only permitted operations (abstraction respected)	10%
No memory errors, no leaks, parent/tail/size fields consistent	10%
Readability and comments	10%
Bonus for a faster or leaner solution	up to +20% of the item

Exam technique.

(i) Write the invariant or the picture *before* the loop. A node-and-arrow sketch of the three cases (empty, one node, general) catches most pointer bugs.

(ii) Handle the edge cases explicitly, with their own `if`: empty structure, one element, first, last, not found, duplicates, everything equal.

(iii) State the complexity in a comment. If you know a faster approach but cannot finish it, submit the correct slower one *and say so*.

(iv) Never call a library routine that does the very thing being asked (e.g. `std::sort` in a problem about sorting); and do not depend on headers beyond `stdio/iostream` unless the statement permits.

Juevesano D. 2026 All rights reserved.

Chapter 7

Section A: Concept Questions

Answer briefly. Where a proof or a trace is requested, show it.

1. Define *abstract data type*. State the difference between an ADT and a data structure, and name the three ADTs of this coverage that are specialisations or relatives of the *List*.

2. Prove or disprove: (a) $23 = O(\log n)$; (b) $5 \log n = O(1)$; (c) $5n - 7 = O(n)$. Give constants c, n_0 where the claim is true.

3. State the *tightest* running time, in Θ-notation, of each operation. (a) `push_back` on an array list that is not full; (b) `insert` at the front of an array list; (c) `push_front` on an SLL; (d) `pop_back` on an SLL with head and tail pointers; (e) `pop_back` on a DLL with head and tail; (f) worst-case `get(i)` on a DLL; (g) searching an unsorted list; (h) worst-case insertion into a BST of n nodes.

4. A function `void f(list l)` receives an array-list `struct` (with an array *inside* it) by value and sets `l.items[0]=100`. Does the caller see the change? What if `l` is a struct with a `head` pointer to nodes, and the function sets `l.head->item=100`? What if the parameter is a raw array `int a[]`? Explain each answer.

5. After `erase(pos)` on an array list, the old last cell still holds a stale copy of the last value. Is this a bug? Which field makes the value harmless, and when would the stale value matter?

6. Distinguish `malloc/free` from `new/delete`. Define a *dangling pointer* and a *memory leak* and give a two-line example of each.

7. A function `void append(node *h, int x)` correctly appends to a non-empty list but silently fails when the list is empty. Why? Give two different repairs.

8. An array list currently holds s items. How many items are moved by `insert(p,x)` (positions $0 \le p \le s$) and by `erase(p)`? For which p is each worst, and what is that worst case?

9. A dynamic array doubles its capacity when full. Show that n successive `push_back` calls cost $O(n)$ in total. Why does growing by a constant $+10$ instead give $\Theta(n^2)$ total?

10. Why is it a mistake to halve the capacity as soon as $n \le cap/2$? Describe a bad sequence and a safe shrinking rule.

 Juevesano D. 2026 All rights reserved.

11. Binary search on a sorted array list needs what precondition and takes what time? Explain why a sorted-list `insert` nevertheless remains $\Theta(n)$ in the worst case even though the position is found in $O(\log n)$.

12. In the circular-array queue with fields (j, n) and capacity CAP: write the index of the front item, of the back item, and of the next free cell. Which values of (j, n) with $CAP = 4$ describe a full queue whose back item is at index 2?

13. For which operations does adding a `tail` pointer to a head-only SLL improve the running time, and from what to what? Which operation stays $\Theta(n)$ even with a tail pointer?

14. Explain precisely why `pop_back` on an SLL is $\Theta(n)$ even though a `tail` pointer is available.

15. A DLL node `c` must be inserted between neighbouring nodes `a` and `b` (with `a->next==b`). List every pointer assignment.

16. List the four cases that occur when removing a node `cur` from a DLL with `head/tail` pointers, and the assignments for each.

17. What is a *sentinel* (dummy) node? Name two special cases it removes from list code, and one cost.

18. Describe the *slow–fast pointer* technique and use it to find the middle node of an SLL in one pass. What is returned for a list of even length, with the usual loop condition?

19. State the loop invariant of the three-pointer in-place reversal of an SLL, and the running time and extra space.

20. Define *shallow copy* and *deep copy* of a linked list. If `l = m;` is executed for two list structs and `m` is then destroyed node by node, what happens to `l`?

21. What is *splicing*? Which pointers change when the whole of list `m` is spliced after node `p` of list `l`, and why is the cost $\Theta(1)$ only if some pointer is already known?

22. An SLL keeps `head`, `tail` *and* `mid`. State the invariant for `mid`. Which of `push_front`, `push_back`, `pop_front`, `pop_back`, `middle` can be $\Theta(1)$ and which not? Which change to the node type would make all of them $\Theta(1)$?

23. Define the *stack* ADT and give three applications. Which end of a singly linked list should serve as the top, and why?

24. Explain why a queue built from two stacks has amortised $\Theta(1)$ cost per operation although a single `dequeue` may cost $\Theta(n)$.

25. Show with the string `([)]` that counting the opening and closing brackets of each kind is not enough to test for balance, and say what data structure is.

26. Convert $(a + b) * c - d/e$ to postfix and to prefix.

27. Evaluate the postfix expression `5 1 2 + 4 * + 3 -`, showing the stack after each token.

Juevesano D. 2026 All rights reserved.

28. For the tree S_1 of Figure 5.1: give the height of the tree, the depth and height of node 6, the leaves, the ancestors of node 4, and the nodes at level 2.

29. What is the maximum number of nodes in a binary tree of height h? What is the minimum height of a binary tree with n nodes?

30. Prove that in a binary tree the number of leaves equals the number of nodes with two children plus one.

31. A tree has preorder `A B D E C F` and inorder `D B E A F C`. Reconstruct it and give its postorder.

32. Why do preorder and postorder together not determine a binary tree? Give two different trees with the same preorder and the same postorder.

33. State the BST property. Is the tree with root 10, left child 5 and 5's right child 12 a BST? Explain, and state the correct recursive test.

34. Insert $50, 30, 70, 20, 40, 60, 80, 35$ into an empty BST. Give the height and the inorder traversal. Then remove 30 and give the preorder traversal of the result.

35. Give the worst-case time of searching a BST in terms of n, an insertion order that attains it, and the best possible height for n keys.

36. Which traversal uses a queue, and why? Which traversals can be made iterative with a stack? State the time and the extra space of each in terms of n and h.

37. A recursive `height` function crashes with a stack overflow on an input of 10^6 keys inserted in increasing order. Explain, and describe an iterative alternative.

 Juevesano D. 2026 All rights reserved.

Chapter 8

Section B: Tracing Problems

Determine the *exact* output of each program, including spaces and line breaks. Each one hides at least one trap; the most common are listed in the checklist below. Unless stated, the code compiles and runs without error. The programs that include a header (`stackqueue.h`, `sll_template.h`, `midlist.h`, `tree_base.h`, `tree_probs.h`, `dll_template.h`) use the functions of Part I and of the problems in Chapter 9; their behaviour is stated before each program.

> Tracing checklist. (i) Evaluation order of function arguments is unspecified; the programs here use sequential statements to avoid it. (ii) Pass by value copies a struct *including* an embedded array, but a pointer member still points to the same nodes. (iii) A loop bound such as `i<=n` or `i<size-1` changes the count by one. (iv) `printf` adds nothing: look for trailing spaces and `\n`. (v) Integer division truncates. (vi) Keep a table with one column per variable and one row per step; do not trace in your head.

Tracing 1. Array list, return values and stale cells

A bounded array list with capacity 5 and *1-based* `setItem` and `itemAt`. Give the exact output of the program.

```
typedef struct { int items[5]; int size; } list;

void init(list *l) { l->size = 0; }
int append(list *l, int x) {
    if (l->size == 5) return 0;
    l->items[l->size++] = x;
    return 1;
}
int insertFront(list *l, int x) {
    int i;
    if (l->size == 5) return 0;
    for (i = l->size; i > 0; i--) l->items[i] = l->items[i - 1];
    l->items[0] = x;
    l->size++;
    return 1;
}
int removeFirst(list *l) {
    int i;
    if (l->size == 0) return 0;
    for (i = 0; i < l->size - 1; i++) l->items[i] = l->items[i + 1];
    l->size--;
    return 1;
}
int removeLast(list *l) { if (l->size == 0) return 0; l->size--; return 1; }
```

Juevesano D. 2026 All rights reserved.

```
int setItem(list *l, int x, int p) {          /* p is 1-based */
    if (p < 1 || p > l->size) return 0;
    l->items[p - 1] = x;
    return 1;
}
int itemAt(list l, int p) { return l.items[p - 1]; }
void display(list l) {
    int i;
    for (i = 0; i < l.size; i++) printf("%d ", l.items[i]);
    printf("\n");
}

int main() {
    list l;
    int r;
    init(&l);
    r = append(&l, 4);
    r = r + append(&l, 8);
    r = r + append(&l, 15);
    printf("%d\n", r);
    r = insertFront(&l, 16);
    r = r * 10 + insertFront(&l, 23);
    printf("%d\n", r);
    printf("%d\n", append(&l, 42));
    display(l);
    removeLast(&l);
    removeFirst(&l);
    display(l);
    r = setItem(&l, 99, 3);
    printf("%d ", r);
    r = setItem(&l, 77, 4);
    printf("%d\n", r);
    printf("%d\n", itemAt(l, 4));
    append(&l, 5);
    display(l);
    printf("%d\n", itemAt(l, 4));
    return 0;
}
```

Write the output, line by line, and the final contents of the raw array `items[0..4]` including cells beyond `size`.

 Juevesano D. 2026 All rights reserved.

Tracing 2. What does the callee really change?

Five calls, five different ways of passing a list. Give the five output lines.

```c
typedef struct { int items[3]; int size; } alist;          /* array inside  */
typedef struct node { int item; struct node *next; } node;
typedef struct { node *head; int size; } slist;           /* pointer inside */

void arrByValue(alist l) { l.items[0] = 100; l.size = 9; }
void arrByPtr(alist *l)  { l->items[0] = 100; l->size = 9; }
void rawArray(int a[])   { a[0] = 100; }
void llByValue(slist l)  { l.head->item = 100; l.head = NULL; l.size = 0; }

int main() {
    alist a = {{1, 2, 3}, 3};
    int raw[3] = {1, 2, 3};
    slist s;
    node *n2 = (node *)malloc(sizeof(node));
    node *n1 = (node *)malloc(sizeof(node));
    n2->item = 2; n2->next = NULL;
    n1->item = 1; n1->next = n2;
    s.head = n1; s.size = 2;

    arrByValue(a);
    printf("A: %d %d\n", a.items[0], a.size);
    arrByPtr(&a);
    printf("B: %d %d\n", a.items[0], a.size);
    a.items[0] = 1; a.size = 3;
    rawArray(a.items);
    printf("C: %d %d\n", a.items[0], a.size);
    rawArray(raw);
    printf("D: %d\n", raw[0]);
    llByValue(s);
    printf("E: %d %d %d\n", s.head == NULL, s.size, n1->item);
    return 0;
}
```

Explain, in one sentence for each of A–E, which parts of the argument were copies and which were shared.

Tracing 3. Counting loop iterations

Give the six printed counts. Then state the Θ-class of each of `c1`–`c6` in terms of n (here $n = 16$).

```c
int main() {
    int n = 16, i, j, c1 = 0, c2 = 0, c3 = 0, c4 = 0, c5 = 0, c6 = 0;

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            c1++;

    for (i = 0; i < n; i++)
        for (j = 0; j <= i; j++)
```

Juevesano D. 2026 All rights reserved.

```
            c2++;

    for (i = 1; i < n; i *= 2)
        for (j = 0; j < n; j++)
            c3++;

    for (i = n; i > 0; i /= 2)
        c4++;

    for (i = 0; i < n; i++)
        for (j = i; j < n; j += 4)
            c5++;

    for (i = 1; i < n; i *= 2)
        for (j = 0; j < i; j++)
            c6++;

    printf("%d %d %d %d %d %d\n", c1, c2, c3, c4, c5, c6);
    return 0;
}
```

Tracing 4. The raw array after erase and insert

The class below is the lecture's ArrayList (positions 0-based, capacity 8, the constructor zeroes the array). `raw()` prints the *whole* array. Give the output.

```
class ArrayList {
    int items[8];
    int size;
public:
    ArrayList() { size = 0; for (int i = 0; i < 8; i++) items[i] = 0; }
    bool push_back(int x) {
        if (size == 8) return false;
        items[size++] = x;
        return true;
    }
    bool erase(int pos) {                       /* 0-based */
        if (pos < 0 || pos >= size) return false;
        for (int i = pos; i < size - 1; i++) items[i] = items[i + 1];
        size--;
        return true;
    }
    bool insert(int pos, int x) {               /* 0-based */
        if (size == 8 || pos < 0 || pos > size) return false;
        for (int i = size; i > pos; i--) items[i] = items[i - 1];
        items[pos] = x;
        size++;
        return true;
    }
    void raw() {
        printf("size=%d  [", size);
        for (int i = 0; i < 8; i++)
```

 Juevesano D. 2026 All rights reserved.

```
            printf("%d%s", items[i], i < 8 - 1 ? " " : "");
        printf("]\n");
    }
};

int main() {
    ArrayList l;
    for (int i = 1; i <= 6; i++) l.push_back(i * 10);
    l.raw();
    l.erase(1);
    l.raw();
    l.insert(3, 99);
    l.raw();
    int r = l.erase(5);
    r = r + l.erase(5);
    printf("%d\n", r);
    l.raw();
    l.insert(0, 7);
    l.insert(0, 8);
    l.raw();
    r = l.insert(9, 1);
    printf("%d ", r);
    r = l.insert(8, 1);
    printf("%d\n", r);
    l.raw();
    return 0;
}
```

Tracing 5. Circular queue: wrap-around

A circular queue with j the index of the front item and n the number of items; capacity 4. `add` fails when full. Give the output; the printed array is the raw memory.

```
// circular queue: item i of the queue lives at a[(j + i) % 4]
typedef struct {
    int a[4];
    int j;          // index of the front item
    int n;          // number of items
} cqueue;

int add(cqueue *q, int x) {
    if (q->n == 4) return 0;
    q->a[(q->j + q->n) % 4] = x;
    q->n++;
    return 1;
}

int removeFront(cqueue *q, int *out) {
    if (q->n == 0) return 0;
    *out = q->a[q->j];
    q->j = (q->j + 1) % 4;
    q->n--;
```

```
    return 1;
}

void show(cqueue *q) {
    printf("j=%d n=%d ", q->j, q->n);
    printf("[%d %d %d %d]\n", q->a[0], q->a[1], q->a[2], q->a[3]);
}

int main() {
    cqueue q = {{0, 0, 0, 0}, 0, 0};
    int x;
    add(&q, 1); add(&q, 2); add(&q, 3); show(&q);
    removeFront(&q, &x); printf("removed %d\n", x); show(&q);
    add(&q, 4); add(&q, 5); show(&q);
    printf("%d\n", add(&q, 6));
    removeFront(&q, &x); removeFront(&q, &x); show(&q);
    add(&q, 7); add(&q, 8); show(&q);
    while (removeFront(&q, &x)) printf("%d ", x);
    printf("\n");
    show(&q);
    return 0;
}
```

Tracing 6. SLL WITH HEAD, TAIL AND SIZE

The header `sll_template.h` provides `sinit`, `pushBack`, `pushFront` and `sdisplay` (prints the list in brackets) for an SLL with `head`, `tail`, `size`. Give the output of every line.

```
int popFront(sll *l, int *out) {
    snode *old;
    if (l->head == NULL) return 0;
    old = l->head;
    *out = old->item;
    l->head = old->next;
    if (l->head == NULL) l->tail = NULL;
    free(old);
    l->size--;
    return 1;
}
int popBack(sll *l, int *out) {
    snode *cur;
    if (l->head == NULL) return 0;
    *out = l->tail->item;
    if (l->head == l->tail) {
        free(l->head);
        l->head = l->tail = NULL;
    } else {
        cur = l->head;
        while (cur->next != l->tail) cur = cur->next;
        free(l->tail);
        cur->next = NULL;
        l->tail = cur;
```

 Juevesano D. 2026 All rights reserved.

```
    }
    l->size--;
    return 1;
}
void state(char *op, sll *l) {
    printf("%-10s size=%d head=", op, l->size);
    if (l->head) printf("%d", l->head->item); else printf("NULL");
    printf(" tail=");
    if (l->tail) printf("%d", l->tail->item); else printf("NULL");
    printf("  ");
    sdisplay(l);
}

int main() {
    sll l;
    int x = -1;
    sinit(&l);
    pushBack(&l, 5);   state("pushBack 5", &l);
    pushFront(&l, 3);  state("pushFront 3", &l);
    pushBack(&l, 8);   state("pushBack 8", &l);
    popBack(&l, &x);   state("popBack", &l);
    popBack(&l, &x);   state("popBack", &l);
    popFront(&l, &x);  state("popFront", &l);
    int r = popFront(&l, &x);
    printf("%d %d\n", r, x);
    pushFront(&l, 1);  state("pushFront 1", &l);
    pushBack(&l, 2);   state("pushBack 2", &l);
    return 0;
}
```

Tracing 7. One star or two?

Both functions append to a singly linked list. Give the two output lines, and for `a`, `b`, `c`, `d` the number of nodes that are *unreachable* (leaked) at the end.

```
typedef struct node { int item; struct node *next; } node;

void appendA(node *h, int x) {          /* single pointer */
    node *n = (node *)malloc(sizeof(node));
    n->item = x; n->next = NULL;
    if (h == NULL) { h = n; return; }
    while (h->next != NULL) h = h->next;
    h->next = n;
}
void appendB(node **h, int x) {         /* double pointer */
    node *n = (node *)malloc(sizeof(node));
    n->item = x; n->next = NULL;
    if (*h == NULL) { *h = n; return; }
    node *cur = *h;
    while (cur->next != NULL) cur = cur->next;
    cur->next = n;
}
```

Juevesano D. 2026 All rights reserved.

```
int length(node *h) { return h == NULL ? 0 : 1 + length(h->next); }

int main() {
    node *a = NULL, *b = NULL, *c = NULL, *d = NULL;
    for (int i = 1; i <= 3; i++) { appendA(a, i); appendB(&b, i); }
    printf("%d %d\n", length(a), length(b));
    appendB(&c, 7);
    appendB(&d, 7);
    for (int i = 1; i <= 3; i++) { appendA(c, i); appendB(&d, i); }
    printf("%d %d\n", length(c), length(d));
    return 0;
}
```

Tracing 8. Erasing from a doubly linked list

`eraseItems` is the function of Problem 13; `display` prints the list from head to tail and `displayBackward` from tail to head. Give the output.

```
int main() {
    int v[] = {2, 2, 5, 2, 7, 2, 2}, i;
    list l;
    init(&l);
    for (i = 0; i < 7; i++) append(&l, v[i]);
    eraseItems(&l, 2);
    display(&l);
    printf("size=%d head=%d tail=%d\n", l.size, l.head->item, l.tail->item);
    displayBackward(&l);
    eraseItems(&l, 9);
    printf("size=%d\n", l.size);
    eraseItems(&l, 7);
    printf("size=%d head=%d tail=%d ", l.size, l.head->item, l.tail->item);
    printf("prev=%d\n", l.tail->prev == NULL);
    eraseItems(&l, 5);
    printf("size=%d ", l.size);
    printf("headIsNULL=%d tailIsNULL=%d\n", l.head == NULL, l.tail == NULL);
    return 0;
}
```

Tracing 9. Recursion on a list

Lists $a = 1 \to 2 \to 3 \to 4$ and $b = 3 \to 5 \to 2 \to 8 \to 6 \to 9$ are built with a pointer-to-pointer. Give the three output lines.

```
typedef struct node { int item; struct node *next; } node;

void mystery(node *p) {
    if (p == NULL) { printf("# "); return; }
    printf("%d ", p->item);
    mystery(p->next);
    printf("%d ", p->item);
}
```

 Juevesano D. 2026 All rights reserved.

```
9
10 int g(node *p) {
11     if (p == NULL) return 0;
12     if (p->item % 2 == 0) return p->item + g(p->next);
13     return g(p->next) - 1;
14 }
15
16 int h(node *p) {
17     if (p == NULL || p->next == NULL) return 0;
18     if (p->item < p->next->item) return 1 + h(p->next);
19     return h(p->next);
20 }
21
22 int main() {
23     int v[] = {1, 2, 3, 4}, w[] = {3, 5, 2, 8, 6, 9}, i;
24     node *a = NULL, *b = NULL, **t = &a;
25     for (i = 0; i < 4; i++) {          // t: address of the last next field
26         *t = (node *)malloc(sizeof(node));
27         (*t)->item = v[i];
28         (*t)->next = NULL;
29         t = &((*t)->next);
30     }
31     t = &b;
32     for (i = 0; i < 6; i++) {
33         *t = (node *)malloc(sizeof(node));
34         (*t)->item = w[i];
35         (*t)->next = NULL;
36         t = &((*t)->next);
37     }
38     mystery(a); printf("\n");
39     printf("%d %d\n", g(a), g(b));
40     printf("%d %d\n", h(a), h(b));
41     return 0;
42 }
```

Show the call stack of `mystery(a)`, and the value of each recursive call of `g(b)`.

Tracing 10. Stacks and queues in the same function

Header `stackqueue.h` (Listing 4.1): the pop and dequeue functions return 1 on success and 0 on an empty container, and then leave the output parameter unchanged. Give the output.

```
1  int main() {
2      stack s;
3      queue q;
4      int x = -1, y = -1, r;
5      stInit(&s);
6      qInit(&q);
7      stPush(&s, 1); stPush(&s, 2); stPush(&s, 3);
8      qEnq(&q, 10); qEnq(&q, 20);
9      stPop(&s, &x);
10     qDeq(&q, &y);
```

Juevesano D. 2026 All rights reserved.

```
    printf("%d %d\n", x, y);
    qEnq(&q, x);
    stPush(&s, y);
    r = stPop(&s, &x);
    r = r + stPop(&s, &x);
    r = r + stPop(&s, &x);
    r = r + stPop(&s, &x);
    printf("%d %d\n", r, x);
    r = qDeq(&q, &y);
    r = r + qDeq(&q, &y);
    r = r + qDeq(&q, &y);
    r = r + qDeq(&q, &y);
    printf("%d %d\n", r, y);
    stPush(&s, 4); stPush(&s, 5);
    while (!stEmpty(&s)) { stPop(&s, &x); qEnq(&q, x); }
    while (!qEmpty(&q)) { qDeq(&q, &y); stPush(&s, y); }
    while (!stEmpty(&s)) { stPop(&s, &x); printf("%d ", x); }
    printf("\n");
    return 0;
}
```

Tracing 11. INFIX TO POSTFIX

Operator precedence: `*`, `/` above `+`, `-`. The program converts an infix string. Give the output: the postfix string and the largest stack size observed after any character.

```
int prec(char op) {
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

int main() {
    char *e = "a+b*(c-d)/e";
    char out[50];
    int w = 0, t, maxDepth = 0;
    stack st;
    stInit(&st);
    for (int i = 0; e[i]; i++) {
        char c = e[i];
        if (c >= 'a' && c <= 'z') out[w++] = c;
        else if (c == '(') stPush(&st, c);
        else if (c == ')') {
            while (stTop(&st, &t) && t != '(') {
                stPop(&st, &t);
                out[w++] = (char)t;
            }
            stPop(&st, &t);
        } else {
            while (stTop(&st, &t) && prec((char)t) >= prec(c)) {
                stPop(&st, &t);
                out[w++] = (char)t;
```

 Juevesano D. 2026 All rights reserved.

```
            }
            stPush(&st, c);
        }
        if (stSize(&st) > maxDepth) maxDepth = stSize(&st);
    }
    while (stPop(&st, &t)) out[w++] = (char)t;
    out[w] = '\0';
    printf("%s %d\n", out, maxDepth);
    return 0;
}
```

Show the stack after each of the 11 characters.

Tracing 12. A QUEUE FROM TWO STACKS

The functions `enq` and `deq` implement a queue with two stacks. `deq` returns -1 for an empty queue. Give the output, and the number of *successful* stack pushes plus pops performed by the *first* call of `deq`.

```
// a queue built from two stacks:
// enqueue pushes onto in; dequeue pops from out (refilled from in when empty)
typedef struct {
    stack in;
    stack out;
} twoQ;

void enq(twoQ *q, int x) { stPush(&q->in, x); }

int deq(twoQ *q) {
    int x;
    if (stEmpty(&q->out))
        while (stPop(&q->in, &x)) stPush(&q->out, x);
    if (!stPop(&q->out, &x)) return -1;
    return x;
}

int main() {
    twoQ q;
    int a;
    stInit(&q.in);
    stInit(&q.out);
    enq(&q, 1); enq(&q, 2); enq(&q, 3);
    a = deq(&q); printf("%d ", a);
    enq(&q, 4);
    a = deq(&q); printf("%d ", a);
    enq(&q, 5); enq(&q, 6);
    a = deq(&q); printf("%d ", a);
    a = deq(&q); printf("%d ", a);
    a = deq(&q); printf("%d ", a);
    a = deq(&q); printf("%d ", a);
    a = deq(&q); printf("%d\n", a);
    printf("in=%d out=%d\n", stSize(&q.in), stSize(&q.out));
    return 0;
```

Juevesano D. 2026 All rights reserved.

```
}
```

Tracing 13. Level-order insertion is not BST insertion

`insertLevel` puts each new key into the first vacant position in level order (it builds a complete tree). `search` is the *BST* search. Give the output.

```
int main() {
    node *t = NULL;
    int v[] = {40, 10, 60, 5, 20, 50, 70, 15};
    for (int i = 0; i < 8; i++) {
        insertLevel(&t, v[i]);
        cout << countNodes(t) << " ";
    }
    cout << endl;
    levelOrder(t);
    preOrder(t);  cout << endl;
    inOrder(t);   cout << endl;
    postOrder(t); cout << endl;
    cout << search(t, 20) << search(t, 60);
    cout << search(t, 15) << search(t, 50) << endl;
    return 0;
}
```

Draw the tree, and explain the last line.

Tracing 14. BST with duplicates and comparison counts

Duplicates go to the *right* (the course convention); `height` counts edges. `find` counts comparisons: two per visited node unless the key matches there. Give the output.

```
// searches for x; counts every comparison in *comparisons
bool find(node *r, int x, int *comparisons) {
    while (r != NULL) {
        (*comparisons)++;
        if (x == r->item) return true;
        (*comparisons)++;
        r = (x < r->item) ? r->left : r->right;
    }
    return false;
}

int main() {
    node *t = NULL;
    int v[] = {50, 30, 70, 30, 60, 50, 80, 30};
    int c;
    for (int i = 0; i < 8; i++) insert(&t, v[i]);
    preOrder(t);   cout << endl;
    inOrder(t);    cout << endl;
    postOrder(t);  cout << endl;
    levelOrder(t);
    cout << height(t) << " " << countNodes(t) << endl;
```

 Juevesano D. 2026 All rights reserved.

```
    c = 0; find(t, 60, &c); cout << c << " ";
    c = 0; find(t, 35, &c); cout << c << " ";
    c = 0; find(t, 30, &c); cout << c << endl;
    return 0;
}
```

Draw the tree.

Tracing 15. BST REMOVALS

`removeKey(&t,x)` removes x by the standard three cases (a node with two children takes the key of its inorder successor) and returns 1 if the key was found, otherwise 0. Give the output; the last line is the level-order traversal.

```
int main() {
    node *t = NULL;
    int v[] = {50, 25, 80, 10, 36, 60, 91, 29, 55, 76, 68, 78};
    for (int i = 0; i < 12; i++) insert(&t, v[i]);
    preOrder(t); cout << endl;
    removeKey(&t, 91);  preOrder(t); cout << endl;
    removeKey(&t, 36);  preOrder(t); cout << endl;
    removeKey(&t, 25);  preOrder(t); cout << endl;
    removeKey(&t, 50);  preOrder(t); cout << endl;
    int r = removeKey(&t, 99);
    cout << r << " " << t->item << " " << (t->parent == NULL) << endl;
    levelOrder(t);
    return 0;
}
```

Draw the tree after each removal.

Tracing 16. A RECURSIVE TREE MYSTERY

The tree is built with the BST `insert`. Give the output of `f` (its `cout`), then the three numbers m, `g(t,0)` and `k(t)`.

```
int f(node *r) {
    if (r == NULL) return 0;
    int a = f(r->left);
    int b = f(r->right);
    cout << r->item << " ";
    return (a > b ? a : b) + r->item;
}
int g(node *r, int d) {
    if (r == NULL) return 0;
    int s = (d % 2 == 0) ? r->item : -r->item;
    return s + g(r->left, d + 1) + g(r->right, d + 1);
}
int k(node *r) {
    if (r == NULL) return 0;
    if (r->left == NULL && r->right == NULL) return 1;
    return k(r->left) * 2 + k(r->right) * 3;
}
```

Juevesano D. 2026 All rights reserved.

```
18
19 int main() {
20     node *t = NULL;
21     int v[] = {8, 3, 10, 1, 6, 14, 4, 7};
22     for (int i = 0; i < 8; i++) insert(&t, v[i]);
23     int m = f(t);
24     cout << endl << m << " " << g(t, 0) << " " << k(t) << endl;
25     return 0;
26 }
```

Say in one sentence what each of `f`, `g`, `k` computes.

Tracing 17. The forgotten parent

`insert` (iterative, as in the course) sets `parent`; `rinsert` (the recursive version) does not. Give the output.

```
1  void rinsert(node **r, int x) {          /* the recursive insert of bst.cpp */
2      if (*r == NULL) *r = new node(x);
3      else if (x < (*r)->item) rinsert(&((*r)->left), x);
4      else rinsert(&((*r)->right), x);
5  }
6  int orphans(node *r) {                   /* nodes whose parent is NULL */
7      if (r == NULL) return 0;
8      return (r->parent == NULL ? 1 : 0) + orphans(r->left) + orphans(r->right);
9  }
10 int depth(node *n) {                     /* edges up to the root via parent */
11     int d = 0;
12     while (n->parent != NULL) { n = n->parent; d++; }
13     return d;
14 }
15
16 int main() {
17     node *a = NULL, *b = NULL;
18     int v[] = {40, 20, 60, 10, 30, 50};
19     for (int i = 0; i < 6; i++) { insert(&a, v[i]); rinsert(&b, v[i]); }
20     cout << countNodes(a) << " " << countNodes(b) << endl;
21     cout << orphans(a) << " " << orphans(b) << endl;
22     cout << depth(a->left->right) << " " << depth(b->left->right) << endl;
23     return 0;
24 }
```

Explain each line.

 Juevesano D. 2026 All rights reserved.

Tracing 18. Iterative inorder with a depth counter

A BST is built, then traversed with an explicit stack. `depth` counts the nodes currently on the stack. Give the output.

```
int main() {
    node *t = NULL;
    int v[] = {40, 20, 60, 10, 30, 50, 70};
    for (int i = 0; i < 7; i++) insert(&t, v[i]);

    nstack st;
    node *cur = t;
    int maxDepth = 0, depth = 0;
    while (cur != NULL || !st.empty()) {
        while (cur != NULL) { st.push(cur); depth++; cur = cur->left; }
        if (depth > maxDepth) maxDepth = depth;
        cur = st.pop();
        depth--;
        cout << cur->item << "(" << depth << ") ";
        cur = cur->right;
    }
    cout << endl << maxDepth << endl;
    return 0;
}
```

Tracing 19. Successor and predecessor

`successor(c)` returns the next node in inorder using parent pointers (or `NULL`); `predecessor` is its mirror image. Give the output.

```
int main() {
    node *t = NULL;
    int v[] = {50, 30, 70, 20, 40, 60, 80, 35, 45};
    for (int i = 0; i < 9; i++) insert(&t, v[i]);
    node *m = t;
    while (m->left != NULL) m = m->left;                // the smallest node
    for (node *c = m; c != NULL; c = successor(c)) {
        node *s = successor(c);
        cout << c->item << ">" << (s ? s->item : -1) << " ";
    }
    cout << endl;
    node *p = predecessor(t->right->left);              // node 60
    node *x = t->left->right->right;                    // node 45
    cout << p->item << " " << successor(x)->item << " ";
    cout << (successor(t->right->right) == NULL) << " "; // node 80
    cout << (predecessor(m) == NULL) << endl;
    return 0;
}
```

Draw the tree.

Juevesano D. 2026 All rights reserved.

Tracing 20. The mid-pointer list

A list with `head`, `tail`, `size` and `mid`, where `mid` always designates the node at index $\lfloor(\texttt{size}-1)/2\rfloor$ (0-based). `middle(&l)` returns its item; the four end operations are `mPushBack`, `mPushFront`, `mPopFront`, `mPopBack`. Give the output.

```
void show(char *op, mlist *l) {
    mnode *c;
    printf("%-12s n=%d mid=", op, l->size);
    if (l->size) printf("%d", middle(l)); else printf("-");
    printf("  [");
    for (c = l->head; c != NULL; c = c->next)
        printf("%d%s", c->item, c->next ? " " : "");
    printf("]\n");
}

int main() {
    mlist l;
    int x;
    minit(&l);
    mPushBack(&l, 10);    show("mPushBack 10", &l);
    mPushBack(&l, 20);    show("mPushBack 20", &l);
    mPushBack(&l, 30);    show("mPushBack 30", &l);
    mPushBack(&l, 40);    show("mPushBack 40", &l);
    mPushFront(&l, 5);    show("mPushFront 5", &l);
    mPushFront(&l, 1);    show("mPushFront 1", &l);
    mPopFront(&l, &x);    show("mPopFront", &l);
    mPopBack(&l, &x);     show("mPopBack", &l);
    mPushBack(&l, 50);    show("mPushBack 50", &l);
    mPopFront(&l, &x);    show("mPopFront", &l);
    return 0;
}
```

For each line state whether `mid` moved, and in which direction.

Tracing 21. Binary search: counting the steps

`lowerBound` returns the first index i with $a[i] \geq x$ (or n if none). The array contains duplicates. Give the output of all three searches.

```
int lowerBound(int a[], int n, int x) {
    int lo = 0, hi = n, mid, step = 1;
    while (lo < hi) {
        mid = lo + (hi - lo) / 2;
        printf("step %d: lo=%d hi=%d ", step++, lo, hi);
        printf("mid=%d a[mid]=%d\n", mid, a[mid]);
        if (a[mid] < x) lo = mid + 1;
        else            hi = mid;
    }
    return lo;
}

int main() {
```

 Juevesano D. 2026 All rights reserved.

```
    int a[] = {2, 4, 4, 4, 7, 9, 12, 15, 15};
    printf("answer %d\n", lowerBound(a, 9, 4));
    printf("answer %d\n", lowerBound(a, 9, 10));
    printf("answer %d\n", lowerBound(a, 9, 20));
    return 0;
}
```

How many iterations does the search for $x = 4$ take, and why is the answer 1 rather than 2 or 3?

Tracing 22. Josephus with a twist

Six people stand in a circle numbered 1..6 and every third is eliminated. Give the output.

```
int main() {
    queue q;
    int x, k = 3, step;
    qInit(&q);
    for (int i = 1; i <= 6; i++) qEnq(&q, i);
    while (qSize(&q) > 1) {
        for (step = 1; step < k; step++) { qDeq(&q, &x); qEnq(&q, x); }
        qDeq(&q, &x);
        printf("out:%d ", x);
    }
    qDeq(&q, &x);
    printf("\nwinner:%d\n", x);
    return 0;
}
```

Change k to 2 and give the winner.

Tracing 23. Splicing lists

`splice(&a,p,&b)` moves *all* nodes of `b` into `a` after the first p nodes of `a` (relinking, no copying) and returns 0 if p is not between 0 and the size of `a`; `spliceAfter(&a,x,&c)` does the same after the node `x`. Give the output.

```
int main() {
    sll a, b, c, d;
    int r;
    sinit(&a); sinit(&b); sinit(&c); sinit(&d);
    for (int i = 1; i <= 5; i++) pushBack(&a, i);          // 1 2 3 4 5
    for (int i = 1; i <= 3; i++) pushBack(&b, i * 10);     // 10 20 30
    for (int i = 7; i <= 9; i++) pushBack(&c, i);          // 7 8 9
    pushBack(&d, 100);
    splice(&a, 2, &b);
    sdisplay(&a); sdisplay(&b);
    printf("%d %d\n", a.size, b.size);
    spliceAfter(&a, a.head->next->next, &c);
    sdisplay(&a); sdisplay(&c);
    printf("%d %d tail=%d\n", a.size, c.size, a.tail->item);
    r = splice(&a, 99, &d);
    printf("%d %d\n", r, d.size);
```

Juevesano D. 2026 All rights reserved.

```
    r = splice(&a, a.size, &d);
    sdisplay(&a);
    printf("%d %d tail=%d\n", r, a.size, a.tail->item);
    return 0;
}
```

Tracing 24. Shallow and deep copies

Give the three lines printed by `show`, and say which lists share nodes.

```
typedef struct node { int item; struct node *next; } node;
typedef struct { node *head; int size; } slist;

slist shallowCopy(slist s) { return s; }
slist deepCopy(slist s) {
    slist d;
    node *c = s.head, **t = &d.head;
    d.head = NULL; d.size = s.size;
    while (c != NULL) {
        *t = (node *)malloc(sizeof(node));
        (*t)->item = c->item; (*t)->next = NULL;
        t = &((*t)->next);
        c = c->next;
    }
    return d;
}
void show(slist s) {
    for (node *c = s.head; c != NULL; c = c->next) printf("%d ", c->item);
    printf("| %d\n", s.size);
}
void pushFront(slist *s, int x) {
    node *n = (node *)malloc(sizeof(node));
    n->item = x; n->next = s->head; s->head = n; s->size++;
}

int main() {
    slist orig = {NULL, 0};
    pushFront(&orig, 3); pushFront(&orig, 2); pushFront(&orig, 1);
    slist sh = shallowCopy(orig), dp = deepCopy(orig);
    sh.head->item = 99;
    dp.head->next->item = 77;
    pushFront(&sh, 0);
    show(orig); show(sh); show(dp);
    return 0;
}
```

 Juevesano D. 2026 All rights reserved.

Tracing 25. EXPRESSION TREE

The tree is built by the constructor calls in `main`. Give the four output lines.

```
using namespace std;

class tnode {
public:
    char c;
    tnode *l, *r;
    tnode(char x, tnode *a = NULL, tnode *b = NULL) { c = x; l = a; r = b; }
};
void pre(tnode *t)  { if (t) { cout << t->c; pre(t->l); pre(t->r); } }
void in(tnode *t)   { if (t) { in(t->l); cout << t->c; in(t->r); } }
void post(tnode *t) { if (t) { post(t->l); post(t->r); cout << t->c; } }
int eval(tnode *t) {
    if (t->l == NULL) return t->c - '0';
    int a = eval(t->l), b = eval(t->r);
    switch (t->c) {
        case '+': return a + b;
        case '-': return a - b;
        case '*': return a * b;
        default:  return a / b;
    }
}

int main() {
    tnode *plus = new tnode('+', new tnode('6'), new tnode('2'));
    tnode *times = new tnode('*', plus, new tnode('3'));
    tnode *div = new tnode('/', new tnode('9'), new tnode('4'));
    tnode *t = new tnode('-', times, div);
    pre(t);  cout << endl;
    in(t);   cout << endl;
    post(t); cout << endl;
    cout << eval(t) << endl;
    return 0;
}
```

Draw the tree. What is the result if `eval` used real division?

Juevesano D. 2026 All rights reserved.

Chapter 9

Section C: Problem Solving

There are 52 problems in four groups. Every problem is of *Easy* or *Medium* difficulty and is written in the style of an online judge: you are *given* a structure and asked to *implement* one operation on it. Each statement shows the function to complete (the starter code), the constraints, the PERMITTED OPERATIONS, the REQUIRED complexity, an optional BONUS for doing better, and sample tests. The code is plain C/C++: no global variables, no `static` and no `const` are needed anywhere, and the solutions in Chapter 12 follow the same rule.

> INTERFACES ASSUMED THROUGHOUT. *Array list*: `items[]` and `size`. *SLL*: Listing 3.1 (`snode`, `sll` with `head`, `tail`, `size`). *DLL*: Listing 3.3 (`node`, `list`). *Stack and queue*: Listing 4.1. *Tree*: Listing 5.1. When a problem names a function, use exactly that signature; when it forbids an operation, even a correct use of it forfeits credit.

9.1 Arrays and Array Lists

Problem 1. STRING COMPRESSION (RUN-LENGTH ENCODING) *Easy*

A *run* is a maximal block of equal consecutive characters. The *run-length encoding* of a string writes, for each run, the character followed by the length of the run. Given a string `s` of letters, implement `rleEncode`: it writes the encoding into `out` and returns the length of the encoding. The count is always written, even when it is 1, and it may have several digits.

Then implement `compress`, which does the same *inside the array* `a` of `n` characters, but a run of length 1 is written as the bare character. It returns the new length. No second array.

```
int rleEncode(char *s, char *out);     // out must end with '\0'
int compress(char *a, int n);          // in place, returns the new length
```

PERMITTED OPERATIONS. Array indexing only. No `strlen`, `strcat`, `sprintf` or `itoa`: write the digits of a count yourself.
CONSTRAINTS. $1 \le n \le 10^5$; the string contains only letters.
REQUIRED. $O(n)$ time; `compress` uses $O(1)$ extra space.
BONUS (EXTRA CREDIT). Implement `rleDecode(char *e, char *out)`.

INPUT	EXPECTED OUTPUT
`aaabccdddd`	`a3b1c2d4`
`wwwwwwwwwwwbbb`	`w12b3`
`compress: aabcccccaaa`	`a2bc5a3 (length 7)`
`compress: abbbbbbbbbbbb`	`ab12 (length 4)`
`decode: x3y2z1`	`xxxyyz`

 Juevesano D. 2026 All rights reserved.

Problem 2. Matrix transpose *Easy*

Given an $n \times m$ matrix `a` (n rows, m columns), implement `transpose`, which stores the $m \times n$ transpose in `t`, so that $t_{ji} = a_{ij}$. Then implement `transposeInPlace` for a *square* $n \times n$ matrix.

```
void transpose(int a[][10], int n, int m, int t[][10]);
void transposeInPlace(int a[][10], int n);
```

Permitted operations. Array indexing only.
Constraints. $1 \le n, m \le 10$.
Required. $O(nm)$ time; `transposeInPlace` uses $O(1)$ extra space.

Input	Expected output
`[[1,2,3],[4,5,6]]`	`[[1,4],[2,5],[3,6]]`
`[[1,2],[3,4]] (in place)`	`[[1,3],[2,4]]`
`[[7]]`	`[[7]]`

Problem 3. The missing number *Easy*

Given an array `a` of `m` distinct integers in *increasing* order, taken from $\{1, 2, \ldots, m+1\}$, exactly one number of that range is missing. Return it.

Assumption. The statement as remembered is ambiguous; this is the usual reading: the array has one element fewer than the range it is drawn from.

```
int missingNumber(int a[], int m);
```

Permitted operations. Array reads only. No sorting, no copy of the array, no counting array.
Constraints. $1 \le m \le 10^4$.
Required. $O(m)$ time and $O(1)$ extra space.
Bonus (extra credit). $O(\log m)$ time, using the fact that the array is sorted (`missingNumberFast`).

Input	Expected output
`1 2 4 5 6 7`	`3`
`1 2 3 4 5 6 8 9 10`	`7`
`2`	`1`
`1`	`2`

Problem 4. A sorted array list *Medium*

An array list `slist` is kept in non-decreasing order at all times. Implement the following operations so that the order is preserved. Use *binary search* wherever the position of a value is needed. Write the helpers `lowerBound` (index of the first item $\ge x$) and `upperBound` (index of the first item $> x$) first.

(i) `insert`: insert x after the equal items; return 0 if the list is full, else 1.
(ii) `erase`: remove *one* occurrence of x; return 1 if found, else 0.
(iii) `search`: the index of the *first* occurrence of x, or -1.

Juevesano D. 2026 All rights reserved.

(iv) `count`: the number of occurrences of x.

Note. The recalled statement named `insert`, `erase` and "one more operation"; `search` is the most likely candidate, and `count` covers another common choice.

```
typedef struct { int items[100]; int size; } slist;

int lowerBound(slist *l, int x);
int upperBound(slist *l, int x);
int insert(slist *l, int x);
int erase(slist *l, int x);
int search(slist *l, int x);
int count(slist *l, int x);
```

Permitted operations. Array access to `items` and `size`. No linear search for the position of a value.
Required. Finding a position, `search` and `count`: $O(\log n)$. `insert`, `erase`: $O(\log n)$ comparisons and $O(n)$ moves.

Input	Expected output
`list 2 4 4 4 7 9: insert 5`	`2 4 4 4 5 7 9`
`list 2 4 4 4 7 9: erase 4`	`2 4 4 7 9`
`list 2 4 4 4 7 9: search 4, search 5`	`1, -1`
`list 2 4 4 4 7 9: count 4, count 9, count 3`	`3, 1, 0`

Problem 5. Rotate an array *Easy*

Given an array `a` of `n` integers, rotate it to the *right* by `k` positions, in place. For example $[1,2,3,4,5,6,7]$ with $k = 3$ becomes $[5,6,7,1,2,3,4]$. The value k may be larger than n.

```
void rotate(int a[], int n, int k);
```

Permitted operations. Array indexing and a scalar temporary. No second array.
Constraints. $0 \le n \le 10^5$, $0 \le k \le 10^9$.
Required. $O(n)$ time (shifting by one place k times costs $O(nk)$ and earns partial credit).
Bonus (extra credit). $O(1)$ extra space with three reversals.

Input	Expected output
`[1,2,3,4,5,6,7], k=3`	`[5,6,7,1,2,3,4]`
`[1,2], k=5`	`[2,1]`
`[1,2,3,4,5,6,7,8,9], k=4`	`[6,7,8,9,1,2,3,4,5]`

 Juevesano D. 2026 All rights reserved.

Problem 6. Merge two sorted arrays *Easy*

Given sorted (non-decreasing) arrays `a` of length `n` and `b` of length `m`, fill `c` with all $n + m$ items in sorted order and return $n + m$. Among equal items, those of `a` come first.

```
int mergeArrays(int a[], int n, int b[], int m, int c[]);
```

Permitted operations. Array indexing only. No sorting routine.
Constraints. $0 \leq n, m \leq 10^4$.
Required. $O(n + m)$ time.

Input	Expected output
`[1,3,5,7], [2,3,6]`	`[1,2,3,3,5,6,7]`
`[], [4,5]`	`[4,5]`
`[1,2], []`	`[1,2]`

Problem 7. Remove duplicates from a sorted array *Easy*

Given a sorted array `a` of `n` integers, keep one copy of every value, in place, in the first positions. Return the new length. The cells after the new length may hold anything.

```
int dedupSorted(int a[], int n);
```

Permitted operations. Array indexing only.
Constraints. $0 \leq n \leq 10^5$.
Required. $O(n)$ time and $O(1)$ extra space.

Input	Expected output
`[1,1,2,2,2,3,5,5]`	`length 4:  [1,2,3,5]`
`[7]`	`length 1:  [7]`
`[]`	`length 0`

Problem 8. Two-sum in a sorted array *Easy*

Given a sorted array `a` of `n` integers and a `target`, find indices $i < j$ with $a[i] + a[j] =$ target. Store them in `*pi` and `*pj` and return 1, or return 0 if there is no such pair.

```
int twoSum(int a[], int n, int target, int *pi, int *pj);
```

Permitted operations. Array indexing only (no hash table).
Constraints. $2 \leq n \leq 10^5$; values fit in `int` and so do the sums.
Required. $O(n^2)$ earns partial credit; $O(n)$ time and $O(1)$ space earns full credit.

Input	Expected output
`[1,3,4,6,8,11], target 10`	`(2,3)`
`[1,3,4,6,8,11], target 20`	`none`
`[2,2], target 4`	`(0,1)`

Juevesano D. 2026 All rights reserved.

Problem 9. Maximum subarray sum *Medium*

Given an array `a` of $n \geq 1$ integers (positive, negative or zero), return the largest sum of a non-empty contiguous subarray.

```
int maxSubarray(int a[], int n);
```

Permitted operations. Array reads only.
Constraints. $1 \leq n \leq 10^5$; $|a[i]| \leq 10^4$.
Required. $O(n^2)$ earns partial credit (try all subarrays with a running sum); $O(n)$ time and $O(1)$ space earns full credit.

Input	Expected output
`[-2,1,-3,4,-1,2,1,-5,4]`	`6 (subarray [4,-1,2,1])`
`[-3,-1,-2]`	`-1`
`[5]`	`5`

Problem 10. Move zeroes *Easy*

Given an array `a` of `n` integers, move all zeros to the end, keeping the order of the non-zero items, in place.

```
void moveZeroes(int a[], int n);
```

Permitted operations. Array indexing only.
Constraints. $0 \leq n \leq 10^5$.
Required. $O(n)$ time, $O(1)$ extra space.

Input	Expected output
`[0,1,0,3,12]`	`[1,3,12,0,0]`
`[0,0]`	`[0,0]`
`[4,5]`	`[4,5]`

Problem 11. Majority element *Easy*

Given an array `a` of `n` integers, return the *majority element*, the element that appears more than $\lfloor n/2 \rfloor$ times. It is guaranteed to exist.

```
int majority(int a[], int n);
```

Permitted operations. Array reads only. No sorting, no counting array indexed by value.
Constraints. $1 \leq n \leq 10^5$.
Required. $O(n)$ time.
Bonus (extra credit). $O(1)$ extra space (Boyer–Moore voting).

Input	Expected output
`[2,2,1,1,2,2,3,2,2]`	`2`
`[3,3,4]`	`3`
`[7]`	`7`

 Juevesano D. 2026 All rights reserved.

Problem 12. Rotate a matrix by 90 degrees *Medium*

Given an $n \times n$ matrix `a`, rotate it by 90° clockwise, in place.

```
void rotate90(int a[][10], int n);
```

Permitted operations. Array indexing only.
Constraints. $1 \le n \le 10$.
Required. $O(n^2)$ time, $O(1)$ extra space.

Input	Expected output
`[[1,2],[3,4]]`	`[[3,1],[4,2]]`
`[[1,2,3],[4,5,6],[7,8,9]]`	`[[7,4,1],[8,5,2],[9,6,3]]`

9.2 Linked Lists

Problem 13. Implementation: eraseItems (DLL) *Medium*

This is one of the implementation problems of the departmental style, reproduced with its sample input and output.

Implement the operation `void eraseItems(list *l, int x)` for a doubly linked list. Without calling `search` and `erase`, remove all items in `l` equal to `x`. The type `list` (Listing 3.3) has the fields `head`, `tail`, `size`; nodes have `item`, `next`, `prev`.

The test program reads n, then n items, then x. It prints the list before, calls your function, prints the list after, then `New size:` and the list from tail to head. When the list becomes empty it also prints a check that `head` and `tail` are both `NULL`. (The template code around your function is reconstructed from the sample output.)

```
void eraseItems(list *l, int x);
```

Permitted operations. Direct access to `head`, `tail`, `size`, `next`, `prev`. *Not* `search`, `erase`, `remove` or any helper that finds or deletes a node for you.
Required. $O(n)$ time in *one pass*; $O(1)$ extra space; `size`, `head`, `tail` and every `prev` link correct afterwards; removed nodes freed.
Bonus (extra credit). Explain why a solution that restarts from the head after every removal is $O(n^2)$ on the input 3 3 3 3 3 with $x = 3$.

Juevesano D. 2026 All rights reserved.

Input	Expected output
5 1 2 3 4 5 6	1 2 3 4 5 1 2 3 4 5 New size: 5 Displaying from tail down to head: 5 4 3 2 1
10 10 5 5 10 10 6 9 10 12 13 10	10 5 5 10 10 6 9 10 12 13 5 5 6 9 12 13 New size: 6 Displaying from tail down to head: 13 12 9 6 5 5
5 3 3 3 3 3 3	3 3 3 3 3 New size: 0 Displaying from tail down to head: Yes, correct. Both the head and tail are NULL.
10 1 1 3 4 5 6 7 8 9 10 1	1 1 3 4 5 6 7 8 9 10 3 4 5 6 7 8 9 10 New size: 8 Displaying from tail down to head: 10 9 8 7 6 5 4 3
10 1 2 3 4 5 6 7 8 9 9 9	1 2 3 4 5 6 7 8 9 9 1 2 3 4 5 6 7 8 New size: 8 Displaying from tail down to head: 8 7 6 5 4 3 2 1
5 3 6 3 3 3 3	3 6 3 3 3 6 New size: 1 Displaying from tail down to head: 6

Problem 14. Implementation: appendList (copying a list) *Medium*

Implement the function `void appendList(list *l, list m)`. The idea is simply to append the items of `m` to `l`, *without calling* `append`. Take note of the edge cases. Make sure that *when `m` is destroyed, the items appended to the list pointed to by `l` are no longer affected.* The type `list` (Listing 3.3) has the fields `head`, `tail`, `size`; nodes have `item`, `next`, `prev`.

The test program reads n and the items of `l`, then $|m|$ and the items of `m`; it prints `list l:`, `l`, `list m:`, `m`, calls the function, destroys `m`, and prints `l`.

```
void appendList(list *l, list m);
```

Permitted operations. Direct access to `head`, `tail`, `size`, `next`, `prev`; `malloc`. Not `append`.
Required. $O(|m|)$ time; the nodes of `l` must be new nodes, never nodes of `m`.
Bonus (extra credit). The call `appendList(&x, x)` (a list appended to itself) must terminate and double the list. Say why a loop *until `NULL`* fails here and a counted loop does not.

 Juevesano D. 2026 All rights reserved.

Input	Expected output
5 1 2 3 4 5 3 9 10 11	list l: 1 2 3 4 5 list m: 9 10 11 1 2 3 4 5 9 10 11
1 10 5 9 8 7 6 5	list l: 10 list m: 9 8 7 6 5 10 9 8 7 6 5
5 5 4 3 2 1 0	list l: 5 4 3 2 1 list m: 5 4 3 2 1
0 5 1 2 3 4 5	list l: list m: 1 2 3 4 5 1 2 3 4 5
10 1 2 3 4 5 6 7 8 9 10 10 11 12 13 14 15 16 17 18 19 20	list l: 1 2 3 4 5 6 7 8 9 10 list m: 11 12 13 14 15 16 17 18 19 20 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Problem 15. Implementation: recursive append with a double pointer *Medium*

Assume the singly linked list has no tail pointer. Implement `append(snode **h, int x)`, which adds a node with item `x` at the end of the list. Notice the double pointer: it lets the function change the real pointer and not a copy. `h` is the address of the pointer that heads the list. Implement it *recursively*. In `main` it is called as `append(&l.head, item)`: since the head must change when the list is empty, what is passed is the address of the head and not the head. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

```
void append(snode **h, int x);   // called as append(&l.head, item)
```

Permitted operations. `malloc`; the fields `item` and `next`. No loops in `append`, no extra helper that returns a new head.

Required. $O(n)$ time, recursive.

Bonus (extra credit). Implement the same operation with a loop but still with a `snode **` (one variable, no special case for the empty list). State the recursion depth and when it is dangerous.

Juevesano D. 2026 All rights reserved.

Input	Expected output
5 1 2 3 4 5	1 2 3 4 5
1 109	109
10 10 9 8 7 6 5 4 3 2 1	10 9 8 7 6 5 4 3 2 1

Problem 16. Reverse a singly linked list *Easy*

Given an SLL, reverse it in place by changing links: do not allocate nodes and do not copy items. Implement the iterative `reverseList`, which must also fix `head` and `tail`. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

```
void reverseList(sll *l);          // iterative; fix head and tail
snode *reverseRec(snode *h);       // bonus: recursive, returns the new head
```

Permitted operations. Pointer manipulation of `next`, `head`, `tail`.
Constraints. $0 \le |l| \le 10^5$.
Required. $O(n)$ time, $O(1)$ extra space (iterative).
Bonus (extra credit). The recursive version `reverseRec`; explain its stack use.

Input	Expected output
[1,2,3,4,5]	[5,4,3,2,1]
[]	[]
[7]	[7]

Problem 17. Middle node and k-th node from the end *Easy*

Given an SLL, implement (a) `middleItem`, which returns the middle item (the *second* middle when n is even; the list is non-empty), and (b) `kthFromEnd`, which stores the k-th item from the end in `*out` and returns 1, or returns 0 if $k < 1$ or $k > n$. Both must work in *one pass* and without using `size`. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

```
int middleItem(sll *l);                 // second middle if n is even
int kthFromEnd(sll *l, int k, int *out);  // k = 1 is the last item
```

Permitted operations. Pointer traversal only. You may not read `size`, count the nodes first, or copy the list into an array.
Constraints. $1 \le n \le 10^5$.
Required. $O(n)$ time, one traversal, $O(1)$ extra space.
Bonus (extra credit). Return the *first* middle for even n by a change of one condition, and explain the change.

 Juevesano D. 2026 All rights reserved.

Input	Expected output
`middle [1,2,3,4]`	`3`
`middle [1,2,3,4,5]`	`3`
`kth [10,20,30,40,50], k=2`	`40`
`kth [10,20,30,40,50], k=5 / k=6 / k=0`	`10 / fails / fails`

Problem 18. Splicing singly linked lists *Medium*

Splicing moves *nodes* from one list to another by relinking, without allocating or copying. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

(i) `spliceAfter`: move *all* nodes of `m` into `l`, immediately after node `p` (`p==NULL` means "at the very front"). Afterwards `m` is empty; `head`, `tail` and both sizes must be correct.

(ii) `splice`: the same, but the place is an index: `pos` nodes of `l` stay in front of the spliced part ($0 \le pos \le |l|$). Return 0 if `pos` is invalid.

```
void spliceAfter(sll *l, snode *p, sll *m);   // p == NULL: at the front
int splice(sll *l, int pos, sll *m);              // 0 if pos is invalid
```

Permitted operations. Pointer relinking only. No `malloc`, no `free`, no item copying.
Required. `spliceAfter`: $O(1)$. `splice`: $O(pos)$ to walk to the anchor, then $O(1)$.
Bonus (extra credit). Show that, given a tail pointer, appending a whole list to another list costs $O(1)$ whereas an array list needs $\Theta(|m|)$.

Input	Expected output
`l=[1,2,5], m=[3,4], splice at pos 2`	`l=[1,2,3,4,5] m=[] sizes 5 0`
`l=[1,2,5], m=[3,4], splice at pos 0`	`l=[3,4,1,2,5]`
`l=[1,2,5], m=[3,4], splice at pos 3`	`l=[1,2,5,3,4] (tail is 4)`
`m empty`	`l unchanged`

Problem 19. Merge two sorted doubly linked lists *Medium*

Two doubly linked lists `a` and `b` are sorted in non-decreasing order. Implement `mergeSorted` so that afterwards `a` holds all the nodes in sorted order and `b` is empty. Relink the existing nodes (no allocation, no freeing); on equal items the node of `a` goes first; all `prev` links, `head`, `tail` and `size` must be correct. The types are those of Listing 3.3: a node has `item`, `next`, `prev`, and `list` has `head`, `tail`, `size`.

```
void mergeSorted(list *a, list *b);
```

Permitted operations. Pointer manipulation of `next`, `prev`, `head`, `tail`, `size`.
Required. $O(|a| + |b|)$ time, $O(1)$ extra space.
Bonus (extra credit). Attach the remainder of the non-empty list in $O(1)$ instead of node by node.

Juevesano D. 2026 All rights reserved.

Input	Expected output
`a=[1,4,7], b=[2,4,5,9]`	`a=[1,2,4,4,5,7,9] size 7; backward: 9 7 5 4 4 2 1; b.size=0`
`a=[], b=[3]`	`a=[3]`
`a=[1], b=[]`	`a=[1]`

Problem 20. Palindromic doubly linked list *Easy*

Given a doubly linked list, return 1 if its items read the same forwards and backwards, else 0. The empty list and a one-element list are palindromes. The types are those of Listing 3.3: a node has `item`, `next`, `prev`, and `list` has `head`, `tail`, `size`.

```
int isPalindrome(list *l);
```

Permitted operations. Pointer traversal. Not an auxiliary array or stack.
Constraints. $0 \le n \le 10^5$.
Required. $O(n)$ time, $O(1)$ extra space.
Bonus (extra credit). Stop after $\lfloor n/2 \rfloor$ comparisons.

Input	Expected output
`[1,2,3,2,1]`	`1`
`[1,2,3,4]`	`0`
`[]`	`1`
`[7]`	`1`
`[4,5,5,4]`	`1`

Problem 21. Reverse a doubly linked list *Easy*

Given a doubly linked list, reverse it in place. The types are those of Listing 3.3: a node has `item`, `next`, `prev`, and `list` has `head`, `tail`, `size`.

```
void reverseDLL(list *l);
```

Permitted operations. Pointer manipulation only.
Required. $O(n)$ time, $O(1)$ space.
Bonus (extra credit). Do it by swapping `next` and `prev` in every node, without moving any item.

Input	Expected output
`[1,2,3,4,5]`	`forward 5 4 3 2 1; backward 1 2 3 4 5`
`[]`	`[]`

 Juevesano D. 2026 All rights reserved.

Problem 22. Remove duplicates from a sorted doubly linked list *Easy*

Given a doubly linked list sorted in non-decreasing order, remove nodes so that each value appears once (keep the first node of each run). Free the removed nodes and keep `tail` and `size` correct. The types are those of Listing 3.3: a node has `item`, `next`, `prev`, and `list` has `head`, `tail`, `size`.

```
void removeDuplicates(list *l);
```

Permitted operations. Pointer manipulation only; no `search` or `erase`.
Required. $O(n)$ time, $O(1)$ space.
Bonus (extra credit). Remove *all* nodes whose value occurs more than once.

Input	Expected output
`[1,1,2,3,3,3,5]`	`[1,2,3,5], size 4`
`[2,2,2]`	`[2]`
`[]`	`[]`

Problem 23. Insert into a sorted doubly linked list *Easy*

Given a doubly linked list sorted in non-decreasing order, insert x so that the list stays sorted (after equal items). Handle the empty list, the new minimum and the new maximum. The types are those of Listing 3.3: a node has `item`, `next`, `prev`, and `list` has `head`, `tail`, `size`.

```
void insertSorted(list *l, int x);
```

Permitted operations. Allocation, `next`/`prev`, `head`/`tail`. No call to `append` except where it is the correct case.
Required. $O(n)$ time.
Bonus (extra credit). $O(1)$ when x is not smaller than the tail item.

Input	Expected output
`[2,4,4,8], insert 1`	`[1,2,4,4,8]`
`then insert 5`	`[1,2,4,4,5,8]`
`then insert 9`	`[1,2,4,4,5,8,9] (tail 9, size 7)`
`then insert 4`	`[1,2,4,4,4,5,8,9]`
`[], insert 3`	`[3]`

Problem 24. An SLL with a middle pointer *Medium*

A singly linked list `mlist` keeps, besides `head`, `tail` and `size`, a pointer `mid`. The *invariant* is

$$\texttt{size} = 0 \;\Rightarrow\; \texttt{head} = \texttt{mid} = \texttt{tail} = \texttt{NULL},$$
$$\texttt{size} > 0 \;\Rightarrow\; \texttt{mid} \text{ points to the node at index } \lfloor (\texttt{size} - 1)/2 \rfloor \;\; (0\text{-based}).$$

Implement `middle`, `mPushBack`, `mPushFront`, `mPopFront` and `mPopBack` so that every operation restores the invariant.

Juevesano D. 2026 All rights reserved.

```
typedef struct mnode { int item; struct mnode *next; } mnode;
typedef struct { mnode *head, *mid, *tail; int size; } mlist;

void minit(mlist *l);
int  middle(mlist *l);                 // O(1)
void mPushBack(mlist *l, int x);        // O(1)
int  mPopFront(mlist *l, int *x);       // O(1); 0 if empty
void mPushFront(mlist *l, int x);       // O(1) or O(n), see below
int  mPopBack(mlist *l, int *x);        // O(n)
```

Permitted operations. Pointer manipulation. A node has only `item` and `next`.
Required. `middle`, `mPushBack`, `mPopFront`: $O(1)$. For `mPushFront` and `mPopBack` state the worst case.
Bonus (extra credit). Explain why a *doubly* linked list would make all four end operations $O(1)$.

Input	Expected output
`pushBack 10,20,...,70 (sizes 1..7)`	`mid after each:  10 10 20 20 30 30 40`
`10,20,30,40; pushFront 5; pushFront 1`	`mid 20; then mid 10 (list 1 5 10 20 30 40)`
`then:  popFront; popBack; insertAt(2,15); eraseAt(3)`	`mid 20; 10; 15; 10 (list 5 10 15 30)`

Problem 25. Rotate a linked list *Medium*

Given an SLL and an integer $k \geq 0$, rotate the list to the *left* by k positions: $[1,2,3,4,5]$ with $k = 2$ becomes $[3,4,5,1,2]$. k may exceed n. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

```
void rotateLeft(sll *l, int k);
```

Permitted operations. Pointer manipulation; `size` is available.
Constraints. $0 \leq n \leq 10^5$, $0 \leq k \leq 10^9$.
Required. $O(n)$ time.
Bonus (extra credit). No node is allocated or freed.

Input	Expected output
`[1,2,3,4,5], k=2`	`[3,4,5,1,2]`
`[1,2,3,4,5], k=7`	`[3,4,5,1,2]`
`[1,2,3,4,5], k=5`	`[1,2,3,4,5]`
`[], k=3`	`[]`

 Juevesano D. 2026 All rights reserved.

Problem 26. Partition a linked list around a value *Medium*

Given an SLL and a value x, rearrange the nodes so that all nodes with item $< x$ precede all nodes with item $\geq x$, keeping the original relative order inside each group. Relink nodes; do not allocate. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

```
void partitionList(sll *l, int x);
```

Permitted operations. Pointer manipulation only.
Required. $O(n)$ time, one pass.
Bonus (extra credit). $O(1)$ extra space using two dummy nodes.

Input	Expected output
`[1,4,3,2,5,2], x=3`	`[1,2,2,4,3,5]`
`[5,6], x=3`	`[5,6]`
`[1,2], x=3`	`[1,2]`
`[], x=3`	`[]`

Problem 27. Remove the n-th node from the end *Medium*

Given an SLL, remove the n-th node from the end ($n = 1$ is the last node) in *one pass*. Return 1, or 0 if $n < 1$ or $n > |l|$. Keep `head`, `tail` and `size` correct and free the node. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

```
int removeNthFromEnd(sll *l, int n);   // n = 1 is the last node
```

Permitted operations. Pointer manipulation; do not use `size` to compute the position.
Required. $O(|l|)$ time, one pass.
Bonus (extra credit). Handle the removal of the head with a dummy node and no special case.

Input	Expected output
`[1,2,3,4,5], n=2`	`[1,2,3,5]`
`[1,2,3,5], n=4`	`[2,3,5] (size 3)`
`[2,3,5], n=1`	`[2,3] (tail 3)`
`[2,3], n=3`	`fails (0), list unchanged`
`[7], n=1`	`[] (head, tail NULL, size 0)`

Problem 28. Swap nodes in pairs *Medium*

Given an SLL, swap every two adjacent *nodes*: $[1, 2, 3, 4] \to [2, 1, 4, 3]$. A final unpaired node stays where it is. Swap nodes, not items. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

```
void swapPairs(sll *l);
```

Permitted operations. Pointer manipulation only.
Required. $O(n)$ time, $O(1)$ space.
Bonus (extra credit). The same with recursion; also keep `tail` correct.

Juevesano D. 2026 All rights reserved.

Input	Expected output
`[1,2,3,4]`	`[2,1,4,3]`
`[1,2,3,4,5]`	`[2,1,4,3,5] (tail 5)`
`[1]`	`[1]`

Problem 29. Cycle detection *Medium*

Given the head of a singly linked list whose last node may point back to an earlier node, return 1 if the list contains a cycle and 0 otherwise. The list is not modified. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

```
int hasCycle(snode *head);             // 1 if the list has a cycle
snode *cycleStart(snode *head);       // bonus: first node of the cycle, or NULL
```

Permitted operations. Pointer traversal. No visited-marker field, no auxiliary array or table.
Constraints. $0 \leq n \leq 10^5$.
Required. $O(n)$ time and $O(1)$ extra space.
Bonus (extra credit). Return the first node of the cycle (or `NULL`), in $O(n)$ time and $O(1)$ space.

Input	Expected output
`[1,2,3,4,5], last node points to the 3rd node`	`1 (cycle starts at item 3)`
`[1,2,3,4,5], last node points to the head`	`1 (starts at item 1)`
`[1,2,3,4,5], no link back`	`0`
`[] (head is NULL)`	`0`

Problem 30. Group odd and even positions *Medium*

Given an SLL, group all nodes at odd positions (1st, 3rd, ...) first, followed by all nodes at even positions, each group keeping its order: $[1,2,3,4,5] \to [1,3,5,2,4]$. The positions are meant, *not* the values. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

```
void oddEvenList(sll *l);
```

Permitted operations. Pointer manipulation only.
Required. $O(n)$ time, $O(1)$ extra space.
Bonus (extra credit). Keep `tail` correct.

Input	Expected output
`[1,2,3,4,5]`	`[1,3,5,2,4] (tail 4)`
`[1,2,3,4]`	`[1,3,2,4]`
`[1,2]`	`[1,2]`

 Juevesano D. 2026 All rights reserved.

Problem 31. Add two numbers stored in linked lists *Medium*

Two non-negative integers are stored one digit per node, *least significant digit first*: 342 is $2 \to 4 \to 3$. Given the lists `a` and `b`, build in `out` the list of their sum, in the same form. The lists may have different lengths and there may be a final carry. The types are those of Listing 3.1: a node `snode` has `item` and `next`, and `sll` has `head`, `tail`, `size`.

```
void addNumbers(sll *a, sll *b, sll *out);
```

Permitted operations. `sinit` and `pushBack` on the result. No conversion to a machine integer (the numbers may have hundreds of digits).
Required. $O(\max(|a|, |b|))$ time.
Bonus (extra credit). The case in which the digits are stored most significant first.

Input	Expected output
`[2,4,3] + [5,6,4]`	`[7,0,8]`
`[9,9] + [1]`	`[0,0,1]`
`[0] + [0]`	`[0]`

9.3 Stacks and Queues

Problem 32. Valid brackets *Easy*

Given a string `s` containing only the characters `(`, `)`, `[`, `]`, `{`, `}`, return 1 if every bracket is closed by the same kind of bracket and in the correct order, else 0. The stack (Listing 4.1) offers `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; the queue offers `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`. Pop, top and dequeue return 1 on success and 0 when empty.

```
int isBalanced(char s[]);
```

Permitted operations. Stack operations only (push, pop, top, empty). No indexing into the stack.
Constraints. $0 \le |s| \le 10^4$.
Required. $O(n)$ time.

Input	Expected output
`{[()]}`	`1`
`([)]`	`0`
`((`	`0`
`))`	`0`
`(empty string)`	`1`
`()[]{}`	`1`

Juevesano D. 2026 All rights reserved.

Problem 33. Infix to postfix and postfix evaluation *Medium*

(a) Given an infix expression `in` made of single-digit operands, the operators `+ - * /` and parentheses (no spaces), write its postfix form into `out`. The operators `*` and `/` have higher precedence than `+` and `-`; operators of equal precedence are left-associative.

(b) Given a valid postfix expression `p` of single digits and `+ - * /`, return its value (integer division). The stack (Listing 4.1) offers `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; the queue offers `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`. Pop, top and dequeue return 1 on success and 0 when empty.

```
void toPostfix(char in[], char out[]);
int evalPostfix(char p[]);
```

Permitted operations. Stack operations only.
Constraints. The input is valid; $|in| \leq 100$.
Required. $O(n)$ time for both.

Input	Expected output
`2+3*4`	`234*+`
`(2+3)*4`	`23+4*`
`8-3-2`	`83-2-`
`8/(4/2)`	`842//`
`evalPostfix 234*+`	`14`
`evalPostfix 83-2-`	`3`

Problem 34. A queue from two stacks *Medium*

Implement a queue using *only two stacks*: `tsqPush` adds an item at the back, `tsqPop` removes the front item into `*x`, `tsqPeek` copies it without removing it, and `tsqEmpty` tells whether the queue is empty. Pop and peek return 0 on an empty queue. The stack (Listing 4.1) offers `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; the queue offers `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`. Pop, top and dequeue return 1 on success and 0 when empty.

```
typedef struct { stack in; stack out; } twoStackQueue;

void tsqInit(twoStackQueue *q);
void tsqPush(twoStackQueue *q, int x);
int  tsqPop(twoStackQueue *q, int *x);
int  tsqPeek(twoStackQueue *q, int *x);
int  tsqEmpty(twoStackQueue *q);
```

Permitted operations. Stack operations on the two stacks `in` and `out`. No array, no list.
Required. Each operation $O(1)$ *amortised*.

Input	Expected output
`push 1, push 2, push 3, pop`	`1`
`push 4, pop, pop, pop`	`2, 3, 4`
`pop on an empty queue`	`fails (0)`

 Juevesano D. 2026 All rights reserved.

Problem 35. A stack with constant-time minimum *Medium*

Design a stack of integers that supports `msPush`, `msPop` and `msMin` (the smallest item currently stored), all in $O(1)$. Pop and min return 0 on an empty stack. The stack (Listing 4.1) offers `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; the queue offers `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`. Pop, top and dequeue return 1 on success and 0 when empty.

```
typedef struct { stack all; stack mins; } minStack;

void msInit(minStack *m);
void msPush(minStack *m, int x);
int  msPop(minStack *m, int *x);
int  msMin(minStack *m, int *x);
```

Permitted operations. Stack operations only (two stacks are given).
Required. All operations $O(1)$.

Input	Expected output
`push 5, 3, 7, 3, 1; min`	`1`
`pop; min`	`3`
`pop, pop; min`	`3`
`pop; min`	`5`

Problem 36. Next greater element *Medium*

Given an array `a` of `n` integers, for every position i find the first value to the right of i that is strictly greater than $a[i]$, or -1 if there is none. Store the answers in `out`. The stack (Listing 4.1) offers `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; the queue offers `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`. Pop, top and dequeue return 1 on success and 0 when empty.

```
void nextGreater(int a[], int n, int out[]);
```

Permitted operations. Stack operations (the stack holds indices).
Constraints. $1 \le n \le 10^4$.
Required. $O(n^2)$ earns partial credit; $O(n)$ earns full credit.

Input	Expected output
`[2,1,5,3]`	`[5,5,-1,-1]`
`[4,5,2,25]`	`[5,25,25,-1]`
`[3,2,1]`	`[-1,-1,-1]`

Juevesano D. 2026 All rights reserved.

Problem 37. Josephus problem *Medium*

n people numbered $1, \ldots, n$ stand in a circle. Starting at person 1, every k-th person is removed, going around the circle, until one person remains. Return the number of the survivor. The stack (Listing 4.1) offers `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; the queue offers `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`. Pop, top and dequeue return 1 on success and 0 when empty.

```
int josephus(int n, int k);
```

Permitted operations. Queue operations only.
Constraints. $1 \leq n \leq 1000$, $1 \leq k \leq 100$.
Required. $O(nk)$ time (simulation).
Bonus (extra credit). `josephusFast`: $O(n)$ time and $O(1)$ space with a recurrence.

Input	Expected output
`n=5, k=2`	`3`
`n=7, k=3`	`4`
`n=6, k=3`	`1`
`n=10, k=2`	`5`
`n=41, k=3`	`31`

Problem 38. Reverse the first k items of a queue *Medium*

Given a queue `q` and an integer k with $0 \leq k \leq$ the size of the queue, reverse the order of the first k items and leave the others in order. The stack (Listing 4.1) offers `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; the queue offers `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`. Pop, top and dequeue return 1 on success and 0 when empty.

```
void reverseFirstK(queue *q, int k);
```

Permitted operations. Queue operations and *one* auxiliary stack. No array.
Required. $O(n)$ time.

Input	Expected output
`queue 1 2 3 4 5, k=3`	`3 2 1 4 5`
`queue 1 2 3 4, k=4`	`4 3 2 1`
`queue 1 2 3, k=1`	`1 2 3`

Problem 39. Sort a stack *Medium*

Given a stack of integers, sort it so that the *smallest* item is on top (popping gives $1, 2, 3, \ldots$). Use *one* additional stack and a few scalar variables. No array, no recursion. The stack (Listing 4.1) offers `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; the queue offers `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`. Pop, top and dequeue return 1 on success and 0 when empty.

```
void sortStack(stack *s);
```

Permitted operations. Stack operations only.
Constraints. $0 \leq n \leq 100$.

 Juevesano D. 2026 All rights reserved.

Required. $O(n^2)$ time.

Input	Expected output
`push 3, 1, 4, 1, 5, 9, 2, 6`	`pops: 1 1 2 3 4 5 6 9`
`empty stack`	`unchanged`

Problem 40. Remove adjacent duplicates *Easy*

Given a string `s` of lowercase letters, repeatedly delete two equal adjacent letters until none remain, and write the result into `out`: `abbaca`→`aaca`→`ca`. The stack (Listing 4.1) offers `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; the queue offers `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`. Pop, top and dequeue return 1 on success and 0 when empty.

```
void removeAdjacent(char s[], char out[]);
```

Permitted operations. Stack operations only (the stack holds characters).
Constraints. $0 \le |s| \le 10^4$.
Required. $O(n)$ time in one pass.

Input	Expected output
`abbaca`	`ca`
`azxxzy`	`ay`
`aaaa`	`(empty)`
`abc`	`abc`

Problem 41. A stack from a queue *Medium*

Implement a stack using a *single* queue: `qsPush`, `qsPop` (into `*x`), `qsTop` and `qsEmpty`. Pop and top return 0 on an empty stack. The stack (Listing 4.1) offers `stInit`, `stPush`, `stPop`, `stTop`, `stEmpty`, `stSize`; the queue offers `qInit`, `qEnq`, `qDeq`, `qFront`, `qEmpty`, `qSize`. Pop, top and dequeue return 1 on success and 0 when empty.

```
typedef struct { queue q; } queueStack;

void qsInit(queueStack *s);
void qsPush(queueStack *s, int x);
int  qsPop(queueStack *s, int *x);
int  qsTop(queueStack *s, int *x);
int  qsEmpty(queueStack *s);
```

Permitted operations. Queue operations only.
Required. Correctness; either push or pop may be $O(n)$.

Input	Expected output
`push 1, 2, 3; top`	`3`
`pop; push 4; pop`	`3, then 4`
`pop, pop`	`2, 1`
`pop on an empty stack`	`fails (0)`

Juevesano D. 2026 All rights reserved.

9.4 Binary Trees and Binary Search Trees

Problem 42. Tree measures *Easy*

Given the root of a binary tree, implement the recursive functions `countNodes`, `height` (number of *edges* on a longest downward path; the empty tree has height -1), `countLeaves` and `sumItems`. The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

```
int countNodes(node *r);
int height(node *r);
int countLeaves(node *r);
int sumItems(node *r);
```

Permitted operations. Recursion on `left` and `right`.
Constraints. $0 \le n \le 10^4$.
Required. $O(n)$ time for each.

Input	Expected output
`S1`	`count 9, height 3, leaves 4, sum 66`
`empty tree`	`count 0, height -1, leaves 0, sum 0`
`a single node 5`	`count 1, height 0, leaves 1, sum 5`

Problem 43. Level order printing *Medium*

Given the root of a binary tree, print its items level by level: one line per level, from left to right, items separated by single spaces. The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

```
void printLevels(node *root);
```

Permitted operations. The queue of node pointers `nqueue` of Listing 5.1.
Constraints. $0 \le n \le 10^3$.
Required. $O(n)$ time.

Input	Expected output

`S1`	`8`
	`3 10`
	`1 6 14`
	`4 7 13`

`empty tree`	`(prints nothing)`

 Juevesano D. 2026 All rights reserved.

Problem 44. VALIDATE A BINARY SEARCH TREE *Medium*

Given the root of a binary tree, return 1 if it is a BST and 0 otherwise. In our BST every item in the left subtree of a node is *smaller* than the node's item, and every item in its right subtree is *greater than or equal to* it, at *every* node. The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

```
bool isBST(node *root);
```

PERMITTED OPERATIONS. Recursion; you may write a helper with extra parameters.
CONSTRAINTS. $0 \leq n \leq 10^4$.
REQUIRED. $O(n)$ time.

INPUT	EXPECTED OUTPUT
`S1`	`true`
`root 5 whose left child is 5`	`false`
`root 5 whose right child is 5`	`true`
`root 5, left child 3 whose right child is 6`	`false`
`root 5, left child 3 whose right child is 4`	`true`

Problem 45. INORDER SUCCESSOR AND PREDECESSOR *Medium*

Given a node `x` of a BST (nodes have a `parent` pointer), return the node that comes right after it in inorder (`successor`) or right before it (`predecessor`), or `NULL` if there is none. Do not use the root and do not traverse the whole tree. The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

```
node *successor(node *x);
node *predecessor(node *x);
```

PERMITTED OPERATIONS. The pointers `left`, `right`, `parent`.
REQUIRED. $O(h)$ time.

INPUT	EXPECTED OUTPUT
`S1: successor of 8, 7, 14, 13`	`10, 8, none, 14`
`S1: predecessor of 8, 1, 10`	`7, none, 8`

Problem 46. K-TH SMALLEST ITEM OF A BST *Medium*

Given the root of a BST and an integer k, store the k-th smallest item ($k = 1$ is the smallest) in `*out` and return 1, or return 0 if $k < 1$ or k is larger than the number of nodes. The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

```
int kthSmallest(node *root, int k, int *out);
```

PERMITTED OPERATIONS. The stack of node pointers `nstack` (Listing 5.1).
REQUIRED. $O(h + k)$ time.

Juevesano D. 2026 All rights reserved.

Input	Expected output
`S1, k=1`	`1`
`S1, k=4`	`6`
`S1, k=9`	`14`
`S1, k=10`	`fails (0)`

Problem 47. Lowest common ancestor in a BST *Easy*

Given the root of a BST with distinct items and two items `p` and `q` that are in the tree, return the lowest node that has both as descendants (a node counts as a descendant of itself). The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

```
node *lowestCommonAncestor(node *root, int p, int q);
```

Permitted operations. A loop or recursion. Do not search for the two nodes first.
Required. $O(h)$ time, $O(1)$ extra space.

Input	Expected output
`S1, (1,7)`	`3`
`S1, (4,7)`	`6`
`S1, (4,13)`	`8`
`S1, (3,7)`	`3`
`S1, (13,14)`	`14`

Problem 48. Mirror, same tree, symmetric, path sum *Medium*

Implement the following recursive functions on binary trees. The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

(i) `mirror`: swap the left and right child of every node.

(ii) `sameTree`: return 1 if two trees have the same shape and the same items.

(iii) `isSymmetric`: return 1 if the left subtree of the root is the mirror image of the right subtree.

(iv) `hasPathSum`: return 1 if some path from the root to a *leaf* has items adding up to `target`.

```
void mirror(node *r);
bool sameTree(node *a, node *b);
bool isSymmetric(node *root);
bool hasPathSum(node *r, int target);
```

Permitted operations. Recursion on `left` and `right`.
Required. $O(n)$ time each.

 Juevesano D. 2026 All rights reserved.

Input	Expected output
`mirror of S1, preorder`	`8 10 14 13 3 6 7 4 1`
`tree 1 (2 (3,4)) (2 (4,3)):`	`true`
`isSymmetric`	
`the same tree with one leaf`	`false`
`changed to 9`	
`S1:  hasPathSum 12 / 21 / 11`	`true / true / false`

Problem 49. Sorted array to balanced BST *Medium*

Given a sorted array `a` of `n` distinct integers, build a BST containing them whose left and right subtrees of every node differ in height by at most 1, and return its root. Set the `parent` pointers too. The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

```
node *sortedToBST(int a[], int n);
```

Permitted operations. The node constructor and recursion. No `insert`.
Constraints. $0 \leq n \leq 10^4$.
Required. $O(n)$ time.

Input	Expected output
`[1,2,3,4,5,6,7]`	`level order 4 / 2 6 / 1 3 5 7; height 2`
`[1,3,5,7]`	`root 3, height 2`
`[]`	`empty tree`

Problem 50. Range count, floor and ceiling *Medium*

Given the root of a BST: (a) `rangeCount` returns the number of items x with $lo \leq x \leq hi$; (b) `floorKey` stores the largest item $\leq x$ in `*out` and returns 1, or returns 0 if there is none; (c) `ceilKey` stores the smallest item $\geq x$ in `*out` and returns 1, or returns 0 if there is none. The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

```
int rangeCount(node *r, int lo, int hi);
int floorKey(node *root, int x, int *out);
int ceilKey(node *root, int x, int *out);
```

Permitted operations. The BST property must be used: do not visit subtrees that cannot contain an answer.
Required. `floorKey`, `ceilKey`: $O(h)$. `rangeCount`: $O(h + m)$ where m is the answer.

Input	Expected output
`S1, range [5,13]`	`5 (6,7,8,10,13)`
`S1, floor 5 / ceil 5`	`4 / 6`
`S1, floor 0 / ceil 15`	`none / none`
`S1, ceil 11`	`13`

Juevesano D. 2026 All rights reserved.

Problem 51. Inorder traversal without recursion *Medium*

Given the root of a binary tree, print its items in inorder, each followed by a space, *without recursion*. The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

```
void inorderIterative(node *root);
```

Permitted operations. The stack of node pointers `nstack` (Listing 5.1).
Required. $O(n)$ time.

Input	Expected output
`S1`	`1 3 4 6 7 8 10 13 14`

Problem 52. Remove a key from a BST *Medium*

Given the address of the root pointer of a BST and a key `x`, remove one node with item `x`. Return 1 if a node was removed, 0 if there is none. Keep the BST property and the `parent` pointers correct. If the node has two children, replace its item by the item of its successor (the smallest item of the right subtree) and remove that node instead. The node class has `item`, `left`, `right`, `parent` (Listing 5.1). The tree S_1 is that of Figure 5.1, built by inserting $8, 3, 10, 1, 6, 14, 4, 7, 13$ into an empty BST.

```
int removeKey(node **root, int x);
```

Permitted operations. Pointer manipulation; the root pointer is passed by address because the root itself may be removed.
Required. $O(h)$ time.

Input	Expected output
`BST of 50 30 70 20 40 60 80:` `remove 20`	`preorder 50 30 40 70 60 80`
`then remove 30`	`50 40 70 60 80`
`then remove 50`	`60 40 70 80`
`then remove 99`	`0 (tree unchanged)`

 Juevesano D. 2026 All rights reserved.

Part III

Solutions and Explanations

Juevesano D. 2026 All rights reserved.

Chapter 10

Solutions to Section A

Model answers to the concept questions of Chapter 7; the numbers agree.

1. An *abstract data type* is a mathematical model consisting of a set of values and a set of operations on them; the representation is hidden. A *data structure* is a concrete way to realise an ADT in memory (an array, linked nodes), and it determines the costs of the operations. Relatives of the List ADT: the *stack*, the *queue* and the *deque*.

2. (a) *True.* For $n \geq 2$, $\log_2 n \geq 1$, so $23 \leq 23 \log_2 n$; take $c = 23$, $n_0 = 2$. (b) *False.* Suppose $5 \log_2 n \leq c$ for all $n \geq n_0$; for $n > \max(n_0, 2^{c/5})$ we get $5 \log_2 n > c$, a contradiction; the function is unbounded. (c) *True.* For $n \geq 1$, $5n - 7 \leq 5n$; take $c = 5$, $n_0 = 1$.

3. (a) $\Theta(1)$. (b) $\Theta(n)$ (every item moves). (c) $\Theta(1)$. (d) $\Theta(n)$ (the predecessor of the tail must be found from the head). (e) $\Theta(1)$. (f) $\Theta(n)$ in the worst case: the cost is $\Theta(\min(i, n - i))$, maximal near the middle. (g) $\Theta(n)$. (h) $\Theta(n)$, the height of a degenerate tree.

4. A struct containing an array is copied *with* the array, so the callee changes its own copy: the caller sees **no change**. For a struct holding a pointer `head`, the pointer is copied but the nodes are shared: `l.head->item=100` **is visible** to the caller, while an assignment such as `l.head=NULL` is not. A raw array parameter `int a[]` is really a pointer to the first element, so the change **is visible**. (See Tracing 2.)

5. It is not a bug. The valid items are exactly `items[0..size-1]`; `size` is the boundary, and cells beyond it are unspecified. The stale value matters only when some code reads beyond `size` without checking (for instance `itemAt(l,p)` that indexes without testing $p \leq size$, returning the leftover value), or when the cell owns a resource (e.g. a pointer that was not released). The next `push_back` simply overwrites it. (Tracing 4.)

6. `malloc/free` are C library functions that allocate raw bytes (returning `void *`) and run no constructor; `new/delete` are C++ operators that are type-aware and call constructors/destructors (`new[]` pairs with `delete[]`); the two families must not be mixed. A *dangling pointer* points to memory that has been freed or has gone out of scope: `int *p=(int*)malloc(4); free(p); *p=1;`. A *memory leak* is allocated memory that is no longer reachable: `int *p=(int*)malloc(4); p=NULL;`.

7. The parameter `h` is a *copy* of the caller's head pointer. For a non-empty list the loop follows the copy to the real nodes and modifies them, which works; for an empty list the function assigns the new node to the *local* copy only, so the caller's head stays `NULL` and the node leaks. Repairs: (1) pass the address, `void append(node **h,int x);` (2) return the (possibly new) head and write `l.head = append(l.head,x)`. (Tracing 7.)

 Juevesano D. 2026 All rights reserved.

8. `insert(p,x)` moves the $s - p$ items at positions $p..s-1$ one place to the right; `erase(p)` moves the $s - p - 1$ items at positions $p+1..s-1$ one place to the left. Both are worst at $p = 0$: s and $s-1$ moves, i.e. $\Theta(n)$; at the right end they need 0 moves.

9. With doubling, the copying done at sizes $1, 2, 4, \ldots, 2^k < n$ totals less than $2n$, plus n writes: at most $3n$ operations, hence $O(1)$ amortised (Theorem 2.3). Growing by a constant $c = 10$ copies $10, 20, 30, \ldots$ items, a total of $10(1 + 2 + \cdots + n/10) = \Theta(n^2)$, i.e. $\Theta(n)$ amortised per `push_back`.

10. Suppose the array has capacity $2m$ and holds m items. Halving gives capacity m, completely full: the next `push` must double again, and the next `pop` halves again. Alternating push/pop costs $\Theta(n)$ per operation (thrashing). The safe rule is to shrink to $cap/2$ only when $n \leq cap/4$ (or $\leq cap/3$): after the shrink the array is half full, and $\Omega(n)$ cheap operations must occur before the next resize.

11. The array must be sorted (non-decreasing); the search takes $\Theta(\log n)$ comparisons. `insert` still needs to open a gap in a contiguous array: the items after the position are shifted, up to n of them, so the worst case is $\Theta(n)$. The $\log n$ saving is for comparisons only.

12. Front: `a[j]`. Back: `a[(j+n-1)%CAP]`. Next free cell: `a[(j+n)%CAP]`. A full queue has $n = 4$; the back index is $(j + 3) \bmod 4 = 2$ exactly when $j = 3$: so $(j, n) = (3, 4)$. (Tracing 5.)

13. `push_back` improves from $\Theta(n)$ (walk to the end) to $\Theta(1)$; `push_front` and `pop_front` are $\Theta(1)$ in both. `pop_back` stays $\Theta(n)$ even with a tail pointer.

14. After removing the tail, the *new* tail is the node before it. A node in an SLL has no `prev` link, so that node can only be found by walking from `head` until `cur->next==tail`, $n - 2$ steps. The tail pointer helps only to reach the old tail, not its predecessor.

15. `c->prev=a; c->next=b; a->next=c; b->prev=c;` and `size++`. If `a` is `NULL` (insertion at the front) then `head=c`; if `b` is `NULL` (at the end) then `tail=c`.

16. (1) Only node: `head = tail = NULL`. (2) Head, with successor: `head = cur->next; head->prev = NULL`. (3) Tail, with predecessor: `tail = cur->prev; tail->next = NULL`. (4) Middle: `cur->prev->next = cur->next; cur->next->prev = cur->prev`. In all cases then `free(cur); size-`. Cases 1–4 collapse into two `if/else` statements: *predecessor exists? set its* `next`*, else set* `head`*; successor exists? set its* `prev`*, else set* `tail`.

17. A *sentinel* (dummy) node holds no item and is linked into a circular list so that no pointer is ever `NULL`. It removes (i) the special case of an empty list and (ii) the separate updates of `head` and `tail`: insertion before a node and removal of a node are the same few assignments everywhere. Costs: one extra node, and traversals must stop at the sentinel.

18. Two pointers start at the head; per iteration `slow` moves one node, `fast` two. When `fast` cannot make another full step (`fast==NULL` or `fast->next==NULL`) `slow` has covered half the distance, so it is at the middle. With the loop condition `fast!=NULL`

Juevesano D. 2026 All rights reserved.

`&& fast->next!=NULL`: odd length gives the exact middle, even length gives the *second* of the two middle nodes (for $1, 2, 3, 4$ it stops on 3).

19. At the top of each iteration the nodes before `cur` form a list in reversed order whose head is `prev`, and the nodes from `cur` on are still in the original order. Time $\Theta(n)$, extra space $O(1)$.

20. A *shallow copy* copies only the references (the struct with `head`, `tail`, `size`); both variables share the same nodes. A *deep copy* allocates new nodes with copies of the items. After `l = m` and the destruction of `m`, the nodes seen by `l` have been freed: `l.head` and `l.tail` are dangling, any use of `l` is undefined behaviour, and destroying `l` again is a double free. (Tracing 24.)

21. *Splicing* moves a chain of nodes from one list into another by changing pointers; nothing is allocated or copied. To place all of `m` after node `p` of `l`: let $q = p.next$; set $p.next = m.head$ and $m.tail.next = q$; if q was `NULL` then `l.tail` $= m.tail$; add the sizes and empty `m`. The cost is $\Theta(1)$ *only if* `p` is already known (or the end is) and `m` has a tail pointer; locating the i-th node costs $\Theta(i)$.

22. Invariant: `mid` points to the node at index $\lfloor(\texttt{size} - 1)/2\rfloor$. In an SLL: `middle`, `pushBack` and `popFront` are always $O(1)$; `pushFront` is $O(1)$ for even size but needs a backward step (a walk from the head, $\Theta(n)$) for odd size; `popBack` is $\Theta(n)$. Giving each node a `prev` pointer (a doubly linked list) makes every backward step $O(1)$ and hence all four end operations $O(1)$.

23. A *stack* is a sequence accessed at one end only: `push`, `pop`, `top`, `empty`, `size`; last in, first out. Applications: bracket matching, infix-to-postfix conversion and postfix evaluation, undo/backtracking, the call stack (recursion), depth-first search. In a singly linked list the *head* should be the top, since `push_front` and `pop_front` are $\Theta(1)$ there, whereas `pop_back` is $\Theta(n)$.

24. Every item is moved from `in` to `out` at most once, because after the transfer it leaves from `out`. Charge 3 units to an enqueue (push onto `in`, pop from `in`, push onto `out`); the later transfer and the pop from `out` are then paid in advance. m operations therefore cost at most $3m$ stack operations, so $\Theta(1)$ amortised, although one dequeue that triggers a transfer costs $\Theta(n)$. (Tracing 12.)

25. In `([)]` each kind of bracket occurs equally often as opener and closer, yet the nesting is wrong (the `[` is closed while the `(` is still open). A *stack* is the right tool: the closer must match the *most recently opened* bracket.

26. Postfix: `ab+c*de/-`. Prefix: `-*+abc/de`.

27. Tokens `5 1 2 + 4 * + 3 -`: push 5 $\to [5]$; push 1 $\to [5, 1]$; push 2 $\to [5, 1, 2]$; `+`: $1 + 2 = 3 \to [5, 3]$; push 4 $\to [5, 3, 4]$; `*`: $3 \cdot 4 = 12 \to [5, 12]$; `+`: $5 + 12 = 17 \to [17]$; push 3 $\to [17, 3]$; `-`: $17 - 3 = 14 \to [14]$. The value is 14.

28. Height of the tree: 3. Node 6: depth 2 (two edges from the root), height 1. Leaves: $1, 4, 7, 13$. Ancestors of 4: $6, 3, 8$ (and 4 itself under the definition that includes the node). Level 2: $1, 6, 14$.

 Juevesano D. 2026 All rights reserved.

29. At most $2^{h+1} - 1$ nodes (a perfect tree). Conversely $n \leq 2^{h+1} - 1$ gives the minimum height $h_{\min} = \lceil \lg(n+1) \rceil - 1$.

30. Let n_0, n_1, n_2 be the numbers of nodes with $0, 1, 2$ children, so $n = n_0 + n_1 + n_2$. Every node except the root has one parent edge, and it is counted at its parent: $n - 1 = n_1 + 2n_2$. Subtracting, $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2$, so $n_0 = n_2 + 1$.

31. Root A (first in preorder). In the inorder sequence `D B E A F C`, the items left of A (`D B E`) form the left subtree and the items right of it (`F C`) the right subtree. Next preorder item B is the root of the left subtree; `D` is left of it and `E` right. C is the root of the right subtree, with `F` as its left child (`F` precedes `C` in inorder). Tree: $A(B(D, E), C(F, \varnothing))$. Postorder: `D E B F C A`.

32. The two preorder/postorder sequences do not tell whether a lone child is a left or a right child. The tree with root r and left child c and the tree with root r and right child c both have preorder $r\,c$ and postorder $c\,r$.

33. For every node v: every key in the left subtree is $<$ the key of v, and every key in the right subtree is $\geq$ the key of v (equal keys go right in the course convention). The given tree is **not** a BST: 12 lies in the left subtree of 10 but is greater than 10, although it is correctly greater than its parent 5. The correct test passes an interval down: `isBST(r,lo,hi)` requires $lo \leq$ key $< hi$, recursing left with $hi =$ key and right with $lo =$ key; or: an inorder traversal must be non-decreasing.

34. Insertion gives the tree $50(30(20,\ 40(35, \varnothing)),\ 70(60, 80))$: height 3 (path $50, 30, 40, 35$), inorder `20 30 35 40 50 60 70 80`. Removing 30 (two children): its inorder successor is the minimum of the right subtree, 35. Copy 35 into the node and delete the leaf 35. Result $50(35(20, 40),\ 70(60, 80))$ with preorder `50 35 20 40 70 60 80`.

35. Worst case $\Theta(n)$ (= the height of the tree). It occurs, for example, when the keys are inserted in sorted or reverse-sorted order, which yields a path with $h = n - 1$. The best possible height is $\lceil \lg(n+1) \rceil - 1 = \lfloor \lg n \rfloor$ (a complete tree).

36. *Level order* uses a **queue**: nodes must be processed in the order in which they were discovered (first in, first out), so that all of a level is processed before any of the next. The depth-first orders (pre-, in-, postorder) use recursion, which is an implicit stack, or an explicit stack. All run in $\Theta(n)$ time. Extra space: $\Theta(h)$ for the depth-first orders, $\Theta(w)$ (the width, up to about $n/2$) for level order.

37. After inserting 10^6 keys in increasing order the tree is a path of height $10^6 - 1$. A recursive `height` uses one stack frame per level: 10^6 frames exceed the typical stack size (a few megabytes) and the program overflows the call stack. Alternatives: a level-order traversal with a queue that counts levels (heap memory, $O(n)$ time, space $O(w) = O(1)$ for a path), or an iterative postorder with an explicit heap-allocated stack. (Building such a tree is itself $\Theta(n^2)$ by successive insertions.)

Juevesano D. 2026 All rights reserved.

Chapter 11

Solutions to Section B

The output shown is the output of the actual program (compiled and run); the explanation shows how to reach it by hand. The answer lines of Chapter 8 should be compared with the listing.

Solution to Tracing 1.

```
3
11
0
23 16 4 8 15
16 4 8
1 0
8
16 4 99 5
5
```

The first line is 3: three successful appends, $1+1+1$. `insertFront(16)` returns 1, then $r = 1 \cdot 10 + \text{insertFront}(23) = 11$; the list is now $23, 16, 4, 8, 15$ and full (capacity $= 5$), so `append(42)` returns 0. `removeLast` makes `size`$= 4$; `removeFirst` shifts the items left and gives $16, 4, 8$ with `size`$= 3$; the raw array is $[16, 4, 8, 8, 15]$ (the cells beyond `size` are stale). `setItem(99,3)` is legal ($3 \leq$ *size*) and writes `items[2]`: result 1; `setItem(77,4)` is illegal ($4 >$ *size*): 0. **The twist:** `itemAt(l,4)` performs no bounds test and returns the stale cell `items[3]`$= 8$. After `append(5)` that cell is overwritten, so the same call now returns 5.

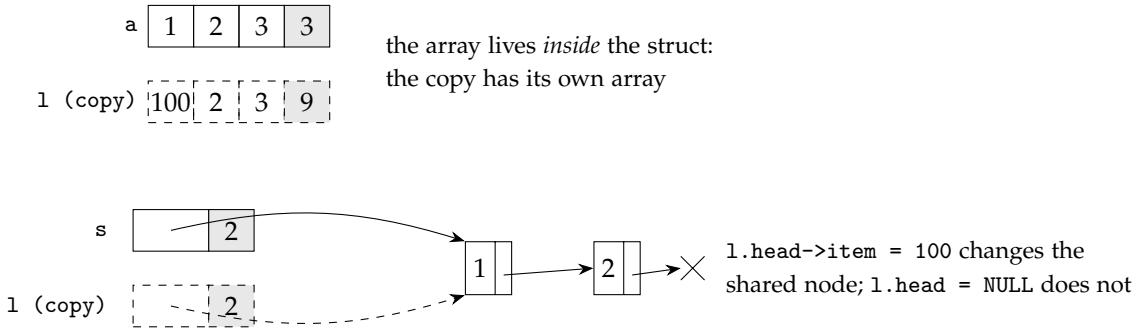
Final raw array: [16, 4, 99, 5, 15] with `size`$= 4$.

Solution to Tracing 2.

```
A: 1 3
B: 100 9
C: 100 3
D: 100
E: 0 2 100
```

A: `arrByValue` receives a copy of the struct, *including the array inside it*; nothing changes: `1 3`. **B:** by pointer, both changes are visible: `100 9`. After the reset, **C:** passing `a.items` gives the callee the address of the first element (an array parameter is a pointer), so `items[0]` changes but `size` does not: `100 3`. **D:** the same for a raw array: `100`. **E:** the struct `slist` is copied, but its pointer `head` still points to the real first node: the item change through `l.head->item` is visible (`100`); assigning `l.head=NULL` and `l.size=0` touches only the copy, so the caller still has a non-`NULL` head (0 printed) and `size` 2.

 Juevesano D. 2026 All rights reserved.



Why `arrByValue` *changes nothing but* `llByValue` *changes a node: copying a struct copies an inner* array *but only the* address *held in a pointer member.*

Solution to Tracing 3.

```
256 136 64 5 40 15
```

Counter	Value	Reasoning and class
`c1`	256	$n \cdot n$; $\Theta(n^2)$
`c2`	136	$1+2+\cdots+16 = n(n+1)/2$; $\Theta(n^2)$
`c3`	64	$i = 1, 2, 4, 8$ gives 4 outer iterations of 16; $\Theta(n \log n)$
`c4`	5	$i = 16, 8, 4, 2, 1$, i.e. $\lfloor \lg n \rfloor + 1$; $\Theta(\log n)$
`c5`	40	for each i the inner loop runs $\lceil (16-i)/4 \rceil$ times: $4 \cdot (4+3+2+1)$; $\Theta(n^2)$
`c6`	15	$1+2+4+8$, a geometric sum; $\Theta(n)$

The traps are `c4` (halving gives a logarithm) and `c6` (a doubling outer loop with an inner loop bounded by the outer variable is *linear*, not $n \log n$).

Solution to Tracing 4.

```
size=6  [10 20 30 40 50 60 0 0]
size=5  [10 30 40 50 60 60 0 0]
size=6  [10 30 40 99 50 60 0 0]
1
size=5  [10 30 40 99 50 60 0 0]
size=7  [8 7 10 30 40 99 50 0]
0 0
size=7  [8 7 10 30 40 99 50 0]
```

Six `push_back` calls fill the first six cells. `erase(1)` shifts four items left and leaves a stale copy of 60 in cell 5; `insert(3,99)` shifts items 3..4 right and overwrites that stale cell. The expression `erase(5)+erase(5)`: the first call is legal (position $5 < size = 6$) and returns 1; the second finds $5 \geq size = 5$ and returns 0; so $r = 1$ and the raw array is unchanged (the item 60 is no longer part of the list but remains in memory). The two insertions at the front shift the items right (the stale 60 is overwritten by 50) and give `size=7`. **Twist:** `insert(8,1)` fails although the capacity is 8: the legal positions are $0..size = 7$ (and the list is not full). Both calls print 0, and the array is unchanged.

Juevesano D. 2026 All rights reserved.

Solution to Tracing 5.

```
j=0 n=3 [1 2 3 0]
removed 1
j=1 n=2 [1 2 3 0]
j=1 n=4 [5 2 3 4]
0
j=3 n=2 [5 2 3 4]
j=3 n=4 [5 7 8 4]
4 5 7 8
j=3 n=0 [5 7 8 4]
```

Item i of the queue lives at `a[(j+i)%4]`. After three `add`s the array is $[1,2,3,0]$ and $j=0$. `removeFront` returns 1 and advances j to 1 (the cell is not erased). `add(4)` writes cell $(1+2)\%4=3$; `add(5)` writes cell $(1+3)\%4=0$ — the *wrap-around* — and the queue is full ($n=4$), so `add(6)` prints 0. Two removals give $j=3$, $n=2$. `add(7)`, `add(8)` write cells $(3+2)\%4=1$ and $(3+3)\%4=2$, giving the raw array $[5,7,8,4]$. Draining the queue visits cells $3,0,1,2$: `4 5 7 8`, and $j=(3+4)\%4=3$ with $n=0$. Note the trailing space before the line break.

Solution to Tracing 6.

```
pushBack 5 size=1 head=5 tail=5  [5]
pushFront 3 size=2 head=3 tail=5  [3 5]
pushBack 8 size=3 head=3 tail=8  [3 5 8]
popBack    size=2 head=3 tail=5  [3 5]
popBack    size=1 head=3 tail=3  [3]
popFront   size=0 head=NULL tail=NULL  []
0 3
pushFront 1 size=1 head=1 tail=1  [1]
pushBack 2 size=2 head=1 tail=2  [1 2]
```

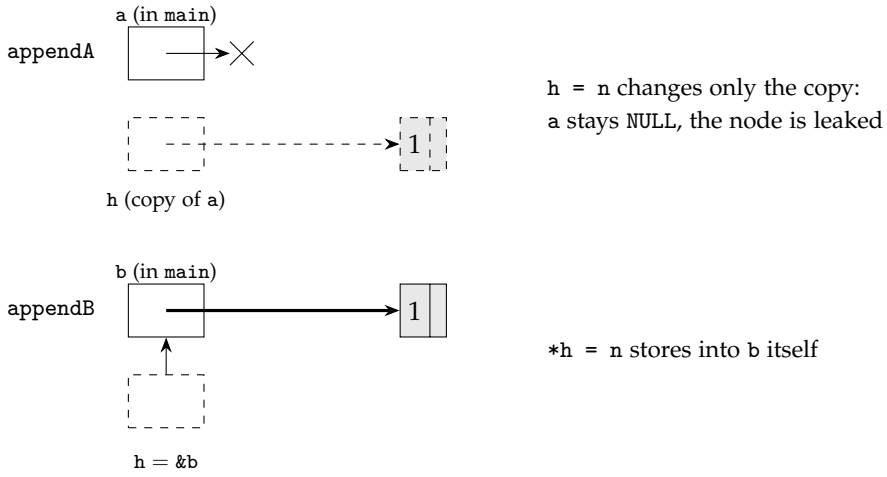
`pushBack(5)` on an empty list sets both `head` and `tail`. `pushFront(3)` changes only `head`; `pushBack(8)` only `tail`. The first `popBack` must walk to the node before the tail: new tail 5; the second `popBack` removes the node whose predecessor is the head, so `head` and `tail` coincide (3). `popFront` of the last node empties the list and must set `tail` to `NULL` as well (otherwise `tail` dangles). The final `popFront` on the empty list fails and returns 0, and leaves `x` untouched, so it still holds the 3 of the previous pop: `0 3`. The last two lines show that after the list became empty `pushFront` and `pushBack` work as for a fresh list.

Solution to Tracing 7.

```
0 3
4 4
```

`appendA` receives a *copy* of the head pointer. With `a==NULL` the statement `h=n` changes only the local copy, so every call leaks a node and `a` stays `NULL`: `length(a)=0`. `appendB` receives the address of the head and stores into the real pointer: 3. For the second line, `c` and `d` start with one node (7). Now `c` is not empty, so `appendA(c,i)` walks the real chain through its copy and appends correctly: 4 nodes. Both give 4. Leaked nodes: three for `a`, none for `b`, `c`, `d`. The single-pointer version works only when the list is non-empty — the typical hidden test is the empty list.

 Juevesano D. 2026 All rights reserved.



Single versus double pointer in `appendA` and `appendB`: only the address of the pointer lets the callee overwrite the caller's `head`.

Solution to Tracing 8.

```
5 7
size=2 head=5 tail=7
7 5
size=2
size=1 head=5 tail=5 prev=1
size=0 headIsNULL=1 tailIsNULL=1
```

$[2, 2, 5, 2, 7, 2, 2]$ with $x = 2$ leaves $[5, 7]$: the head is removed twice, the middle 2 once, the tail twice. The forward and backward displays agree because the `prev` links were repaired: `7 5`. `eraseItems(&l,9)` changes nothing (size 2). Removing 7 removes the tail: `tail` moves to 5, and since the list has a single node left, `tail->prev==NULL` (1). Removing 5, now the only node, sets both `head` and `tail` to `NULL` and `size` to 0.

Solution to Tracing 9.

```
1 2 3 4 # 4 3 2 1
4 13
3 3
```

`mystery(a)` prints each item on the way down, then `#` at the end of the list, then each item again on the way back: `1 2 3 4 # 4 3 2 1`. `g`: odd items give `g(rest)-1`, even items `item+g(rest)`. For a: $g(4) = 4$, $g(3) = 3$, $g(2) = 5$, $g(1) = 4$. For $b = 3, 5, 2, 8, 6, 9$ working from the end: $g(9) = -1$, $g(6) = 5$, $g(8) = 13$, $g(2) = 15$, $g(5) = 14$, $g(3) = 13$. `h` counts the positions where the next item is larger (ascents): a has $1 < 2$, $2 < 3$, $3 < 4$; b has $3 < 5$, $2 < 8$, $6 < 9$: three each.

Solution to Tracing 10.

```
3 10
3 1
2 3
4 5
```

Juevesano D. 2026 All rights reserved.

Stack $[1, 2, 3]$ and queue $[10, 20]$ (front first). `stPop` gives 3, `qDeq` gives 10: `3 10`. The queue becomes $[20, 3]$ and the stack $[1, 2, 10]$. Four pops: 10, 2, 1 succeed and the fourth fails and leaves `x`$= 1$ unchanged, so $r = 3$ and `x`$= 1$: `3 1`. Four dequeues: 20, 3 succeed, the others fail and leave `y`$= 3$: `2 3`. Finally the stack $[4, 5]$ is emptied into the queue (reversing: $5, 4$), the queue is emptied into the stack (order preserved: the top is now 4), and popping prints `4 5`. Emptying the stack into the queue and the queue back onto a stack reverses the stack once, which is why 4 is printed before 5.

Solution to Tracing 11.

```
abcd-*e/+ 4
```

Stack contents after each character of `a+b*(c-d)/e` (bottom to top), and the output:

Char	Action	Stack	Output
`a`	output		`a`
`+`	push	`+`	`a`
`b`	output	`+`	`ab`
`*`	`+` binds weaker: push	`+*`	`ab`
`(`	push	`+*(`	`ab`
`c`	output	`+*(`	`abc`
`-`	top is `(`: push	`+*(-`	`abc`
`d`	output	`+*(-`	`abcd`
`)`	pop to `(`	`+*`	`abcd-`
`/`	`*` binds equally: pop it, then push	`+/`	`abcd-*`
`e`	output	`+/`	`abcd-*e`
end	pop all		`abcd-*e/+`

The largest stack size observed is 4 (after the `-`). The twist is the `/`: because `*` and `/` have equal precedence and are left-associative, the `*` is popped first.

Solution to Tracing 12.

```
1 2 3 4 5 6 -1
in=0 out=0
```

The seven calls of `deq` return the items in the order of arrival, $1 \ldots 6$, and then -1 for the empty queue: `1 2 3 4 5 6 -1`. The first `deq` finds `out` empty, pops $3, 2, 1$ from `in` and pushes them onto `out` (reversed, so 1 is on top), then pops 1: $3 + 3 + 1 = 7$ **successful stack operations**. (The failing fourth `stPop` on the empty `in` is not counted.) Later calls cost one pop each, except the one that triggers the second transfer of $4, 5, 6$. Both stacks are empty at the end. This is the amortised argument: the expensive call is paid for by the cheap ones.

Solution to Tracing 13.

```
1 2 3 4 5 6 7 8
40 10 60 5 20 50 70 15
40 10 5 15 20 60 50 70
```

 Juevesano D. 2026 All rights reserved.

```
15 5 10 20 40 50 60 70
15 5 20 10 50 70 60 40
1101
```

Level-order insertion fills the first vacancy, so the tree is the complete tree with level order $40, 10, 60, 5, 20, 50, 70, 15$: node 40 has children $10, 60$; 10 has $5, 20$; 60 has $50, 70$; 5 has the left child 15. The counts print 1..8. Preorder `40 10 5 15 20 60 50 70`, inorder `15 5 10 20 40 50 60 70`, postorder `15 5 20 10 50 70 60 40`. The inorder is *not* sorted, because the tree is not a BST. The searches use the BST rule: 20: < 40 left, > 10 right, found (1); 60: right, found (1); 15: < 40, > 10 goes right to 20, < 20 goes left: `NULL`, **not found (0) although** 15 **is in the tree**; 50: right, < 60 left, found (1): `1101`.

Solution to Tracing 14.

```
50 30 30 30 70 60 50 80
30 30 30 50 50 60 70 80
30 30 30 50 60 80 70 50
50 30 70 30 60 80 30 50
3 8
5 8 3
```

Duplicates go to the right, so the tree is $50\,(30\,(\cdot, 30\,(\cdot, 30)),\ 70\,(60\,(50, \cdot), 80))$: the first 30 has the second 30 as its right child, which has the third as its right child; the second 50 is the left child of 60. Hence preorder `50 30 30 30 70 60 50 80`, inorder `30 30 30 50 50 60 70 80` (sorted, with duplicates), postorder `30 30 30 50 60 80 70 50`, level order `50 30 70 30 60 80 30 50`; height 3 (the path of three 30's below the root, or $50, 70, 60, 50$), count 8. `find(60)`: root (2 comparisons, right), 70 (2, left), 60 (1, match) $= 5$. `find(35)`: 50, 30, 30, 30 at two comparisons each and then `NULL`: 8. `find(30)`: 50 (2), first 30 (1) $= 3$: the search stops at the *first* occurrence.

Solution to Tracing 15.

```
50 25 10 36 29 80 60 55 76 68 78 91
50 25 10 36 29 80 60 55 76 68 78
50 25 10 29 80 60 55 76 68 78
50 29 10 80 60 55 76 68 78
55 29 10 80 60 76 68 78
0 55 1
55 29 80 10 60 76 68 78
```

Initial tree: $50\,(25\,(10,\ 36\,(29, \cdot)),\ 80\,(60\,(55,\ 76\,(68, 78)),\ 91))$, preorder as printed. Remove 91: a leaf. Remove 36: one child, which (29) takes its place. Remove 25: two children; the inorder successor is the minimum of the right subtree, 29; 29 is copied into the node and the old leaf 29 deleted. Remove 50 (the root): the successor is the leftmost node of the right subtree, 55; 55 replaces the key of the root and the leaf 55 is deleted. `removeKey(99)` returns 0; the root has key 55 and its `parent` is `NULL` (1). Level order of the final tree: `55 29 80 10 60 76 68 78`. The root *node* is the same; only its key changed.

Solution to Tracing 16.

Juevesano D. 2026 All rights reserved.

```
1 4 7 6 3 14 10 8
32 5 43
```

`f` is a postorder traversal that prints the nodes after both children: `1 4 7 6 3 14 10 8`. It returns the largest sum on a root-to-leaf path: $f(6) = 13$, $f(3) = 16$, $f(10) = 24$, $f(8) = 32$ (path $8, 10, 14$). `g` adds the keys at even depths and subtracts those at odd depths: $8 - (3 + 10) + (1 + 6 + 14) - (4 + 7) = 5$. `k` returns 1 at leaves and $2k(\ell) + 3k(r)$ elsewhere: $k(6) = 5$, $k(3) = 17$, $k(10) = 3$ (a missing left child contributes 0), $k(8) = 2 \cdot 17 + 3 \cdot 3 = 43$.

Solution to Tracing 17.

```
6 6
1 6
2 0
```

Both trees contain the same six keys, so the counts are 6 and 6. The iterative `insert` sets `parent`; the recursive `rinsert` never does. Therefore the tree `a` has exactly one node without a parent (its root: 1), but in `b` every node has `parent==NULL`: 6. Node 30 is at depth 2 in `a`; walking up through the parent links in `b` stops immediately, giving 0. The two trees look identical when printed, so the bug is invisible until a parent pointer is used.

Solution to Tracing 18.

```
10(2) 20(1) 30(1) 40(0) 50(1) 60(0) 70(0)
3
```

The tree is the perfect tree $40\,(20\,(10, 30),\ 60\,(50, 70))$. The inner loop pushes the left spine: $40, 20, 10$ (`depth` 3, the maximum). Popping 10 leaves 2 items on the stack, so it prints `10(2)`; then its right child is empty, 20 is popped (`20(1)`), its right child 30 is pushed and immediately popped (`30(1)`), then `40(0)`; the right subtree is handled in the same way: 50 is pushed on top of 60 (`50(1)`), then `60(0)`, `70(0)`. The printed number is the number of ancestors still waiting on the stack. Maximum stack size: 3, the height $+1$ (extra space $O(h)$).

Solution to Tracing 19.

```
20>30 30>35 35>40 40>45 45>50 50>60 60>70 70>80 80>-1
50 50 1 1
```

The tree is $50\,(30\,(20,\ 40\,(35, 45)),\ 70\,(60, 80))$. Starting at the minimum 20 and following `successor` visits the keys in sorted order, and the last key 80 has no successor (-1). `predecessor(60)`: 60 has no left subtree, so climb: 60 is a left child of 70 (keep climbing), 70 is a right child of 50 — so 50. The successor of 45: 45 has no right subtree, so climb: 45 is a right child of 40, and 40 is a right child of 30 (keep climbing), and 30 is a left child of 50 — so 50. The node 80 is the maximum, so it has no successor, and 20 is the minimum, so it has no predecessor (both print 1, meaning `NULL`).

Solution to Tracing 20.

 Juevesano D. 2026 All rights reserved.

```
mPushBack 10 n=1 mid=10  [10]
mPushBack 20 n=2 mid=10  [10 20]
mPushBack 30 n=3 mid=20  [10 20 30]
mPushBack 40 n=4 mid=20  [10 20 30 40]
mPushFront 5 n=5 mid=20  [5 10 20 30 40]
mPushFront 1 n=6 mid=10  [1 5 10 20 30 40]
mPopFront    n=5 mid=20  [5 10 20 30 40]
mPopBack     n=4 mid=10  [5 10 20 30]
mPushBack 50 n=5 mid=20  [5 10 20 30 50]
mPopFront    n=4 mid=20  [10 20 30 50]
```

The invariant is that `mid` is at index $\lfloor (n-1)/2 \rfloor$. `mPushBack 10`: the first node is the middle. `mPushBack 20` ($n = 1$, odd): `mid` stays. `mPushBack 30` ($n = 2$, even): `mid` moves *forward* to 20. `mPushBack 40` ($n = 3$): stays. `mPushFront 5` ($n = 4$, even): stays (all indices rose by one but the target index rose by one too). `mPushFront 1` ($n = 5$, odd): the target index does not rise, so `mid` moves *backward* to 10 (an $O(n)$ step in an SLL). `mPopFront` ($n = 6$, even): forward to 20. `mPopBack` ($n = 5$, odd): backward to 10. `mPushBack 50` ($n = 4$, even): `mid` moves forward to 20. `mPopFront` ($n = 5$, odd): `mid` stays at 20, which is now at index 1 of 4.

Solution to Tracing 21.

```
step 1: lo=0 hi=9 mid=4 a[mid]=7
step 2: lo=0 hi=4 mid=2 a[mid]=4
step 3: lo=0 hi=2 mid=1 a[mid]=4
step 4: lo=0 hi=1 mid=0 a[mid]=2
answer 1
step 1: lo=0 hi=9 mid=4 a[mid]=7
step 2: lo=5 hi=9 mid=7 a[mid]=15
step 3: lo=5 hi=7 mid=6 a[mid]=12
step 4: lo=5 hi=6 mid=5 a[mid]=9
answer 6
step 1: lo=0 hi=9 mid=4 a[mid]=7
step 2: lo=5 hi=9 mid=7 a[mid]=15
step 3: lo=8 hi=9 mid=8 a[mid]=15
answer 9
```

`lowerBound` keeps the candidate: if `a[mid]<x` the answer is to the right of `mid` (`lo=mid+1`), otherwise `mid` itself may be the answer (`hi=mid`). For $x = 4$ this takes 4 iterations and ends with $lo = hi = 1$: the *first* index with $a[i] \geq 4$, even though $a[2]$ and $a[3]$ are also 4 — the search keeps moving left on equality. For $x = 10$ the answer is 6 (the first item ≥ 10 is 12). For $x = 20$ every probe goes right and the answer is $9 = n$, i.e. "no such item". Each search takes at most $\lceil \lg(n+1) \rceil = 4$ iterations.

Solution to Tracing 22.

```
out:3 out:6 out:4 out:2 out:5
winner:1
```

With $k = 3$ the loop rotates two people to the back and eliminates the third: $[1..6] \to$ out 3, $[4,5,6,1,2] \to$ out 6, $[1,2,4,5] \to$ out 4, $[5,1,2] \to$ out 2, $[5,1] \to$ out 5, leaving 1. For $k = 2$ the eliminations are $2,4,6,3,1$ and the winner is 5 (check with $J(1) = 0$, $J(m) = (J(m-1)+2) \bmod m$: $0,0,2,0,2,4$, i.e. person $J(6)+1 = 5$).

Juevesano D. 2026 All rights reserved.

Solution to Tracing 23.

```
[1 2 10 20 30 3 4 5]
[]
8 0
[1 2 10 7 8 9 20 30 3 4 5]
[]
11 0 tail=5
0 1
[1 2 10 7 8 9 20 30 3 4 5 100]
1 12 tail=100
```

`splice(&a,2,&b)` keeps the first two nodes of `a`, inserts all of `b`, and appends the rest: `[1 2 10 20 30 3 4 5]`; `b` is empty and the sizes are 8 and 0. `spliceAfter(&a,x,&c)`, with `x` the node 10 (the third node), inserts `c`$=[7\,8\,9]$ right after it: `[1 2 10 7 8 9 20 30 3 4 5]`; `c` is empty, the sizes are 11 and 0, and the tail is still 5 because the insertion was not at the end. `splice(&a,99,&d)`: 99 is larger than the size, so nothing happens and the call returns 0 (`d` still has 1 node). `splice(&a,a.size,&d)` appends `d`$=[100]$ at the very end: the call returns 1, the size is 12 and the tail becomes 100 (the function must update `tail` in this case).

Solution to Tracing 24.

```
99 2 3 | 3
0 99 2 3 | 4
1 77 3 | 3
```

`shallowCopy` copies the struct, so `sh` shares all nodes with `orig`; `deepCopy` allocates new nodes. `sh.head->item=99` therefore also changes `orig` (first line starts with 99). `dp.head->next->item=77` changes only the deep copy. `pushFront(&sh,0)` adds a node to `sh` and changes `sh.head` and `sh.size` only: `orig` still starts at 99 and still reports size 3, so the sharing is now invisible but dangerous. The lists `orig` and `sh` share three nodes; `dp` shares none.

Solution to Tracing 25.

```
-*+623/94
6+2*3-9/4
62+3*94/-
22
```

The tree is $-(*(+(6,2),3),\ /(9,4))$. Preorder (prefix): `-*+623/94`. Inorder, *printed without parentheses*: `6+2*3-9/4` — which, read by the usual precedence rules, would mean $6+6-2=10$, not the tree's value; an inorder print needs parentheses to be unambiguous. Postorder (postfix): `62+3*94/-`. Evaluation: $(6+2)\cdot 3=24$ and $9/4=2$ with integer division, so $24-2=22$. With real division the value would be $24-2.25=21.75$.

 Juevesano D. 2026 All rights reserved.

Chapter 12

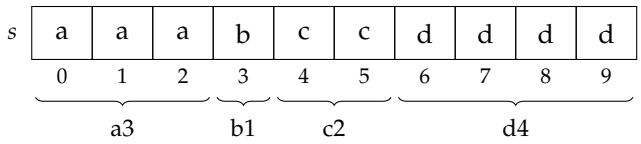
Solutions to Section C

Each solution gives the idea, a complete program with comments, the complexity, and the typical mistakes. All listings were compiled and run against the sample tests of Chapter 9, against brute-force models on random inputs, and (for the pointer-heavy ones) under the address and undefined-behaviour sanitizers. A listing contains only the relevant functions; the types it uses are those of the cited template listings.

12.1 Arrays and Array Lists

Solution to Problem 1.

Idea. Scan with a read index `i`. For each run, count its length by advancing `i` while `s[i]` equals the run's character; write the character, then the count in decimal. To write a count with several digits, first find how many digits it has, then fill them from the right. The in-place version is the same loop, except a run of length 1 writes no digit. It is safe because the encoding of a run never takes more cells than the run itself, so the write index never passes the read index.



Each run is one letter and one count. The encoding of a run is never longer than the run itself, so `compress` may overwrite its own input.

```
// Writes the encoding of s into out (a letter followed by its run length).
// Returns the length of the encoding.
int rleEncode(char *s, char *out) {
    int i = 0;                      // read position in s
    int w = 0;                      // write position in out

    while (s[i] != '\0') {
        char c = s[i];
        int run = 0;
        while (s[i] == c) {         // count the run of c
            run++;
            i++;
        }
        out[w] = c;
        w++;

```

```
        int digits = 0;               // how many digits does run have?
        int t = run;
        while (t > 0) {
            digits++;
            t = t / 10;
        }
        for (int d = digits - 1; d >= 0; d--) {
            out[w + d] = '0' + run % 10;   // fill the digits from the right
            run = run / 10;
        }
        w = w + digits;
    }
    out[w] = '\0';
    return w;
}

// Compresses a[0..n-1] in place; a run of length 1 keeps no number.
// Returns the new length.
int compress(char *a, int n) {
    int r = 0;                        // read position
    int w = 0;                        // write position (never passes r)

    while (r < n) {
        char c = a[r];
        int run = 0;
        while (r < n && a[r] == c) {
            run++;
            r++;
        }
        a[w] = c;
        w++;

        if (run > 1) {
            int digits = 0;
            int t = run;
            while (t > 0) {
                digits++;
                t = t / 10;
            }
            for (int d = digits - 1; d >= 0; d--) {
                a[w + d] = '0' + run % 10;
                run = run / 10;
            }
            w = w + digits;
        }
    }
    return w;
}

// Bonus: decodes "a3b1" into "aaab". Returns the decoded length.
int rleDecode(char *e, char *out) {
    int i = 0;
    int w = 0;
```

 Juevesano D. 2026 All rights reserved.

```
70
71      while (e[i] != '\0') {
72          char c = e[i];
73          i++;
74          int count = 0;
75          while (e[i] >= '0' && e[i] <= '9') {
76              count = count * 10 + (e[i] - '0');
77              i++;
78          }
79          for (int k = 0; k < count; k++) {
80              out[w] = c;
81              w++;
82          }
83      }
84      out[w] = '\0';
85      return w;
86  }
```

Listing 12.1: String compression (run-length encoding)

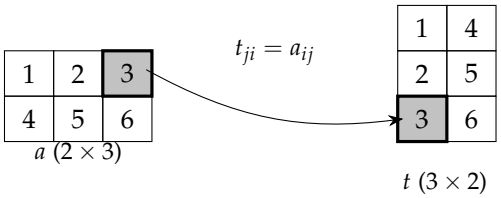
Complexity. $\Theta(n)$ time (each character is read once). Extra space $O(1)$ for `compress`.

Common mistakes.

- Forgetting the last run (a loop that writes a run only when it *sees a different character* never flushes the final one).
- Writing the digits of a multi-digit count in the wrong order, or writing `'0'+run` for a count of 10 or more.
- In `compress`: writing `1` for runs of length 1, or not terminating `out` with `'\0'`.

Solution to Problem 2.

Idea. The element in row i, column j of the input goes to row j, column i of the output: two nested loops suffice, and the output has m rows and n columns. For the in-place version swap only the pairs above the diagonal ($j > i$): the diagonal stays, and the pairs below are the same pairs seen from the other side.



Element (i, j) of the input goes to position (j, i) of the output, here $a_{0,2} = 3 \mapsto t_{2,0}$.

```
1   // t becomes the m x n transpose of the n x m matrix a.
2   void transpose(int a[][10], int n, int m, int t[][10]) {
3       for (int i = 0; i < n; i++) {
4           for (int j = 0; j < m; j++) {
5               t[j][i] = a[i][j];
```

```
        }
    }
}

// Bonus: transposes a square n x n matrix in place.
void transposeInPlace(int a[][10], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {      // only above the diagonal
            int tmp = a[i][j];
            a[i][j] = a[j][i];
            a[j][i] = tmp;
        }
    }
}
```

Listing 12.2: Matrix transpose

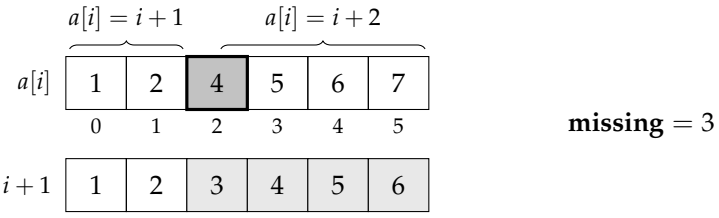
Complexity. $\Theta(nm)$ time. `transpose`: $\Theta(nm)$ extra space (the output); `transposeInPlace`: $O(1)$ extra space.

Common mistakes.

- Looping over the *whole* square matrix when swapping in place: every pair is swapped twice and nothing changes.
- Declaring the result $n \times m$ instead of $m \times n$ for a non-square input.

Solution to Problem 3.

Idea. *Sum.* With $n = m + 1$ the total $1 + \cdots + n = n(n+1)/2$ is known; subtract the sum of the array. *Binary search (bonus).* Before the gap $a[i] = i + 1$; from the gap on $a[i] = i + 2$. This is true on a prefix and false afterwards, so binary search for the first index with $a[i] \neq i + 1$; the missing number is that index plus 1 (or $m + 1$ if there is no such index).



Left of the gap $a[i] = i + 1$; from the gap on $a[i] = i + 2$. The first index where the two rows differ gives the missing number $i + 1$.

```
// O(n): the sum 1 + 2 + ... + n is known, so subtract the array sum.
int missingNumber(int a[], int m) {
    int n = m + 1;
    int expected = n * (n + 1) / 2;
    int actual = 0;
    for (int i = 0; i < m; i++) {
        actual = actual + a[i];
    }
```

 Juevesano D. 2026 All rights reserved.

```
    return expected - actual;
}

// Bonus, O(log n): before the gap a[i] == i + 1, after it a[i] == i + 2.
// Binary search for the first index where a[i] != i + 1.
int missingNumberFast(int a[], int m) {
    int lo = 0;
    int hi = m;                         // the answer index is in [0, m]
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == mid + 1) lo = mid + 1;    // gap is to the right
        else hi = mid;                          // gap is here or to the left
    }
    return lo + 1;
}
```

Listing 12.3: The missing number

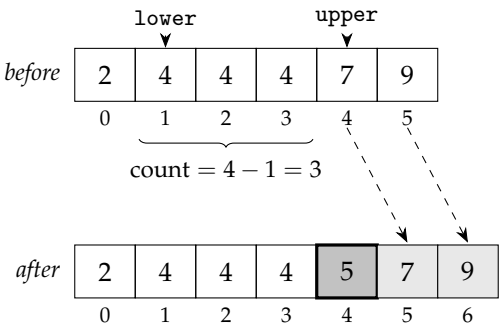
Complexity. Sum: $\Theta(m)$ time, $O(1)$ space. Binary search: $\Theta(\log m)$ time.

Common mistakes.

- Binary search with the wrong predicate, or an off-by-one when the last value ($m+1$) is the missing one.
- Forgetting that the gap may be at the very beginning ($a[0] = 2$).

Solution to Problem 4.

Idea. Both helpers keep the invariant "all items left of `lo` are on the wrong side, all items from `hi` on are on the right side", with `hi` starting at `size`. Then `search` is `lowerBound` plus an equality check, `count` is `upperBound`$-$`lowerBound`, `insert` places x at `upperBound` (after equals) after shifting the suffix right, and `erase` shifts the suffix left over the position found.



Top: `lowerBound(4)`= 1 and `upperBound(4)`= 4, so `count(4)`= 3. Bottom: `insert(5)` goes to `upperBound(5)`= 4; the suffix shifts one place right (from the end!).

```
// Index of the first item >= x (size if every item is smaller).
int lowerBound(slist *l, int x) {
    int lo = 0;
    int hi = l->size;
```

```
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (l->items[mid] < x) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

// Index of the first item > x (size if no item is larger).
int upperBound(slist *l, int x) {
    int lo = 0;
    int hi = l->size;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (l->items[mid] <= x) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

// Index of the first occurrence of x, or -1.
int search(slist *l, int x) {
    int p = lowerBound(l, x);
    if (p < l->size && l->items[p] == x) return p;
    return -1;
}

// Inserts x so the list stays sorted. Returns 0 if the list is full.
int insert(slist *l, int x) {
    if (l->size == 100) return 0;
    int p = upperBound(l, x);                 // slot after equal items
    for (int i = l->size; i > p; i--) {       // shift the tail right
        l->items[i] = l->items[i - 1];
    }
    l->items[p] = x;
    l->size++;
    return 1;
}

// Removes one occurrence of x. Returns 1 if x was found, else 0.
int erase(slist *l, int x) {
    int p = search(l, x);
    if (p == -1) return 0;
    for (int i = p; i < l->size - 1; i++) {   // shift the tail left
        l->items[i] = l->items[i + 1];
    }
    l->size--;
    return 1;
}

// Bonus: number of occurrences of x in O(log n).
int count(slist *l, int x) {
    return upperBound(l, x) - lowerBound(l, x);
```

 Juevesano D. 2026 All rights reserved.

```
58 }
```

Listing 12.4: A sorted array list

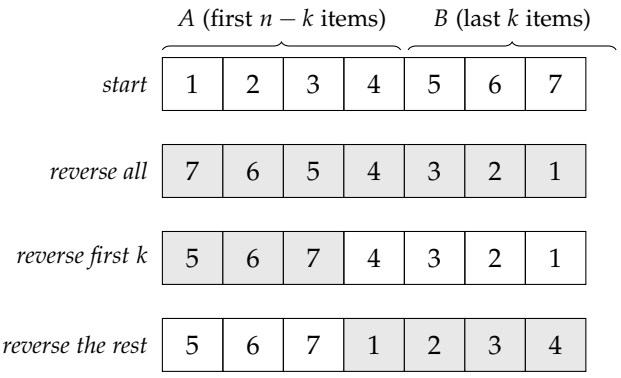
Complexity. Position queries $\Theta(\log n)$; `insert/erase` $\Theta(n)$ in the worst case because of the shifts. Binary search does *not* make insertion $O(\log n)$ overall.

Common mistakes.

- Binary search over `[0, size-1]` with `lo<hi`: the last element is never examined.
- Shifting in the wrong direction and overwriting items.
- Inserting before the equal items (still sorted, but no longer stable).

Solution to Problem 5.

Idea. First replace k by $k \bmod n$. Reversing the whole array turns $[A \mid B]$ (with $|B| = k$) into $[B^R \mid A^R]$. Reversing the first k items and then the remaining $n - k$ items gives $[B \mid A]$, which is the rotation.



Rotating right by $k = 3$: reverse everything, then reverse the first k items, then the remaining $n - k$ items.

```
1  void reverse(int a[], int lo, int hi) {
2      while (lo < hi) {
3          int tmp = a[lo];
4          a[lo] = a[hi];
5          a[hi] = tmp;
6          lo++;
7          hi--;
8      }
9  }
10
11 // Rotates a[0..n-1] to the right by k places.
12 void rotate(int a[], int n, int k) {
13     if (n == 0) return;
14     k = k % n;
15     reverse(a, 0, n - 1);        // [1 2 3 4 5 6 7] -> [7 6 5 4 3 2 1]
16     reverse(a, 0, k - 1);        // first k items    -> [5 6 7 4 3 2 1]
17     reverse(a, k, n - 1);        // remaining items  -> [5 6 7 1 2 3 4]
18 }
```

Listing 12.5: Rotate an array

Juevesano D. 2026 All rights reserved.

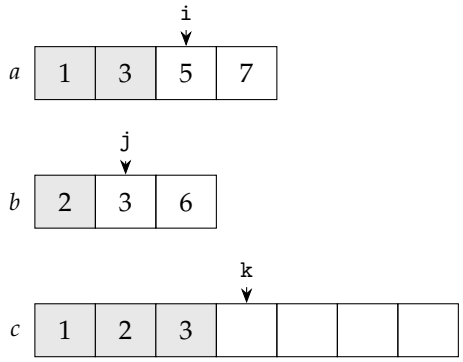
Complexity. $\Theta(n)$ time, $O(1)$ extra space.

Common mistakes.

- Forgetting k mod n, or dividing by zero when $n = 0$.
- Reversing the wrong block sizes: for a right rotation the *last* k elements form the second block.

Solution to Problem 6.

Idea. Keep one index into each input. Repeatedly copy the smaller front item (taking `a` on ties). When one input is exhausted, copy what is left of the other.



After three steps (1, 2, 3 *copied; on the tie* 3 *the item of* `a` *went first)* `i`, `j`, `k` *point to the next item to read from* `a`, *from* `b`, *and the next free cell of* `c`.

```
1  // Merges sorted a[0..n-1] and b[0..m-1] into c. Returns n + m.
2  int mergeArrays(int a[], int n, int b[], int m, int c[]) {
3      int i = 0;
4      int j = 0;
5      int k = 0;
6
7      while (i < n && j < m) {
8          if (a[i] <= b[j]) {          // <= keeps equal items of a first
9              c[k] = a[i];
10             i++;
11         } else {
12             c[k] = b[j];
13             j++;
14         }
15         k++;
16     }
17     while (i < n) {                  // one of these loops does nothing
18         c[k] = a[i];
19         i++;
20         k++;
21     }
22     while (j < m) {
23         c[k] = b[j];
24         j++;
```

 Juevesano D. 2026 All rights reserved.

```
25          k++;
26      }
27      return k;
28  }
```

Listing 12.6: Merge two sorted arrays

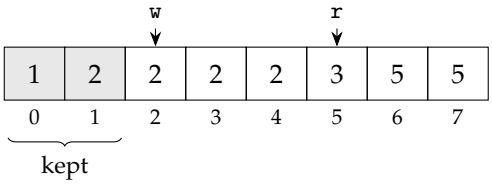
Complexity. $\Theta(n+m)$ time and $\Theta(n+m)$ output space.

Common mistakes.

- Using `<` instead of `<=` when comparing: on ties `b` goes first.
- Reading past the end of an exhausted array in the comparison.

Solution to Problem 7.

Idea. Use two indices: `r` reads every item; `w` is the length of the unique prefix. An item is new exactly when it differs from the last kept item `a[w-1]`. Copy new items forward.



The prefix `a[0..w-1]` holds the distinct values found so far. `a[r]=3` differs from `a[w-1]=2`, so it is copied to `a[w]` and `w` grows.

```
1   // Keeps one copy of each value of the sorted array. Returns the new length.
2   int dedupSorted(int a[], int n) {
3       if (n == 0) return 0;
4       int w = 1;                      // a[0..w-1] holds the unique items
5       for (int r = 1; r < n; r++) {
6           if (a[r] != a[w - 1]) {     // a new value
7               a[w] = a[r];
8               w++;
9           }
10      }
11      return w;
12  }
```

Listing 12.7: Remove duplicates from a sorted array

Complexity. $\Theta(n)$ time, $O(1)$ space.

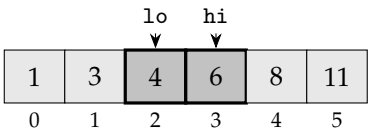
Common mistakes.

- Not treating $n = 0$ separately.
- Using the technique on an unsorted array.

Solution to Problem 8.

Juevesano D. 2026 All rights reserved.

IDEA. Put `lo` at the beginning and `hi` at the end. If the sum is too small, no pair involving `a[lo]` can work, so discard `lo`; if the sum is too large, discard `hi`. Each step discards one index, so at most $n - 1$ steps are performed.



lo	hi	sum	action
0	5	12	> 10: `hi--`
0	4	9	< 10: `lo++`
1	4	11	> 10: `hi--`
1	3	9	< 10: `lo++`
2	3	10	found

Each step discards one index: too big ⇒ discard `hi`; too small ⇒ discard `lo`. Discarded cells are shaded.

```
// Finds i < j with a[i] + a[j] == target in a sorted array.
// Returns 1 and sets *pi, *pj, or returns 0 if there is no such pair.
int twoSum(int a[], int n, int target, int *pi, int *pj) {
    int lo = 0;
    int hi = n - 1;
    while (lo < hi) {
        int sum = a[lo] + a[hi];
        if (sum == target) {
            *pi = lo;
            *pj = hi;
            return 1;
        }
        if (sum < target) lo++;      // need a bigger sum
        else hi--;                   // need a smaller sum
    }
    return 0;
}
```

Listing 12.8: Two-sum in a sorted array

COMPLEXITY. $\Theta(n)$ time, $O(1)$ space. The double loop is $\Theta(n^2)$.

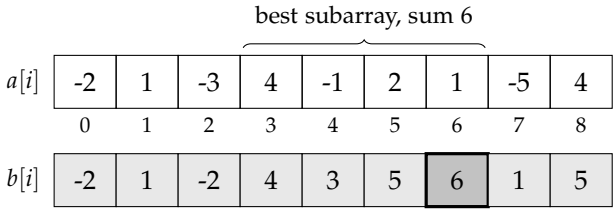
COMMON MISTAKES.

- Using $\leq$ instead of $<$ in the loop condition and pairing an element with itself.
- Moving both pointers after a miss.

Solution to Problem 9.

IDEA. Let b_i be the best sum of a subarray *ending* at index i. It either consists of a_i alone or extends the best subarray ending at $i - 1$: $b_i = \max(a_i,\, b_{i-1} + a_i)$. The answer is $\max_i b_i$. Start with the first element (not with 0), so the all-negative case works.

 Juevesano D. 2026 All rights reserved.



$b[i]$ is the best sum of a subarray ending at i: $b[i] = \max(a[i],\, b[i-1] + a[i])$. The answer is the largest $b[i]$, here 6, the subarray $[4, -1, 2, 1]$.

```
// Largest sum of a non-empty contiguous subarray (n >= 1).
int maxSubarray(int a[], int n) {
    int endingHere = a[0];           // best sum of a subarray ending at i
    int best = a[0];
    for (int i = 1; i < n; i++) {
        if (endingHere > 0) endingHere = endingHere + a[i];   // extend
        else endingHere = a[i];                               // start over
        if (endingHere > best) best = endingHere;
    }
    return best;
}

// Slower version, O(n^2): try every starting index with a running sum.
int maxSubarraySlow(int a[], int n) {
    int best = a[0];
    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = i; j < n; j++) {
            sum = sum + a[j];
            if (sum > best) best = sum;
        }
    }
    return best;
}
```

Listing 12.9: Maximum subarray sum

Complexity. $\Theta(n)$ time, $O(1)$ space.

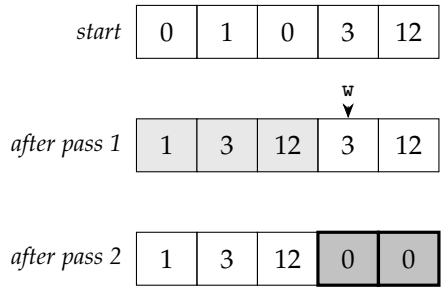
Common mistakes.

- Initialising the best value with 0, which returns 0 for an all-negative array (the subarray must be non-empty).
- Resetting the running sum to 0 and forgetting to update the answer first.

Solution to Problem 10.

Idea. Let `w` be the next free slot for a non-zero item. One pass copies each non-zero item to `a[w++]`; a second loop fills `a[w..n-1]` with zeros.

Juevesano D. 2026 All rights reserved.



Pass 1 copies the non-zero items to the front (w counts them); pass 2 fills the rest with zeros. The order of the non-zero items never changes.

```
// Moves all zeros to the end; the other items keep their order.
void moveZeroes(int a[], int n) {
    int w = 0;                          // next place for a non-zero item
    for (int r = 0; r < n; r++) {
        if (a[r] != 0) {
            a[w] = a[r];
            w++;
        }
    }
    while (w < n) {
        a[w] = 0;
        w++;
    }
}
```

Listing 12.10: Move zeroes

Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes.

- Shifting the whole suffix for every zero ($O(n^2)$).
- Destroying the order of the non-zero items.

Solution to Problem 11.

Idea. Keep a candidate and a counter. For each item: if the counter is 0, adopt the item as the candidate; if the item equals the candidate increment the counter, else decrement it. Pairs of different items cancel; a majority element has more than half of the items, so it survives all the cancellations.

 Juevesano D. 2026 All rights reserved.

$a[i]$	3	3	4	2	4	4	2	4	4
	0	1	2	3	4	5	6	7	8
candidate	3	3	3	3	4	4	4	4	4
count	1	2	1	0	1	2	1	2	3

Boyer–Moore voting on $[3, 3, 4, 2, 4, 4, 2, 4, 4]$*: different items cancel in pairs, and the majority element survives all the cancellations.*

```
// Returns the element that appears more than n/2 times (it always exists).
// Boyer-Moore voting: different items cancel each other in pairs.
int majority(int a[], int n) {
    int candidate = a[0];
    int votes = 0;
    for (int i = 0; i < n; i++) {
        if (votes == 0) candidate = a[i];
        if (a[i] == candidate) votes++;
        else votes--;
    }
    return candidate;
}

// Slower version, O(n^2): count every item.
int majoritySlow(int a[], int n) {
    for (int i = 0; i < n; i++) {
        int c = 0;
        for (int j = 0; j < n; j++) {
            if (a[j] == a[i]) c++;
        }
        if (c > n / 2) return a[i];
    }
    return -1;
}
```

Listing 12.11: Majority element

Complexity. $\Theta(n)$ time, $O(1)$ space.

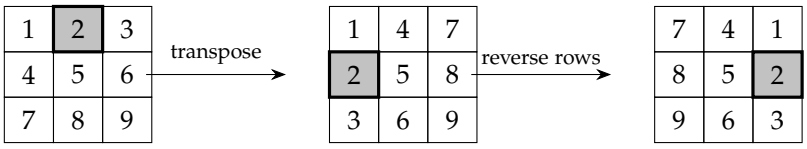
Common mistakes.

- Sorting to find the middle element ($\Theta(n \log n)$) when linear time is required.
- Not updating the candidate when the counter reaches 0.

Solution to Problem 12.

Idea. A clockwise rotation is a transpose followed by a reversal of every row: the element at (i, j) goes to (j, i) by the transpose and then to $(j, n-1-i)$ by reversing row j, which is where a clockwise rotation puts it.

Juevesano D. 2026 All rights reserved.



Clockwise rotation = transpose, then reverse every row.

```
// Rotates an n x n matrix 90 degrees clockwise, in place.
// Clockwise rotation = transpose, then reverse every row.
void rotate90(int a[][10], int n) {
    for (int i = 0; i < n; i++) {             // transpose
        for (int j = i + 1; j < n; j++) {
            int tmp = a[i][j];
            a[i][j] = a[j][i];
            a[j][i] = tmp;
        }
    }
    for (int i = 0; i < n; i++) {             // reverse each row
        int lo = 0;
        int hi = n - 1;
        while (lo < hi) {
            int tmp = a[i][lo];
            a[i][lo] = a[i][hi];
            a[i][hi] = tmp;
            lo++;
            hi--;
        }
    }
}
```

Listing 12.12: Rotate a matrix by 90 degrees

Complexity. $\Theta(n^2)$ time, $O(1)$ space.

Common mistakes.

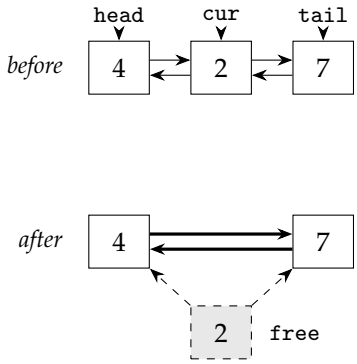
- Reversing columns instead of rows, which gives the counter-clockwise rotation.
- Transposing the whole square, which undoes the transpose.

12.2 Linked Lists

Solution to Problem 13.

Idea. Walk a pointer `cur` from the head. *Before* touching `cur`, save `nxt = cur->next`, because `cur` may be freed. If `cur->item==x`, unlink it: the predecessor's `next` (or `l->head` when there is no predecessor) skips `cur`, and the successor's `prev` (or `l->tail` when there is no successor) skips it too; then free the node and decrement `size`. Continue with `nxt`. The four cases (head, tail, middle, only node) are all covered by the two `if/else` pairs, so no separate case analysis is needed. This is the same structure as the reference solution of the examination.

 Juevesano D. 2026 All rights reserved.



Erasing the node `cur` *holding* 2*: the neighbours are joined first, then* `cur` *is freed. Save* `cur->next` before *the free.*

```c
// Removes every node whose item equals x from the doubly linked list.
void eraseItems(list *l, int x) {
    node *cur = l->head;

    while (cur != NULL) {
        node *nxt = cur->next;       // save it: cur may be freed below

        if (cur->item == x) {
            // make the node before cur skip it (or move the head)
            if (cur->prev != NULL) cur->prev->next = cur->next;
            else l->head = cur->next;

            // make the node after cur skip it (or move the tail)
            if (cur->next != NULL) cur->next->prev = cur->prev;
            else l->tail = cur->prev;

            free(cur);
            l->size--;
        }
        cur = nxt;
    }
}
```

Listing 12.13: Implementation: eraseItems (DLL)

Trace of the fourth sample.. List $1\,1\,3\,4\ldots10$ with $x = 1$: the first node is the head, so `l->head` moves to the second node (whose `prev` is then set to `NULL` by the successor update); the second node is the new head, and is removed in the same way. After that no node equals 1. The final `size` is $10 - 2 = 8$.

Why `while (cur != NULL)` with a saved `nxt`.. Reading `cur->next` after `free(cur)` is undefined behaviour; sample 3 (all items equal) exercises exactly this and removes the head five times in a row.

Complexity. $\Theta(n)$ time, $O(1)$ extra space.

Common mistakes.

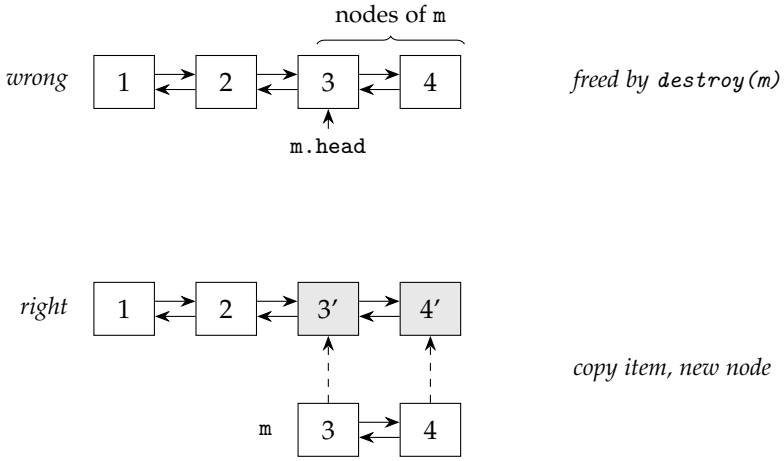
- Reading `cur->next` after `free(cur)`.

Juevesano D. 2026 All rights reserved.

- Forgetting to update `l->tail` when the last node is removed, or `l->head` when the first is (sample 3 checks that both become `NULL`).
- Updating only the forward links, leaving the backward traversal printing freed or stale nodes.
- Not decrementing `size`.

Solution to Problem 14.

IDEA. `m` is passed *by value*: the function receives a copy of the struct, but the copy's `head` still points to the caller's nodes. Linking those very nodes into `l` would make `l` share nodes with `m`, and destroying `m` would free memory that `l` still uses (a dangling pointer). So the function performs a *deep copy*: for each of the `m.size` nodes, allocate a new node, copy the item, and attach it after the current tail of `l`, setting `prev` to the old tail. Edge cases: `l` empty (the first new node becomes the head as well as the tail) and `m` empty (the loop runs zero times and `l` is unchanged).



Top: copying only the head pointer *of `m` shares its nodes with `l`; after `destroy(m)` the end of `l` dangles. Bottom: new nodes (shaded) are created for every item of `m`.*

```
// Appends a copy of every item of m to the end of l (without append).
// m is a copy of the caller's struct, but its nodes are the caller's nodes,
// so we must create NEW nodes: otherwise destroying m would free l's nodes.
void appendList(list *l, list m) {
    node *src = m.head;

    for (int k = 0; k < m.size; k++) {     // count; do not stop at NULL
        node *copy = (node *)malloc(sizeof(node));
        copy->item = src->item;
        copy->next = NULL;
        copy->prev = l->tail;

        if (l->tail == NULL) l->head = copy;   // l was empty
        else l->tail->next = copy;
        l->tail = copy;
        l->size++;
```

 Juevesano D. 2026 All rights reserved.

```
17
18          src = src->next;
19      }
20  }
```

Listing 12.14: Implementation: appendList (copying a list)

Why the loop is counted.. If the caller writes `appendList(&x, x)`, the parameter `m` is a copy whose `head` is the head of the same chain that `l` extends. Walking `m` *until* `NULL` would follow the nodes that were just appended, and never terminate. Walking exactly `m.size` nodes (the size captured by the copy) copies only the original nodes.

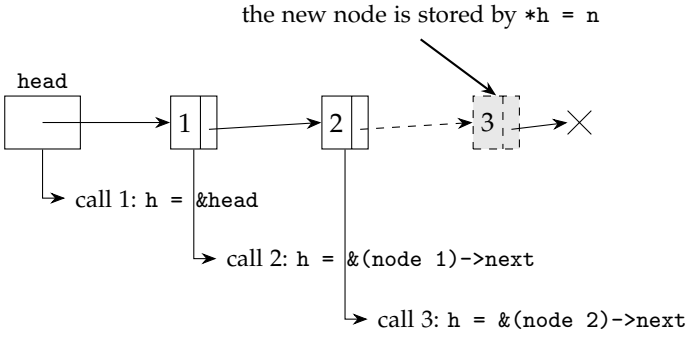
Complexity. $\Theta(|m|)$ time and $\Theta(|m|)$ extra space for the new nodes.

Common mistakes.

- Linking `m.head` to `l->tail->next` (a shallow copy): correct output, but freed memory after `m` is destroyed.
- Not handling an empty `l` (`l->tail` is `NULL`, and dereferencing it crashes).
- Forgetting to set `prev` of the first appended node, or `l->tail` and `l->size` at the end.
- Calling `append` for every item (forbidden here).

Solution to Problem 15.

Idea. Think of `*h` as *the pointer that points to the rest of the list*. If `*h==NULL` we stand where the new node must go: allocate it and store it *into* `*h`. Otherwise the list continues, so recurse on the address of the `next` field of the first node, `&((*h)->next)`. Because the recursive call receives the address of that field, the assignment `*h = n` in the base case writes into the previous node's `next`, or into `l.head` itself when the list was empty. No special case for the empty list is needed.



`h` always holds the address of a pointer*: first of `head`, then of the `next` field of node* 1*, then of node* 2*. The base case stores the new node through `*h`, which changes the real pointer.*

```
1  // Appends x at the end of a list that has no tail pointer.
2  // h is the address of the pointer that points to the rest of the list.
3  void append(snode **h, int x) {
4      if (*h == NULL) {                  // reached the end: put the node here
```

```
5           snode *n = (snode *)malloc(sizeof(snode));
6           n->item = x;
7           n->next = NULL;
8           *h = n;
9       } else {                            // otherwise append to the rest
10          append(&((*h)->next), x);
11      }
12  }
```

Listing 12.15: Implementation: recursive append with a double pointer

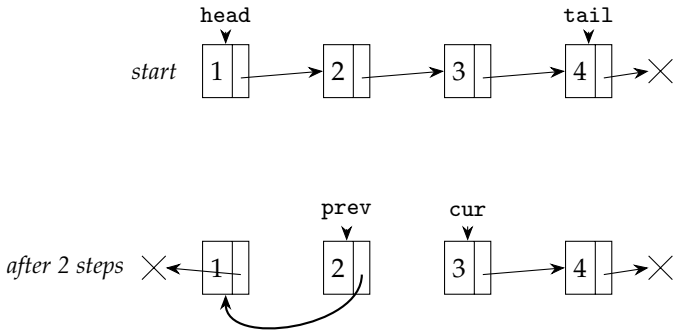
Complexity. $\Theta(n)$ time; recursion depth n (stack space $\Theta(n)$). A loop version needs $O(1)$ space.

Common mistakes.

- Declaring `snode *h` (a single pointer): the assignment in the base case changes a local copy (Tracing 7).
- Recursing with `(*h)->next` instead of its *address*.
- Forgetting to set the new node's `next` to `NULL`.

Solution to Problem 16.

Idea. Maintain `prev` (initially `NULL`) and `cur` (initially the head). At each step save `nxt = cur->next`, point `cur->next` at `prev`, then advance both. Invariant: the nodes before `cur` already form a reversed list headed by `prev`. When `cur` is `NULL`, `prev` is the new head, and the old head is the new tail. The recursive version reverses the rest of the list, then appends the first node at its end: `h->next->next = h; h->next = NULL`.



After two iterations the first two links already point backwards. `prev` is the head of the reversed part, `cur` the first node not yet handled; `nxt` is saved before `cur->next` is overwritten.

```
1  // Reverses the list in place by changing the links.
2  void reverseList(sll *l) {
3      snode *prev = NULL;
4      snode *cur = l->head;
5
6      while (cur != NULL) {
7          snode *nxt = cur->next;      // remember the rest
```

 Juevesano D. 2026 All rights reserved.

```
8           cur->next = prev;            // point backwards
9           prev = cur;
10          cur = nxt;
11      }
12      l->tail = l->head;               // the old head is the new tail
13      l->head = prev;
14  }
15
16  // Bonus: recursive version. Returns the new head of the reversed list.
17  snode *reverseRec(snode *h) {
18      if (h == NULL || h->next == NULL) return h;
19      snode *newHead = reverseRec(h->next);
20      h->next->next = h;               // the next node now points back to h
21      h->next = NULL;
22      return newHead;
23  }
```

Listing 12.16: Reverse a singly linked list

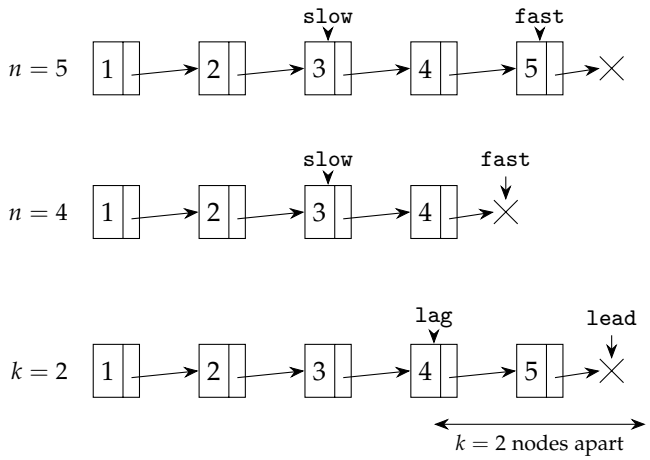
Complexity. Iterative: $\Theta(n)$ time, $O(1)$ space. Recursive: $\Theta(n)$ time, $\Theta(n)$ stack.

Common mistakes.

- Not saving `cur->next` before overwriting it (the rest of the list is lost).
- Forgetting to set the old head's `next` to `NULL` (a cycle).
- Forgetting to update `tail`.

Solution to Problem 17.

Idea. *Middle.* Two pointers: `slow` advances one node, `fast` two, per iteration. When `fast` cannot advance further, `slow` has covered half the distance. With the loop condition `fast!=NULL && fast->next!=NULL`, an odd list leaves `slow` at the exact middle and an even list at the second middle. *k-th from the end.* First advance `lead` by k nodes (failing if the list is shorter), then advance `lead` and `lag` together until `lead` is `NULL`: the two pointers remain k nodes apart, so `lag` stops on the k-th node from the end.



`slow` moves one node and `fast` two: when `fast` runs off the end, `slow` is in the middle (second middle for even n). For the k-th from the end, `lead` goes k nodes ahead, then both move until `lead` falls off: `lag` is the answer.

Juevesano D. 2026 All rights reserved.

```c
// Returns the middle item (the second one if the length is even).
// slow moves 1 node per step, fast moves 2, so slow is halfway when fast ends.
int middleItem(sll *l) {
    snode *slow = l->head;
    snode *fast = l->head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow->item;
}

// Stores the k-th item from the end (k = 1 is the last) in *out.
// Returns 1 on success, 0 if k < 1 or k > the length.
int kthFromEnd(sll *l, int k, int *out) {
    if (k < 1) return 0;
    snode *lead = l->head;
    for (int i = 0; i < k; i++) {    // lead goes k nodes ahead
        if (lead == NULL) return 0;
        lead = lead->next;
    }
    snode *lag = l->head;
    while (lead != NULL) {           // both move until lead falls off
        lead = lead->next;
        lag = lag->next;
    }
    *out = lag->item;
    return 1;
}
```

Listing 12.17: Middle node and k-th node from the end

Complexity. $\Theta(n)$ time, $O(1)$ space; exactly one pass.

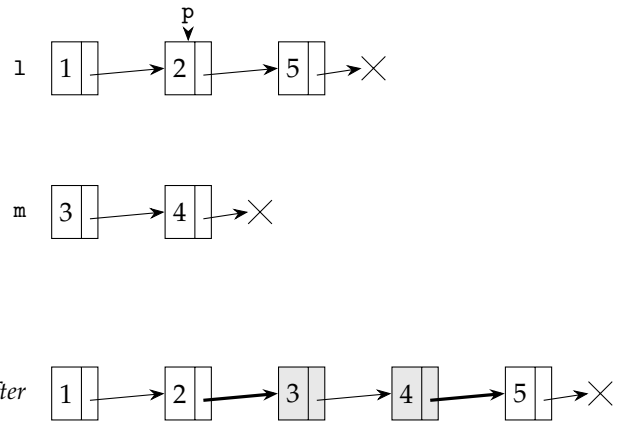
Common mistakes.

- Dereferencing `fast->next->next` without testing `fast->next`.
- Off-by-one in the gap: `k` nodes ahead, not $k-1$ or $k+1$.
- Not rejecting $k = 0$ or $k > n$.

Solution to Problem 18.

Idea. Four pointers change. Let p be the anchor, $q = p.next$, and let m have nodes $m_1 \dots m_k$. Set $p.next = m_1$ and $m_k.next = q$. If q was `NULL` (we spliced at the end), the tail of `l` becomes m_k; if p was `NULL` the head of `l` becomes m_1. Add the sizes and clear `m`. Edge cases: `m` empty (do nothing), `l` empty. For `moveRange`, cut the nodes $i..j-1$ out by relinking the predecessor of node i to node j, then append the cut chain to `dst`, whose tail pointer lets this be $O(1)$.

 Juevesano D. 2026 All rights reserved.



Only two pointers change: the node before the splice point now leads into `m`, and the last node of `m` leads to what used to follow. Afterwards `m` is empty.

```
// Moves all nodes of m into l, right after node p (p == NULL: at the front).
// No node is created or freed; only pointers change. m becomes empty.
void spliceAfter(sll *l, snode *p, sll *m) {
    if (m->head == NULL) return;                // nothing to move

    snode *after = (p == NULL) ? l->head : p->next;
    if (p == NULL) l->head = m->head;           // spliced at the front
    else p->next = m->head;
    m->tail->next = after;
    if (after == NULL) l->tail = m->tail;       // spliced at the end

    l->size = l->size + m->size;
    m->head = NULL;
    m->tail = NULL;
    m->size = 0;
}

// Same, but pos nodes of l stay in front (0 <= pos <= size).
// Returns 0 if pos is invalid.
int splice(sll *l, int pos, sll *m) {
    if (pos < 0 || pos > l->size) return 0;
    snode *p = NULL;                            // the node at index pos - 1
    if (pos > 0) {
        p = l->head;
        for (int i = 1; i < pos; i++) p = p->next;
    }
    spliceAfter(l, p, m);
    return 1;
}
```

Listing 12.18: Splicing singly linked lists

Complexity. Relinking is $\Theta(1)$ once the anchor is known; finding the anchor by index is $\Theta(pos)$. No node is allocated.

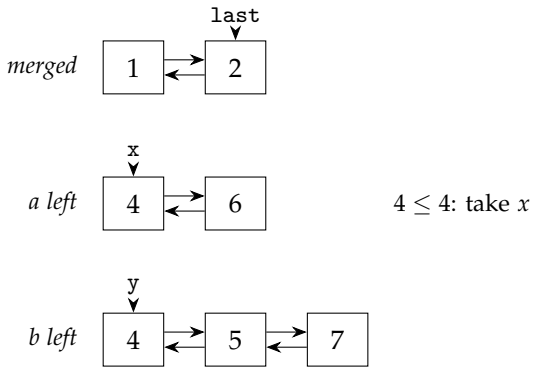
Common mistakes.

Juevesano D. 2026 All rights reserved.

- Forgetting that the tail of `l` changes when splicing at the end (the next `pushBack` then corrupts the list).
- Leaving `m` non-empty: the same nodes are then reachable from two lists, and freeing both is a double free.
- Copying the items instead of moving the nodes ($\Theta(|m|)$ and not a splice).

Solution to Problem 19.

IDEA. Use a `tail`-building technique: keep pointers `x` into `a` and `y` into `b`, and a pointer `last` to the last node of the merged chain. Repeatedly detach the smaller front node (taking `a`'s on ties) and link it after `last`, setting both `next` and `prev`. When one list is exhausted, the other is already linked and sorted, so attach it whole; its old tail is the new tail. Finally set the fields of `a` and empty `b`.



The merged chain is built node by node by taking the smaller front node; on a tie the node of `a` is taken (`x->item <= y->item`). Here 1 *and* 2 *are already placed and the tie* 4/4 *is decided in favour of `a`.*

```
// Merges the sorted lists a and b by relinking nodes. a gets all nodes, b is
// left empty. On equal items the node of a goes first.
void mergeSorted(list *a, list *b) {
    node *x = a->head;
    node *y = b->head;
    node *head = NULL;              // head and last build the merged chain
    node *last = NULL;

    while (x != NULL || y != NULL) {
        node *take;
        if (y == NULL || (x != NULL && x->item <= y->item)) {
            take = x;
            x = x->next;
        } else {
            take = y;
            y = y->next;
        }
        take->prev = last;
        take->next = NULL;
        if (last == NULL) head = take;
        else last->next = take;
```

 Juevesano D. 2026 All rights reserved.

```
        last = take;
    }
    a->head = head;
    a->tail = last;
    a->size = a->size + b->size;
    b->head = NULL;
    b->tail = NULL;
    b->size = 0;
}
```

Listing 12.19: Merge two sorted doubly linked lists

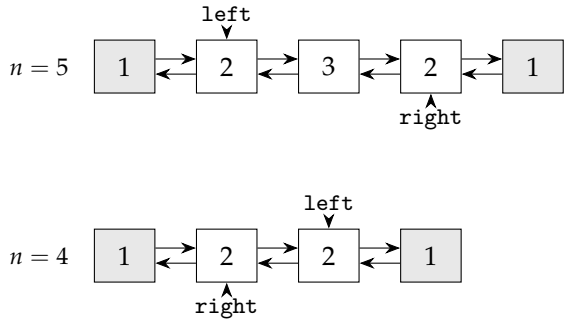
Complexity. $\Theta(|a| + |b|)$ time, $O(1)$ extra space.

Common mistakes.

- Setting only `next`, leaving every `prev` stale.
- Forgetting to update `tail` when the remainder comes from `b`.
- Leaving `b` pointing at nodes that now belong to `a`.

Solution to Problem 20.

Idea. Place one pointer at the head and one at the tail. While they have not met or crossed, compare the items; if they differ, return 0; otherwise move the left pointer forward and the right pointer backward. They have crossed when `left==right` (odd length) or `left->prev==right` (even length). The backward link is what makes this possible in $O(1)$ space.



`left` and `right` walk towards each other, comparing items. They stop when they meet (odd length) or when they have crossed, i.e. `left->prev == right` (even length).

```
// Returns 1 if the list reads the same forwards and backwards.
int isPalindrome(list *l) {
    node *left = l->head;
    node *right = l->tail;

    // stop when the two pointers meet or cross
    while (left != right && left->prev != right) {
        if (left->item != right->item) return 0;
        left = left->next;
```

```
        right = right->prev;
    }
    return 1;
}
```

Listing 12.20: Palindromic doubly linked list

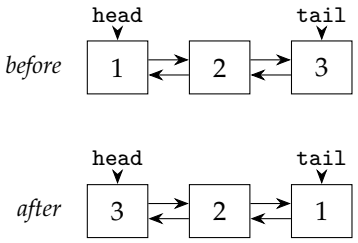
Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes.

- Termination test `left!=right` only: for even lengths the pointers cross without meeting and the loop runs off the list.
- Dereferencing `head` of an empty list.

Solution to Problem 21.

Idea. Every node simply swaps its `next` and `prev`; then `head` and `tail` are swapped. No traversal order matters, but the walk must read the old `next` *before* the swap.



Every node swaps its `next` and `prev`; `head` and `tail` swap places. No item moves.

```
// Reverses the list in place: every node swaps its next and prev.
void reverseDLL(list *l) {
    node *cur = l->head;
    while (cur != NULL) {
        node *nxt = cur->next;       // read before swapping
        cur->next = cur->prev;
        cur->prev = nxt;
        cur = nxt;
    }
    node *tmp = l->head;             // head and tail trade places
    l->head = l->tail;
    l->tail = tmp;
}
```

Listing 12.21: Reverse a doubly linked list

Complexity. $\Theta(n)$ time, $O(1)$ space.

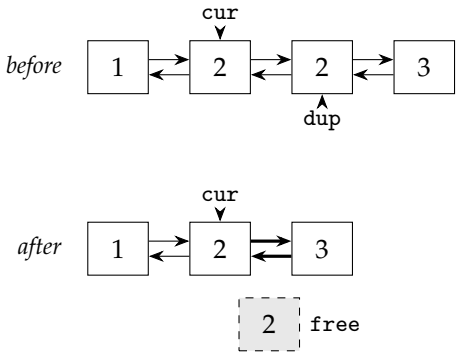
Common mistakes.

- Walking with `cur=cur->next` after the swap (it now goes backwards).
- Forgetting to swap `head` and `tail`.

 Juevesano D. 2026 All rights reserved.

Solution to Problem 22.

IDEA. Because the list is sorted, duplicates are adjacent. Walk with `cur`; while `cur->next` holds the same value as `cur`, unlink and free `cur->next`. Update `tail` if the last node was removed.



The duplicate is unlinked and freed; `cur` does not *advance, because the new `cur->next` may be a duplicate too.*

```
// Keeps one node for each value of a sorted list.
void removeDuplicates(list *l) {
    node *cur = l->head;
    while (cur != NULL && cur->next != NULL) {
        if (cur->next->item == cur->item) {
            node *dup = cur->next;           // unlink the duplicate
            cur->next = dup->next;
            if (dup->next != NULL) dup->next->prev = cur;
            else l->tail = cur;
            free(dup);
            l->size--;
        } else {
            cur = cur->next;                 // advance only when no removal
        }
    }
}
```

Listing 12.22: Remove duplicates from a sorted doubly linked list

COMPLEXITY. $\Theta(n)$ time, $O(1)$ space.

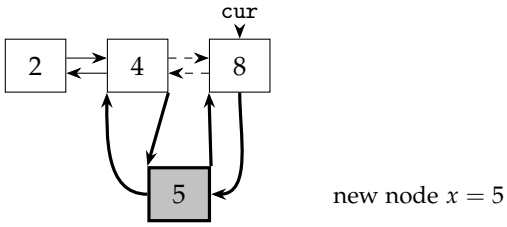
COMMON MISTAKES.

- Advancing `cur` after every removal, which skips comparisons.
- Not updating `tail`.

Solution to Problem 23.

IDEA. Find the first node whose item is greater than x. If there is none, x becomes the new tail; if it is the head, x becomes the new head; otherwise link x between that node's predecessor and the node, setting four pointers.

Juevesano D. 2026 All rights reserved.



Walk until `cur` is the first node greater than x (or `NULL`). The new node goes between `cur->prev` and `cur`: four pointers are set (thick).

```
// Inserts x into the sorted list, after any equal items.
void insertSorted(list *l, int x) {
    node *n = (node *)malloc(sizeof(node));
    n->item = x;

    node *cur = l->head;
    while (cur != NULL && cur->item <= x) {  // first node greater than x
        cur = cur->next;
    }
    // n goes between cur->prev and cur
    n->next = cur;
    if (cur != NULL) {
        n->prev = cur->prev;
        cur->prev = n;
    } else {                                 // x is the new maximum
        n->prev = l->tail;
        l->tail = n;
    }
    if (n->prev != NULL) n->prev->next = n;
    else l->head = n;                        // x is the new minimum
    l->size++;
}
```

Listing 12.23: Insert into a sorted doubly linked list

Complexity. $\Theta(n)$ worst case.

Common mistakes.

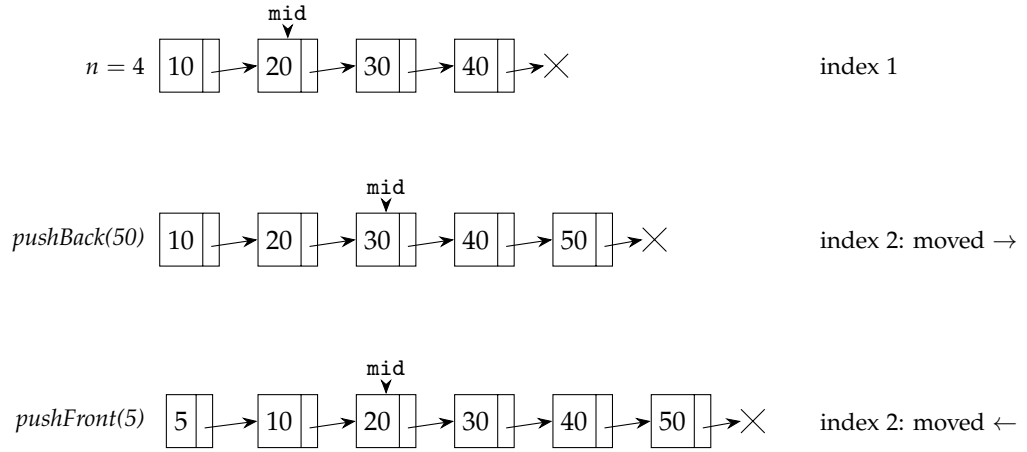
- Missing one of the four pointer assignments in the middle case.
- Not updating `head` when inserting at the front, or `tail` at the end.

Solution to Problem 24.

Idea. Let n be the size before the operation and $\mu(n) = \lfloor (n-1)/2 \rfloor$. Each operation changes the index that `mid` must have by -1, 0 or $+1$, relative to the node it currently points to; the only question is the direction. *Forward* moves cost $O(1)$ (`mid=mid->next`); *backward* moves need the predecessor, which an SLL does not have, so they cost a walk from `head`. Working out the cases (the table below): `mPushBack` adds a node after the middle, so `mid` moves forward when n is even and stays when n is odd; `mPopFront` shifts every index down by one, so `mid` moves forward when n is even and stays when n is odd;

 Juevesano D. 2026 All rights reserved.

`mPushFront` shifts every index up, so it stays when n is even and needs a step *backward* when n is odd; `mPopBack` leaves the indices alone but shrinks the list, so again `mid` stays for even n and steps backward for odd n — and finding the new tail already costs $\Theta(n)$ in an SLL. `middle` simply reads `mid->item`.



`mid` always points at index $\lfloor (n-1)/2 \rfloor$. `pushBack` on an even size moves it one node right *(cheap); `pushFront` on an odd size moves it one node* left*, and a singly linked list cannot step back, so it is found again from the head.*

Operation	n even	n odd	Worst case
`mPushBack`	`mid=mid->next`	`mid` stays	$O(1)$
`mPopFront`	`mid=mid->next`	`mid` stays	$O(1)$
`mPushFront`	`mid` stays	one step *back*	$O(1)$ / $O(n)$
`mPopBack`	`mid` stays	one step *back*	$O(n)$ (finding the new tail)
`middle`	read `mid->item`		$O(1)$

```
typedef struct mnode {
    int item;
    struct mnode *next;
} mnode;

typedef struct {
    mnode *head;
    mnode *mid;      // the node at index (size - 1) / 2
    mnode *tail;
    int size;
} mlist;

void minit(mlist *l) {
    l->head = NULL;
    l->mid = NULL;
    l->tail = NULL;
    l->size = 0;
}

// O(1)
```

```c
int middle(mlist *l) {
    return l->mid->item;
}

// O(1): the new node is after mid; mid moves right when size was even.
void mPushBack(mlist *l, int x) {
    mnode *n = (mnode *)malloc(sizeof(mnode));
    n->item = x;
    n->next = NULL;
    if (l->size == 0) {
        l->head = n;
        l->mid = n;
    } else {
        l->tail->next = n;
        if (l->size % 2 == 0) l->mid = l->mid->next;
    }
    l->tail = n;
    l->size++;
}

// O(1) if size is even; O(n) if odd, because mid must move one node LEFT.
void mPushFront(mlist *l, int x) {
    mnode *n = (mnode *)malloc(sizeof(mnode));
    n->item = x;
    n->next = l->head;
    l->head = n;
    if (l->size == 0) {
        l->mid = n;
        l->tail = n;
    } else if (l->size % 2 == 1) {
        l->mid = l->head;                    // walk from the head
        for (int i = 0; i < l->size / 2; i++) l->mid = l->mid->next;
    }
    l->size++;
}

// O(1): mid moves right when size was even.
int mPopFront(mlist *l, int *x) {
    if (l->size == 0) return 0;
    mnode *old = l->head;
    *x = old->item;
    l->head = old->next;
    if (l->size == 1) {
        l->mid = NULL;
        l->tail = NULL;
    } else if (l->size % 2 == 0) {
        l->mid = l->mid->next;
    }
    free(old);
    l->size--;
    return 1;
}

```

 Juevesano D. 2026 All rights reserved.

```c
// O(n): the new tail is the node before the old tail, which needs a walk.
// If size was odd, mid also moves one node LEFT.
int mPopBack(mlist *l, int *x) {
    if (l->size == 0) return 0;
    *x = l->tail->item;
    if (l->size == 1) {
        free(l->head);
        l->head = NULL;
        l->mid = NULL;
        l->tail = NULL;
    } else {
        mnode *prev = l->head;               // will be the node before tail
        while (prev->next != l->tail) prev = prev->next;
        free(l->tail);
        prev->next = NULL;
        l->tail = prev;
        if (l->size % 2 == 1) {              // recompute mid from the head
            l->mid = l->head;
            for (int i = 0; i < (l->size - 2) / 2; i++) l->mid = l->mid->next;
        }
    }
    l->size--;
    return 1;
}
```

Listing 12.24: An SLL with a middle pointer

Why the table is what it is.. For `mPopFront`, the new list has $n-1$ items and the middle must be at index $\lfloor (n-2)/2 \rfloor$ of the new list, i.e. at the old index $\lfloor (n-2)/2 \rfloor + 1$. For even n this is $n/2$, one more than the old $\lfloor (n-1)/2 \rfloor = n/2 - 1$, so `mid` moves forward; for odd n it equals the old index and `mid` stays. The other rows follow in the same way. When the list becomes empty, all pointers are set to `NULL`.

Why a singly linked list cannot do better.. A step *back* of `mid` needs the previous node, which an SLL cannot reach in $O(1)$; likewise the new tail after `mPopBack`. With a `prev` pointer in every node, `mid=mid->prev` is $\Theta(1)$ and so are all four operations.

Complexity. `middle`, `mPushBack`, `mPopFront`: $\Theta(1)$. `mPushFront`: $\Theta(1)$ for even n, $\Theta(n)$ for odd n (in the listing `mid` is found again from the head). `mPopBack`: $\Theta(n)$. A stress test of 20 000 random operations against an array model passes under the sanitizers.

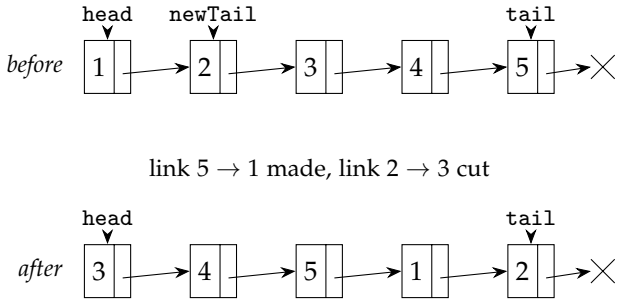
Common mistakes.

- Updating `mid` before updating `size` (or the reverse) and so using the wrong parity.
- Forgetting the one-node and empty cases (`mid` must become `NULL` with `head/tail`).
- Claiming that all five operations are $O(1)$ in a singly linked list; `mPopBack` is $\Theta(n)$ regardless of `mid`.

Solution to Problem 25.

Juevesano D. 2026 All rights reserved.

Idea. Replace k by k mod n; if it is 0 nothing changes. Walk to the node at index $k-1$ (the new tail); the node after it becomes the head; link the old tail to the old head and cut the list at the new tail. The old tail and the new tail are the two affected pointers.



Rotating left by $k = 2$: the k-th node becomes the new tail, the old tail is linked to the old head (closing a ring), and the ring is cut after the new tail.

```
// Rotates the list left by k places: [1 2 3 4 5], k = 2 -> [3 4 5 1 2].
void rotateLeft(sll *l, int k) {
    if (l->size == 0) return;
    k = k % l->size;
    if (k == 0) return;

    snode *newTail = l->head;                // the k-th node
    for (int i = 1; i < k; i++) newTail = newTail->next;

    l->tail->next = l->head;                 // old tail points to old head
    l->head = newTail->next;                 // the node after becomes head
    newTail->next = NULL;
    l->tail = newTail;
}
```

Listing 12.25: Rotate a linked list

Complexity. $\Theta(n)$ time, $O(1)$ space.

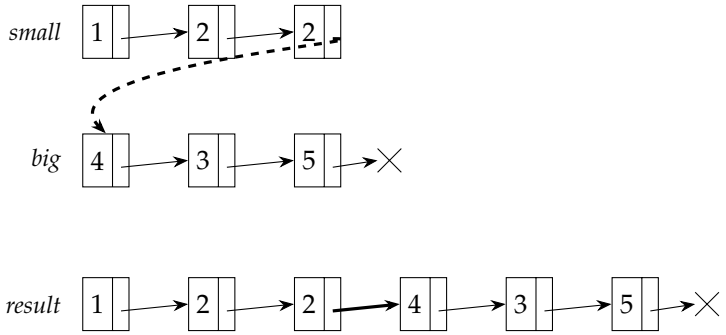
Common mistakes.

- Forgetting k mod n.
- Not setting the new tail's `next` to `NULL` (a cycle).
- Not updating `head` and `tail`.

Solution to Problem 26.

Idea. Build two chains in one pass, one for the nodes $< x$ and one for the others, each with its own dummy head and tail pointer so that appending is $O(1)$ and the order is preserved. Finally link the end of the first chain to the head of the second, terminate the second chain with `NULL`, and repair `head` and `tail`.

 Juevesano D. 2026 All rights reserved.



Two chains are built in one pass (nodes with item < 3, and the others), each keeping the original order; then the end of the first chain is linked to the start of the second.

```
// Moves nodes with item < x in front of the others; order inside each group
// is kept. Two chains are built and joined; no node is created.
void partitionList(sll *l, int x) {
    snode smallDummy;               // dummy heads remove the empty cases
    snode bigDummy;
    snode *small = &smallDummy;     // last node of each chain
    snode *big = &bigDummy;
    smallDummy.next = NULL;
    bigDummy.next = NULL;

    snode *cur = l->head;
    while (cur != NULL) {
        if (cur->item < x) {
            small->next = cur;
            small = cur;
        } else {
            big->next = cur;
            big = cur;
        }
        cur = cur->next;
    }
    big->next = NULL;               // end of the second chain
    small->next = bigDummy.next;    // join the two chains

    l->head = smallDummy.next;
    if (big != &bigDummy) l->tail = big;
    else if (small != &smallDummy) l->tail = small;
    else l->tail = NULL;
}
```

Listing 12.26: Partition a linked list around a value

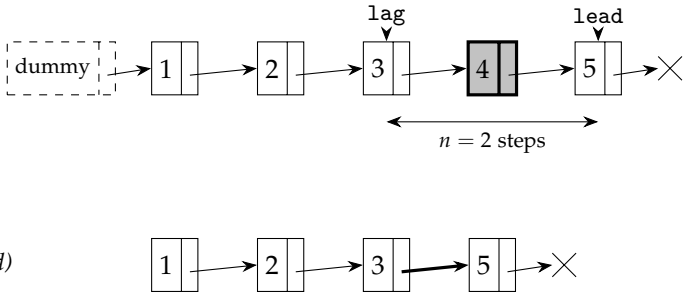
Complexity. $\Theta(n)$ time, $O(1)$ extra space.

Common mistakes.

- Forgetting to terminate the second chain (it still points into the middle of the old list: a cycle).
- Losing stability by pushing nodes to the front of a chain.

Solution to Problem 27.

IDEA. Use a dummy node in front of the head so that removing the head needs no special case. Advance `lead` by n nodes (fail if the list is too short), then advance `lead` and `lag` until `lead` is the last node; `lag->next` is then the node to remove. Unlink it, free it, and if it was the tail set `tail=lag` (or `NULL` when the list becomes empty).



`lead` is moved n nodes from the dummy; then both pointers advance until `lead` is the last node. `lag` then stands just before the node to remove, so no node is counted twice.

```
1  // Removes the n-th node from the end (n = 1 is the last) in one pass.
2  // Returns 1 on success, 0 if n < 1 or n > the length.
3  int removeNthFromEnd(sll *l, int n) {
4      if (n < 1) return 0;
5      snode dummy;                    // a dummy in front of the head means
6      dummy.next = l->head;           // the head needs no special case
7
8      snode *lead = &dummy;
9      for (int i = 0; i < n; i++) {    // lead goes n nodes ahead
10         if (lead->next == NULL) return 0;
11         lead = lead->next;
12     }
13     snode *lag = &dummy;
14     while (lead->next != NULL) {     // lag stops just before the target
15         lead = lead->next;
16         lag = lag->next;
17     }
18     snode *target = lag->next;
19     int wasTail = (target == l->tail);
20     lag->next = target->next;
21     free(target);
22     l->size--;
23
24     l->head = dummy.next;
25     if (l->head == NULL) l->tail = NULL;
26     else if (wasTail) l->tail = lag;
27     return 1;
28 }
```

Listing 12.27: Remove the n-th node from the end

COMPLEXITY. $\Theta(|l|)$ time, one pass, $O(1)$ space.

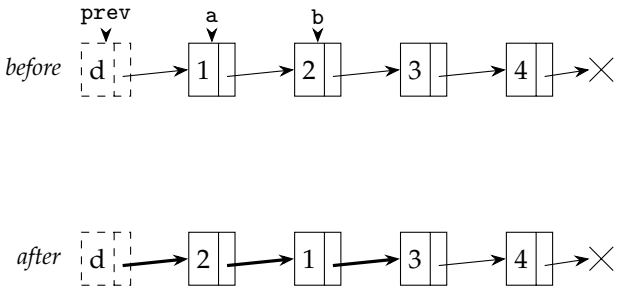
COMMON MISTAKES.

 Juevesano D. 2026 All rights reserved.

- Forgetting to update `tail` (the next `pushBack` then links to a freed node).
- Special-casing the head separately and getting the empty result wrong.

Solution to Problem 28.

Idea. With a dummy head and a pointer `prev` to the node before the pair (a, b): set `prev->next=b`, `a->next=b->next`, `b->next=a`, then move `prev` to `a`. Repeat while two nodes remain. The tail changes only if the list has an even length.



Swapping the pair `a`, `b` after `prev` takes three pointer assignments (thick), in this order: `a->next = b->next`, `b->next = a`, `prev->next = b`.

```
// Swaps every two adjacent nodes: [1 2 3 4] -> [2 1 4 3].
// The nodes move; the items stay inside their nodes.
void swapPairs(sll *l) {
    snode dummy;
    dummy.next = l->head;
    snode *prev = &dummy;            // the node before the pair

    while (prev->next != NULL && prev->next->next != NULL) {
        snode *a = prev->next;
        snode *b = a->next;
        a->next = b->next;
        b->next = a;
        prev->next = b;
        prev = a;                    // a is now the node before the next pair
    }
    l->head = dummy.next;
    // the tail is the last node reached from head
    snode *t = l->head;
    while (t != NULL && t->next != NULL) t = t->next;
    l->tail = t;
}
```

Listing 12.28: Swap nodes in pairs

Complexity. $\Theta(n)$ time, $O(1)$ space.

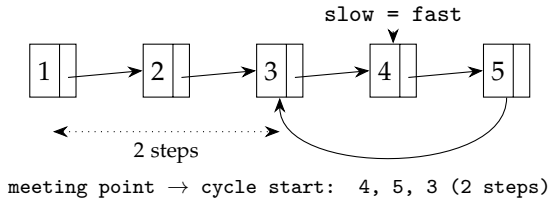
Common mistakes.

- Doing the three assignments in a different order and losing `b->next`.
- Forgetting that the tail changes for even n.

Juevesano D. 2026 All rights reserved.

Solution to Problem 29.

IDEA. Floyd's tortoise and hare: move `slow` by one and `fast` by two. If there is a cycle they meet inside it; if `fast` reaches `NULL`, there is none. To find the start, put one pointer back at `head` and move both by one: they meet at the first node of the cycle. (If the distance from the head to the cycle is a, the cycle length c, and they meet b steps into the cycle, then $2(a+b) = a+b+kc$, hence $a = kc - b$: the walk of a steps from the meeting point returns to the cycle start.)



`fast` gains one node per step on `slow`, so inside a cycle they must meet (here at node 4). Bonus: the distance from the head to the cycle start equals the distance from the meeting point to the cycle start, so two pointers moving one step at a time meet exactly there.

```
// Returns 1 if the list starting at head has a cycle, 0 otherwise.
// Floyd: if fast ever meets slow, there is a cycle.
int hasCycle(snode *head) {
    snode *slow = head;
    snode *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return 1;
    }
    return 0;
}

// Bonus: returns the first node of the cycle, or NULL if there is none.
snode *cycleStart(snode *head) {
    snode *slow = head;
    snode *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            snode *p = head;         // walk from the head and from the meeting
            while (p != slow) {      // point; they meet at the cycle start
                p = p->next;
                slow = slow->next;
            }
            return p;
        }
    }
    return NULL;
}
```

 Juevesano D. 2026 All rights reserved.

Listing 12.29: Cycle detection

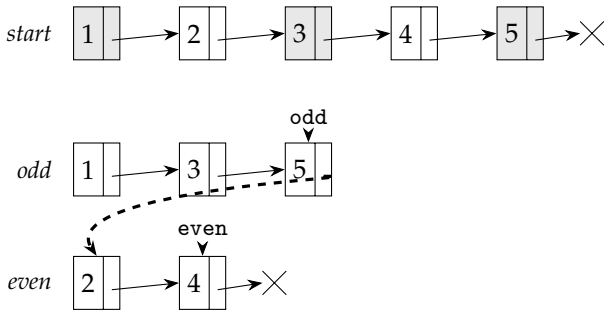
Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes.

- Testing `fast->next->next` without testing `fast->next`.
- Comparing items instead of node addresses.

Solution to Problem 30.

Idea. Two chains threaded through the same list: `odd` starts at the head, `even` at the second node. Repeatedly set `odd->next=odd->next->next`, advance `odd`, and likewise for `even`. At the end link the last odd node to the first even node.



Two chains are woven in one pass: the odd chain $(1, 3, 5)$ and the even chain $(2, 4)$. The even chain is then appended to the odd chain.

```
// Puts the nodes at odd positions (1st, 3rd, ...) before those at even
// positions: [1 2 3 4 5] -> [1 3 5 2 4]. Positions, not values.
void oddEvenList(sll *l) {
    if (l->head == NULL || l->head->next == NULL) return;

    snode *odd = l->head;               // last node of the odd chain
    snode *even = l->head->next;        // last node of the even chain
    snode *evenHead = even;
    while (even->next != NULL) {
        odd->next = even->next;         // odd chain takes the next odd node
        odd = odd->next;
        even->next = odd->next;         // even chain takes the next even node
        if (even->next != NULL) even = even->next;
    }
    odd->next = evenHead;               // odd chain, then even chain
    l->tail = even;                     // the even chain ends the whole list
}
```

Listing 12.30: Group odd and even positions

Complexity. $\Theta(n)$ time, $O(1)$ space.

Common mistakes.

- Grouping by value parity instead of position.
- Not terminating the even chain with `NULL`.

Solution to Problem 31.

IDEA. Simulate column addition: walk both lists with a carry. At each step the digit sum is $a_i + b_i +$ carry (a missing digit counts as 0); append sum mod 10 to the result and set the carry to $\lfloor \text{sum}/10 \rfloor$. Continue while either list has digits or the carry is non-zero.

carry in	0	0	1	$4 + 6 = 10$: write 0, carry 1 to the next column
a	2	4	3	
b	5	6	4	
sum	7	0	8	least significant digit first

Digits are added from the least significant one; each column adds the two digits and the incoming carry, writes `sum % 10` and carries `sum / 10`. $342 + 465 = 807$.

```
1  // Adds two numbers stored as lists of digits, least significant digit first.
2  // [2 4 3] + [5 6 4] -> [7 0 8]   (342 + 465 = 807). The sum goes to *out.
3  void addNumbers(sll *a, sll *b, sll *out) {
4      snode *x = a->head;
5      snode *y = b->head;
6      int carry = 0;
7      sinit(out);
8
9      while (x != NULL || y != NULL || carry > 0) {
10         int sum = carry;
11         if (x != NULL) {
12             sum = sum + x->item;
13             x = x->next;
14         }
15         if (y != NULL) {
16             sum = sum + y->item;
17             y = y->next;
18         }
19         pushBack(out, sum % 10);
20         carry = sum / 10;
21     }
22 }
```

Listing 12.31: Add two numbers stored in linked lists

COMPLEXITY. $\Theta(\max(|a|, |b|))$ time and space.

COMMON MISTAKES.

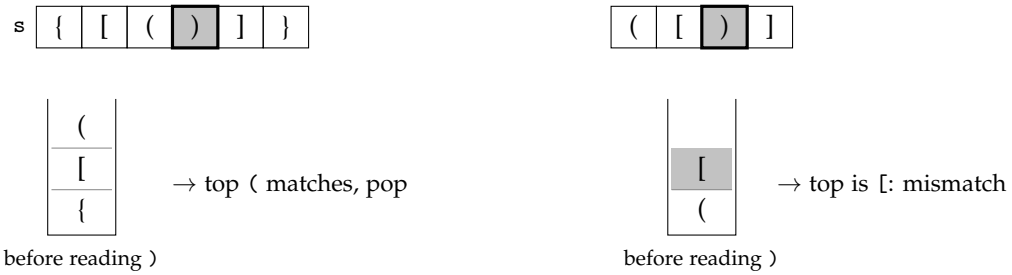
- Dropping the final carry.
- Stopping when the shorter list ends.

 Juevesano D. 2026 All rights reserved.

12.3 Stacks and Queues

Solution to Problem 32.

IDEA. Brackets close in last-opened-first-closed order, which is exactly a stack. Push each opener. For a closer, the stack must not be empty and its top must be the matching opener; pop it. At the end the stack must be empty, otherwise some opener was never closed. Counting each kind of bracket is not enough: `([)]` has balanced counts and is invalid.



Opening brackets are pushed; a closing bracket must match the top. Left: `{[()]}` reaches the `)` with `(` on top, which matches, and the stack ends empty. Right: in `([)]` the `)` meets `[` on top, a mismatch, so the answer is 0.

```
// Returns 1 if every bracket in s is closed by the same kind of bracket,
// in the right order.
// s contains only the characters ( ) [ ] { }.
int isBalanced(char s[]) {
    stack st;
    stInit(&st);

    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') {
            stPush(&st, c);                  // remember the opener
        } else {
            int open;
            if (!stPop(&st, &open)) return 0; // closer with nothing to close
            if (c == ')' && open != '(') return 0;
            if (c == ']' && open != '[') return 0;
            if (c == '}' && open != '{') return 0;
        }
    }
    return stEmpty(&st);                     // leftover openers: not balanced
}
```

Listing 12.32: Valid brackets

COMPLEXITY. $\Theta(n)$ time, $O(n)$ stack space in the worst case `((((`.

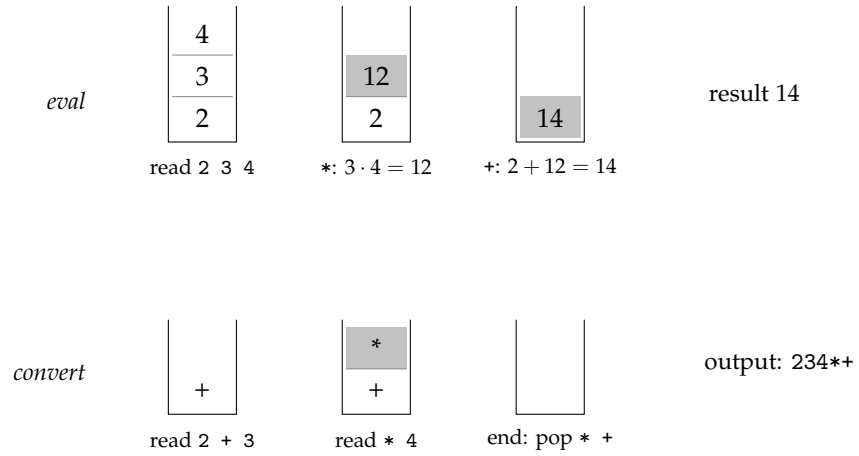
COMMON MISTAKES.

- Popping from an empty stack when a closer comes first `)(`.
- Forgetting to test for leftover openers at the end.
- Comparing only counts.

Juevesano D. 2026 All rights reserved.

Solution to Problem 33.

IDEA. *Evaluation:* push operands; on an operator pop the right operand first, then the left, apply the operator, and push the result. *Conversion:* operands go straight to the output. An operator first pops from the stack every operator that binds at least as tightly and sends it to the output, then is pushed. `(` is pushed. `)` pops to the output until the matching `(`, which is discarded. At the end pop everything.



Top: evaluating `234*+`. Operands are pushed; an operator pops two, applies itself ($3 * 4$, then $2 + 12$) and pushes the result. Bottom: converting `2+3*4`. An operator waits on the stack; `*` binds tighter than `+`, so it is pushed on top and leaves first.

```
// Precedence of an operator: * and / bind tighter than + and -.
int prec(char op) {
    if (op == '*' || op == '/') return 2;
    return 1;
}

// Converts infix to postfix. Operands are single digits; operators are
// + - * / and parentheses; there are no spaces. "2+3*4" -> "234*+".
void toPostfix(char in[], char out[]) {
    stack ops;                      // operators waiting for their operands
    int k = 0;                      // next free place in out
    stInit(&ops);

    for (int i = 0; in[i] != '\0'; i++) {
        char c = in[i];
        if (c >= '0' && c <= '9') {
            out[k++] = c;           // operands go straight to the output
        } else if (c == '(') {
            stPush(&ops, c);
        } else if (c == ')') {
            int top;                // pop until the matching '('
            while (stPop(&ops, &top) && top != '(') out[k++] = top;
        } else {
            int top;                // pop operators that must run first
            while (stTop(&ops, &top) && top != '(' && prec(top) >= prec(c)) {
```

 Juevesano D. 2026 All rights reserved.

```
                stPop(&ops, &top);
                out[k++] = top;
            }
            stPush(&ops, c);
        }
    }
    int top;
    while (stPop(&ops, &top)) out[k++] = top;
    out[k] = '\0';
}

// Evaluates a postfix expression of single digits and + - * /
// (integer division).
// "234*+" -> 14. The input is always valid.
int evalPostfix(char p[]) {
    stack st;
    stInit(&st);

    for (int i = 0; p[i] != '\0'; i++) {
        char c = p[i];
        if (c >= '0' && c <= '9') {
            stPush(&st, c - '0');
        } else {
            int b, a;                // b was pushed last: the right side
            stPop(&st, &b);
            stPop(&st, &a);
            if (c == '+') stPush(&st, a + b);
            if (c == '-') stPush(&st, a - b);
            if (c == '*') stPush(&st, a * b);
            if (c == '/') stPush(&st, a / b);
        }
    }
    int result;
    stPop(&st, &result);
    return result;
}
```

Listing 12.33: Infix to postfix and postfix evaluation

Complexity. $\Theta(n)$ time and $O(n)$ stack space.

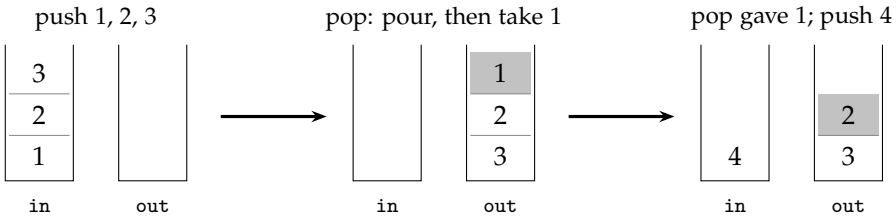
Common mistakes.

- Using $>$ instead of $\geq$ when popping equal-precedence operators, which makes `8-3-2` right-associative.
- Applying the operator with its operands swapped (the *first* popped is the right operand).
- Letting an operator pop a `(`.

Solution to Problem 34.

Idea. `tsqPush` pushes onto `in`. `tsqPop` pops from `out`; if `out` is empty it first pours *all* of `in` into `out`, which reverses the order so that the oldest item ends on top. Every item

is pushed and popped at most twice in total, so m operations cost at most $4m$ stack operations, although a single pop may cost $\Theta(n)$.



Pushes go onto `in`*. The first pop finds* `out` *empty, so all of* `in` *is poured into* `out`*, which reverses it: the oldest item* 1 *is now on top. Items are moved only when* `out` *is empty, so each moves at most once.*

```
// A queue built from two stacks. Items enter through "in" and leave through
// "out"; moving everything from in to out reverses the order, which turns
// last-in-first-out into first-in-first-out.
typedef struct {
    stack in;
    stack out;
} twoStackQueue;

void tsqInit(twoStackQueue *q) {
    stInit(&q->in);
    stInit(&q->out);
}

void tsqPush(twoStackQueue *q, int x) {
    stPush(&q->in, x);
}

// Moves items only when out is empty, so each item moves at most once.
void shift(twoStackQueue *q) {
    if (stEmpty(&q->out)) {
        int x;
        while (stPop(&q->in, &x)) stPush(&q->out, x);
    }
}

// Removes the front into *x. Returns 0 if the queue is empty.
int tsqPop(twoStackQueue *q, int *x) {
    shift(q);
    return stPop(&q->out, x);
}

// Copies the front into *x without removing it. Returns 0 if empty.
int tsqPeek(twoStackQueue *q, int *x) {
    shift(q);
    return stTop(&q->out, x);
}

int tsqEmpty(twoStackQueue *q) {
```

 Juevesano D. 2026 All rights reserved.

```
39      return stEmpty(&q->in) && stEmpty(&q->out);
40  }
```

Listing 12.34: A queue from two stacks

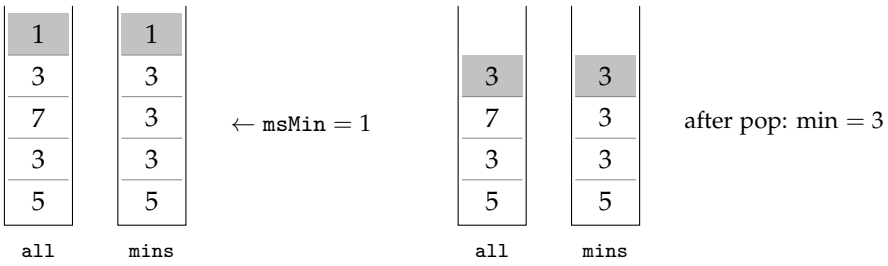
Complexity. Amortised $\Theta(1)$ per operation; the worst case of one pop is $\Theta(n)$.

Common mistakes.

- Pouring `in` into `out` when `out` is not empty (the order is destroyed).
- Popping from `in`.

Solution to Problem 35.

Idea. Keep a second stack of minima. Push x onto `all`, and push $\min(x, \text{current minimum})$ onto `mins`; pop both together. The top of `mins` is always the minimum of everything currently in `all`, because each entry records the minimum *of the stack up to and including itself*.



`mins` holds, level by level, the smallest value at or below that level of `all`. Pushing 3 *on top of* 7 *copies the current minimum* 3*; popping removes one entry from both stacks, so the minimum is always the top of `mins`.*

```
1   // A stack that also returns its minimum in O(1).
2   // "mins" holds, for every level of "all", the smallest value at or below it.
3   typedef struct {
4       stack all;
5       stack mins;
6   } minStack;
7
8   void msInit(minStack *m) {
9       stInit(&m->all);
10      stInit(&m->mins);
11  }
12
13  void msPush(minStack *m, int x) {
14      int smallest;
15      stPush(&m->all, x);
16      if (stTop(&m->mins, &smallest) && smallest < x) {
17          stPush(&m->mins, smallest);       // the old minimum still wins
18      } else {
19          stPush(&m->mins, x);              // x is the new minimum
20      }
21  }
```

```
22
23 // Removes the top into *x. Returns 0 if empty.
24 int msPop(minStack *m, int *x) {
25     int dummy;
26     if (!stPop(&m->all, x)) return 0;
27     stPop(&m->mins, &dummy);              // keep the two stacks in step
28     return 1;
29 }
30
31 // Copies the minimum into *x. Returns 0 if empty.
32 int msMin(minStack *m, int *x) {
33     return stTop(&m->mins, x);
34 }
```

Listing 12.35: A stack with constant-time minimum

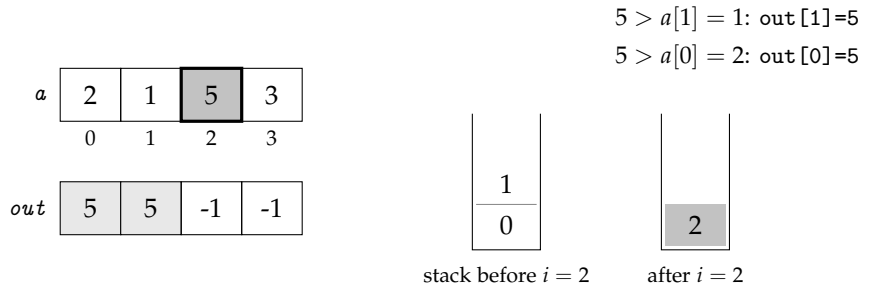
Complexity. All operations $\Theta(1)$; $\Theta(n)$ extra space.

Common mistakes.

- Storing a single `min` variable: after popping the minimum the previous minimum is lost.
- Pushing onto `mins` only for new minima and then popping it blindly.

Solution to Problem 36.

Idea. Scan left to right with a stack of indices whose answer is not yet known; their values decrease from bottom to top. When $a[i]$ arrives, every index on the stack with a smaller value has just found its answer, $a[i]$: pop them, then push i. Indices still on the stack at the end have no greater element. Each index is pushed once and popped once.



The stack holds indices whose answer is still unknown, with decreasing values from bottom to top. A new value pops (and answers) every smaller index: at $i = 2$ the value 5 answers index 1 and index 0, then waits itself.

```
1 // out[i] = the first value to the right of a[i] that is greater than a[i],
2 // or -1 if there is none.  [2 1 5 3] -> [5 5 -1 -1]
3 // The stack holds indices whose answer is still unknown; their values
4 // decrease from bottom to top.
5 void nextGreater(int a[], int n, int out[]) {
6     stack waiting;
7     stInit(&waiting);
8
```

 Juevesano D. 2026 All rights reserved.

```
    for (int i = 0; i < n; i++) out[i] = -1;

    for (int i = 0; i < n; i++) {
        int j;
        // a[i] answers every waiting index whose value is smaller
        while (stTop(&waiting, &j) && a[j] < a[i]) {
            out[j] = a[i];
            stPop(&waiting, &j);
        }
        stPush(&waiting, i);
    }
}
```

Listing 12.36: Next greater element

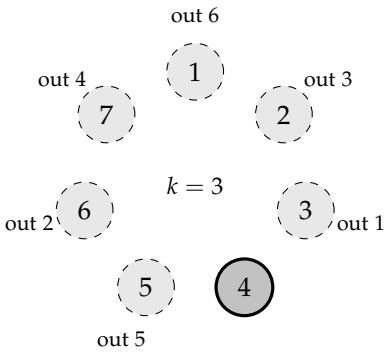
COMPLEXITY. $\Theta(n)$ time, $O(n)$ space.

COMMON MISTAKES.

- Pushing values instead of indices (the answer cannot be written back).
- Popping with $\leq$ instead of $<$ (equal values do not answer each other).

Solution to Problem 37.

IDEA. Put $1, \ldots, n$ in a queue. To remove one person: dequeue $k-1$ people and enqueue each at the back, then dequeue the next person and discard. Repeat until one person remains. *Recurrence (bonus):* number the people $0, \ldots, m-1$. After the first removal the circle has $m-1$ people, and the survivor's index shifts by k modulo m. Hence $J(1) = 0$ and $J(m) = (J(m-1) + k) \bmod m$; the answer is $J(n) + 1$.



queue (front at left)	
1 2 3 4 5 6 7	*start*
4 5 6 7 1 2	3 *removed*
7 1 2 4 5	6 *removed*
4 5 7 1	2 *removed*

$n = 7$, $k = 3$. *Counting clockwise from person* 1*, the people leave in the order* $3, 6, 2, 7, 5, 1$ *(numbers beside the circle); person* 4 *survives. With a queue: dequeue and re-enqueue* $k-1$ *people, then dequeue and drop the k-th.*

```
// n people stand in a circle, numbered 1..n. Starting at 1, every k-th
// person is removed until one is left. Returns that person's number.
int josephus(int n, int k) {
    queue q;
    int x;
    qInit(&q);
    for (int i = 1; i <= n; i++) qEnq(&q, i);
```

```
8
9       while (qSize(&q) > 1) {
10          for (int i = 1; i < k; i++) {    // the first k-1 are skipped:
11              qDeq(&q, &x);                // they go to the back
12              qEnq(&q, x);
13          }
14          qDeq(&q, &x);                    // the k-th is removed
15      }
16      qDeq(&q, &x);
17      return x;
18  }
19
20  // Bonus, O(n) time and O(1) space. With i people the survivor's 0-based
21  // position is (survivor with i-1 people + k) % i.
22  int josephusFast(int n, int k) {
23      int pos = 0;                         // one person: position 0
24      for (int i = 2; i <= n; i++) pos = (pos + k) % i;
25      return pos + 1;
26  }
```

Listing 12.37: Josephus problem

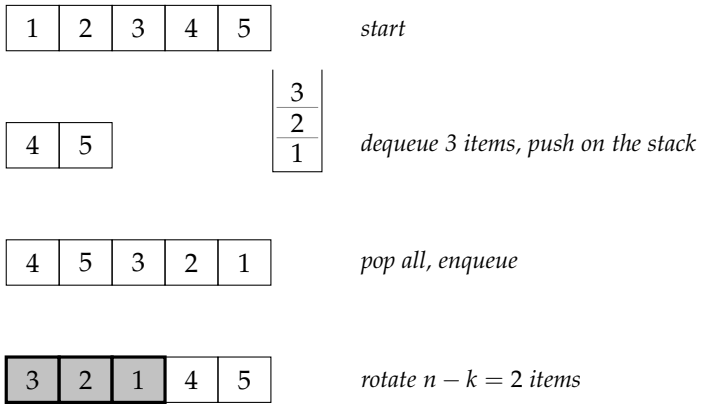
Complexity. Simulation $\Theta(nk)$ time, $\Theta(n)$ space. Recurrence $\Theta(n)$ time, $O(1)$ space.

Common mistakes.

- Rotating k instead of $k-1$ people before the removal.
- Returning $J(n)$ instead of the person number $J(n)+1$.

Solution to Problem 38.

Idea. Dequeue the first k items onto the stack: they come out in reverse order. Pop them one by one and enqueue them again: the k reversed items are now at the back, behind the $n-k$ untouched ones. Rotate the $n-k$ untouched items from the front to the back so that they stand behind the reversed ones.



$k=3$ *on* $[1,2,3,4,5]$. *The first k items go through a stack (which reverses them) back to the end of the queue; then the remaining n − k items are rotated behind them.*

 Juevesano D. 2026 All rights reserved.

```c
// Reverses the order of the first k items of the queue; the rest stay.
// [1 2 3 4 5], k = 3 -> [3 2 1 4 5]. Assumes 0 <= k <= size.
void reverseFirstK(queue *q, int k) {
    stack st;
    int x;
    stInit(&st);

    for (int i = 0; i < k; i++) {        // the stack reverses the first k
        qDeq(q, &x);
        stPush(&st, x);
    }
    while (stPop(&st, &x)) qEnq(q, x);   // put them back at the end

    for (int i = 0; i < qSize(q) - k; i++) {   // rotate the other items
        qDeq(q, &x);                           // behind the reversed ones
        qEnq(q, x);
    }
}
```

Listing 12.38: Reverse the first k items of a queue

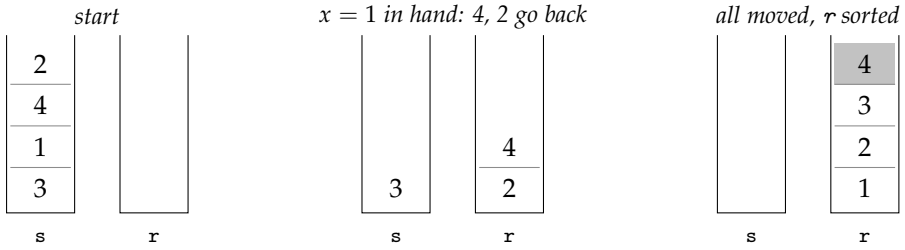
Complexity. $\Theta(n)$ time, $O(k)$ extra space.

Common mistakes.

- Forgetting the final rotation of the $n - k$ remaining items.
- Rotating k instead of $n - k$ items.

Solution to Problem 39.

Idea. Insertion sort with two stacks. Keep the second stack `r` sorted with its *largest* item on top. Take an item `x` from the input; while the top of `r` is larger than `x`, move that top back to the input to make room; then push `x` onto `r`. At the end `r` has the largest on top; moving all items back to the input reverses the order and puts the smallest on top.



Items are taken from `s` one at a time (here $x = 1$) and inserted into `r`, which is kept sorted with the largest on top. Larger items that are in the way (4 and 2) go back to `s`. At the end `r` is sorted, and pouring it back reverses it: the smallest ends on top.

```c
// Sorts the stack so that the smallest item is on top (popping gives
// 1, 2, 3, ...). Only one extra stack is used.
void sortStack(stack *s) {
```

```
    stack r;                              // sorted, largest on top
    int x, y;
    stInit(&r);

    while (stPop(s, &x)) {
        // items in r larger than x must go back to s so that x fits below them
        while (stTop(&r, &y) && y > x) {
            stPop(&r, &y);
            stPush(s, y);
        }
        stPush(&r, x);
    }
    while (stPop(&r, &x)) stPush(s, x);  // moving back reverses the order
}
```

Listing 12.39: Sort a stack

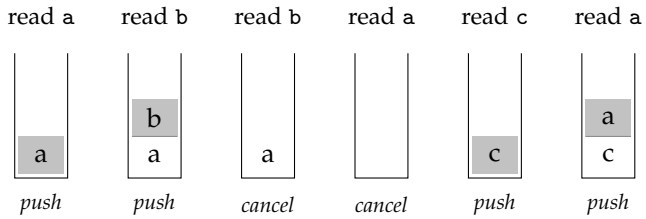
Complexity. $\Theta(n^2)$ time (an item may be moved $O(n)$ times), $\Theta(n)$ extra space.

Common mistakes.

- Using the opposite comparison and sorting in the wrong order.
- Dropping `x` while shuffling items between the stacks.

Solution to Problem 40.

Idea. The stack holds the survivors so far. A new letter cancels the top if they are equal, and is pushed otherwise. A deletion can expose new equal neighbours; the stack pairs them with the next letter automatically. At the end the stack holds the answer, last letter on top, so fill `out` from the back.



The stack holds the letters kept so far. If the next letter equals the top it cancels it (pop); otherwise it is pushed. At the end the stack, read bottom to top, is the answer `ca`.

```
// Repeatedly removes two equal neighbouring letters. "abbaca" -> "ca".
// Result is written to out.
void removeAdjacent(char s[], char out[]) {
    stack st;
    int top;
    stInit(&st);

    for (int i = 0; s[i] != '\0'; i++) {
        if (stTop(&st, &top) && top == s[i]) {
            stPop(&st, &top);            // equal to the top: both vanish
```

 Juevesano D. 2026 All rights reserved.

```
        } else {
            stPush(&st, s[i]);
        }
    }
    int k = stSize(&st);                // the stack holds the answer
    out[k] = '\0';
    while (stPop(&st, &top)) out[--k] = top;
}
```

Listing 12.40: Remove adjacent duplicates

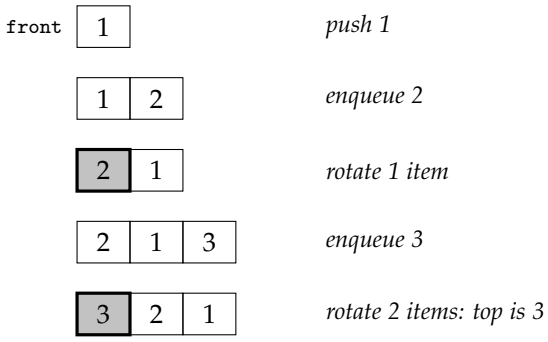
Complexity. $\Theta(n)$ time, $O(n)$ space.

Common mistakes.

- Doing a single left-to-right pass without re-checking after a deletion.
- Restarting the scan after every deletion ($O(n^2)$).

Solution to Problem 41.

Idea. On `qsPush`, enqueue x and then rotate the older items (dequeue and enqueue each one) so that the newest item sits at the front. `qsPop` and `qsTop` are then a plain dequeue and front.



After each push the older items are rotated behind the new one, so the front of the queue is always the top of the stack. Pushing 3: enqueue it, then move the two older items to the back.

```
// A stack built from one queue. After each push the new item is rotated to
// the front of the queue, so the front is always the top of the stack.
typedef struct {
    queue q;
} queueStack;

void qsInit(queueStack *s) {
    qInit(&s->q);
}

void qsPush(queueStack *s, int x) {
    int y;
    qEnq(&s->q, x);
    for (int i = 0; i < qSize(&s->q) - 1; i++) {   // older items go behind x
```

Juevesano D. 2026 All rights reserved.

```
        qDeq(&s->q, &y);
        qEnq(&s->q, y);
    }
}

int qsPop(queueStack *s, int *x) {
    return qDeq(&s->q, x);
}

int qsTop(queueStack *s, int *x) {
    return qFront(&s->q, x);
}

int qsEmpty(queueStack *s) {
    return qEmpty(&s->q);
}
```

Listing 12.41: A stack from a queue

Complexity. `qsPush` $\Theta(n)$; `qsPop` and `qsTop` $\Theta(1)$.

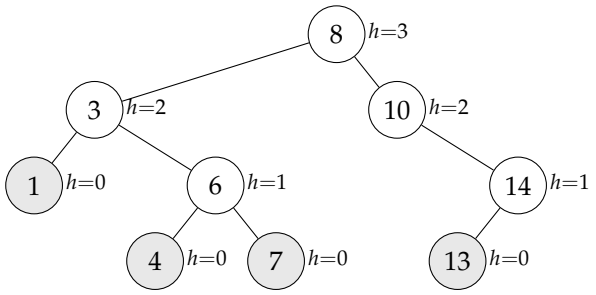
Common mistakes.

- Rotating all $n+1$ items (a full cycle, which restores the old order).
- Rotating before enqueueing x.

12.4 Binary Trees and Binary Search Trees

Solution to Problem 42.

Idea. All the measures have the shape `f(NULL) = BASE; f(r) = COMBINE(f(left), f(right))`. For `countNodes`, BASE is 0 and COMBINE is $1+\ell+r$. For `height`, BASE is -1 and COMBINE is $1+\max(\ell, r)$. A leaf is a node whose two children are `NULL`, so `countLeaves` needs that case before the general one.



The tree S_1 with the height of every node. A leaf has height 0, an empty subtree -1, so $h(v) = 1 + \max(h(\mathit{left}), h(\mathit{right}))$. Leaves (shaded): $1, 4, 7, 13$.

```
// Number of nodes. An empty tree has 0.
int countNodes(node *r) {
    if (r == NULL) return 0;
```

 Juevesano D. 2026 All rights reserved.

```
    return 1 + countNodes(r->left) + countNodes(r->right);
}

// Height in edges: a leaf has height 0 and the empty tree has height -1.
int height(node *r) {
    if (r == NULL) return -1;
    int lh = height(r->left);
    int rh = height(r->right);
    if (lh > rh) return 1 + lh;
    return 1 + rh;
}

// Number of leaves (nodes without children).
int countLeaves(node *r) {
    if (r == NULL) return 0;
    if (r->left == NULL && r->right == NULL) return 1;
    return countLeaves(r->left) + countLeaves(r->right);
}

// Sum of all items.
int sumItems(node *r) {
    if (r == NULL) return 0;
    return r->item + sumItems(r->left) + sumItems(r->right);
}
```

Listing 12.42: Tree measures

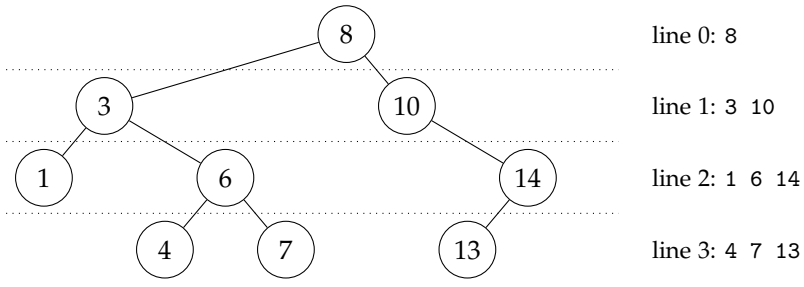
Complexity. $\Theta(n)$ time; the recursion depth is the height, so the extra space is $\Theta(h)$.

Common mistakes.

- Defining the height of the empty tree as 0 (every height is then off by one).
- Counting a node with one child as a leaf.

Solution to Problem 43.

Idea. Before processing a level, record $k =$ the current queue size; then dequeue exactly k nodes, print them, and enqueue their children. The children form the next level.



Level order visits the nodes level by level; the queue holds the nodes of the current level followed by the children already discovered. The output is one line per level.

Juevesano D. 2026 All rights reserved.

```
// Prints the tree level by level, one level per line, left to right.
//        8
//      /   \          prints   8
//     3     10                 3 10
//    / \      \                1 6 14
//   1   6     14
void printLevels(node *root) {
    if (root == NULL) return;
    nqueue q;
    q.push(root);

    while (!q.empty()) {
        int width = q.size();                // number of nodes on this level
        for (int i = 0; i < width; i++) {
            node *cur = q.pop();
            if (i > 0) cout << " ";
            cout << cur->item;
            if (cur->left != NULL) q.push(cur->left);
            if (cur->right != NULL) q.push(cur->right);
        }
        cout << endl;
    }
}
```

Listing 12.43: Level order printing

Complexity. $\Theta(n)$ time, $O(w)$ extra space where w is the width of the tree.

Common mistakes.

- Reading the queue size inside the loop (it changes as children are added).
- A space after the last item of a line.

Solution to Problem 44.

Idea. Comparing a node only with its children is not enough: in the fourth test, $6 > 3$ is fine locally but 6 lies in the left subtree of 5. Pass down the two nodes that bound the allowed items: going left, the current node becomes the new upper bound; going right, it becomes the new lower bound. An item x is acceptable if $x \geq lo$ (when there is a lower bound) and $x < hi$ (when there is an upper bound). Passing the bounding *nodes* avoids choosing a "minus infinity" value.



Left: comparing every node only with its parent *accepts this tree, but* 6 *lies in the left subtree of* 5 *and must be smaller than* 5*. Passing the interval of allowed values down catches it:* $6 \notin (3,5)$*. Right: all items fit their intervals.*

 Juevesano D. 2026 All rights reserved.

```c
// Every item in r must be >= lo->item (if lo exists) and < hi->item (if hi
// exists). Going left, the parent becomes the new upper bound; going right,
// it becomes the new lower bound.
bool inRange(node *r, node *lo, node *hi) {
    if (r == NULL) return true;
    if (lo != NULL && r->item < lo->item) return false;
    if (hi != NULL && r->item >= hi->item) return false;
    return inRange(r->left, lo, r) && inRange(r->right, r, hi);
}

// A BST: left subtree items < node item <= right subtree items, at every node.
bool isBST(node *root) {
    return inRange(root, NULL, NULL);
}
```

Listing 12.44: Validate a binary search tree

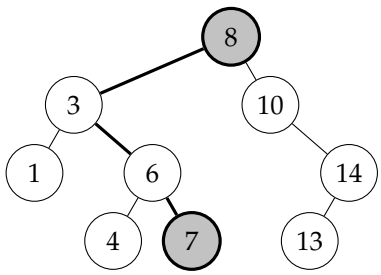
Complexity. $\Theta(n)$ time, $\Theta(h)$ space.

Common mistakes.

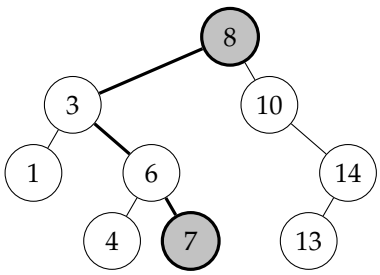
- Checking only parent–child pairs.
- Using $\leq$ on the wrong side (equal items go right in this course).

Solution to Problem 45.

Idea. *Successor.* If x has a right subtree, the successor is the leftmost node of it. Otherwise climb: go up while we come from a right child; the first ancestor reached from its left child is the successor (if we run out of ancestors there is none). The predecessor is the mirror image.



successor of 7



predecessor of 8

Left: node 7 has no right child, so climb until you arrive from a left *child; that parent (8) is the successor. Right: node* 8 *has a left child, so its predecessor is the largest item of the left subtree: go left once, then right as far as possible.*

```c
// Leftmost node of a non-empty subtree = its smallest item.
node *minNode(node *r) {
    while (r->left != NULL) r = r->left;
    return r;
}

```

Juevesano D. 2026 All rights reserved.

```
node *maxNode(node *r) {
    while (r->right != NULL) r = r->right;
    return r;
}

// The node with the next larger item in inorder, or NULL if x is the last.
node *successor(node *x) {
    if (x->right != NULL) return minNode(x->right);   // smallest on the right
    node *child = x;
    node *up = x->parent;
    while (up != NULL && up->right == child) {        // from the right: go up
        child = up;
        up = up->parent;
    }
    return up;                                        // first one from the left
}

// The node with the next smaller item in inorder, or NULL if x is the first.
node *predecessor(node *x) {
    if (x->left != NULL) return maxNode(x->left);
    node *child = x;
    node *up = x->parent;
    while (up != NULL && up->left == child) {
        child = up;
        up = up->parent;
    }
    return up;
}
```

Listing 12.45: Inorder successor and predecessor

Complexity. $\Theta(h)$ time, $O(1)$ space.

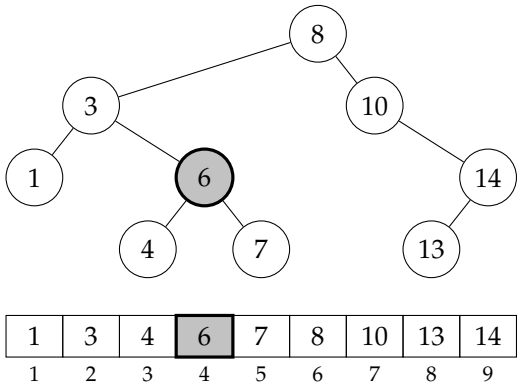
Common mistakes.

- Looking only at the right child when it is `NULL`.
- Climbing while `up->left == child` for the successor (the condition is on the *right* child).

Solution to Problem 46.

Idea. The inorder traversal of a BST visits the items in increasing order, so the answer is the k-th node visited. Traverse inorder with an explicit stack and stop after k nodes.

 Juevesano D. 2026 All rights reserved.



Inorder visits the items in increasing order, so the k-th node visited is the k-th smallest. Stop as soon as the counter reaches k ($k = 4$ gives 6).

```
// Finds the k-th smallest item (k = 1 is the smallest) and stores it in *out.
// Returns 1 on success, 0 if k < 1 or k > number of nodes.
// Walks inorder with an explicit stack and stops after k nodes.
int kthSmallest(node *root, int k, int *out) {
    nstack st;
    node *cur = root;
    int seen = 0;

    while (cur != NULL || !st.empty()) {
        while (cur != NULL) {          // go as far left as possible
            st.push(cur);
            cur = cur->left;
        }
        cur = st.pop();                // the next node in inorder
        seen++;
        if (seen == k) {
            *out = cur->item;
            return 1;
        }
        cur = cur->right;
    }
    return 0;
}
```

Listing 12.46: k-th smallest item of a BST

Complexity. $\Theta(h + k)$ time, $\Theta(h)$ space.

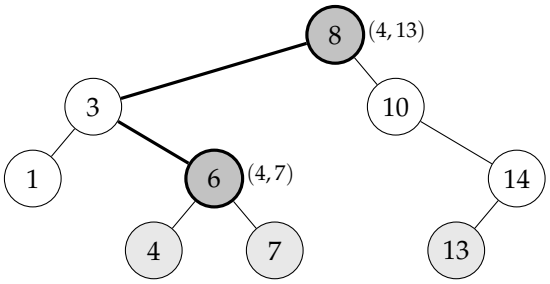
Common mistakes.

- Counting from 0.
- Traversing the whole tree and sorting.

Solution to Problem 47.

Idea. Start at the root. If both items are smaller than the current item, the answer is in the left subtree; if both are larger, it is in the right subtree. Otherwise the two items are

on different sides of the current node (or one of them *is* the current node), so the current node is the lowest common ancestor.



At each node compare p and q with the item. If both are smaller go left, if both are larger go right; the first node that is neither (one on each side, or equal to one of them) is the answer. $(4,7)$*:* $8 \to 3 \to 6$*, and* 6 *splits them.* $(4,13)$ *splits already at the root* 8*.*

```
// Lowest common ancestor of the keys p and q in a BST with distinct keys.
// Both keys are in the tree. Walk down while p and q are on the same side.
node *lowestCommonAncestor(node *root, int p, int q) {
    node *cur = root;
    while (cur != NULL) {
        if (p < cur->item && q < cur->item) cur = cur->left;
        else if (p > cur->item && q > cur->item) cur = cur->right;
        else return cur;       // they split here, or cur is p or q itself
    }
    return NULL;
}
```

Listing 12.47: Lowest common ancestor in a BST

Complexity. $\Theta(h)$ time, $O(1)$ space.

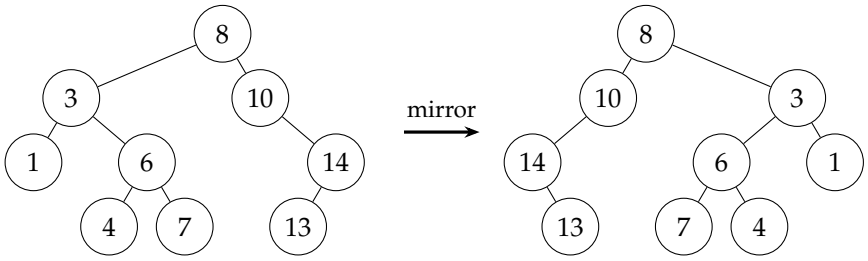
Common mistakes.

- Forgetting that a node may be the ancestor of the other item (the case $(3,7)$).
- Using <= and walking past the answer.

Solution to Problem 48.

Idea. `sameTree` and the helper of `isSymmetric` compare two trees at the same time: two empty trees match, one empty and one not do not match, otherwise the items must be equal and the children must match (for the mirror comparison, the *left* child of one against the *right* child of the other). `hasPathSum` subtracts the item of each node from the target; at a leaf the remaining target must equal the leaf's item. An internal node with a missing child is *not* a leaf, so $11 = 8 + 3$ is false.

 Juevesano D. 2026 All rights reserved.



Mirroring swaps the two children of every node, so the result is the reflection of the tree in a vertical line. The preorder of the mirrored S_1 is $8, 10, 14, 13, 3, 6, 7, 4, 1$.

```
// Swaps the left and right child of every node.
void mirror(node *r) {
    if (r == NULL) return;
    node *tmp = r->left;
    r->left = r->right;
    r->right = tmp;
    mirror(r->left);
    mirror(r->right);
}

// True if a and b have the same shape and the same items.
bool sameTree(node *a, node *b) {
    if (a == NULL && b == NULL) return true;
    if (a == NULL || b == NULL) return false;
    return a->item == b->item
        && sameTree(a->left, b->left)
        && sameTree(a->right, b->right);
}

// True if the left side is the mirror image of the right side.
bool mirrorOf(node *a, node *b) {
    if (a == NULL && b == NULL) return true;
    if (a == NULL || b == NULL) return false;
    return a->item == b->item
        && mirrorOf(a->left, b->right)
        && mirrorOf(a->right, b->left);
}

bool isSymmetric(node *root) {
    if (root == NULL) return true;
    return mirrorOf(root->left, root->right);
}

// True if some root-to-leaf path has items adding up to target.
bool hasPathSum(node *r, int target) {
    if (r == NULL) return false;
    if (r->left == NULL && r->right == NULL) return r->item == target;
    return hasPathSum(r->left, target - r->item)
        || hasPathSum(r->right, target - r->item);
}
```

Juevesano D. 2026 All rights reserved.

Listing 12.48: Mirror, same tree, symmetric, path sum

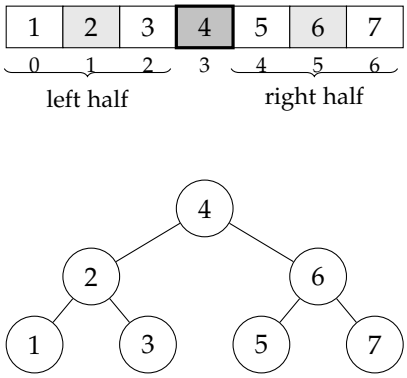
Complexity. $\Theta(n)$ time, $\Theta(h)$ space.

Common mistakes.

- In `isSymmetric`, comparing left with left.
- In `hasPathSum`, accepting a path that ends at a node with one child.

Solution to Problem 49.

Idea. The middle item is the root; the items before it form the left subtree and the items after it the right subtree. Both are built by the same rule. The two sides have sizes that differ by at most 1, so the heights differ by at most 1.



The middle item becomes the root; the left half builds the left subtree and the right half the right subtree, recursively. The depths of the two sides differ by at most 1 *because the halves differ in size by at most* 1.

```
// Builds a tree from a[lo..hi] (sorted): the middle item is the root, the
// left part builds the left subtree, the right part the right subtree.
node *build(int a[], int lo, int hi, node *parent) {
    if (lo > hi) return NULL;
    int mid = (lo + hi) / 2;
    node *r = new node(a[mid]);
    r->parent = parent;
    r->left = build(a, lo, mid - 1, r);
    r->right = build(a, mid + 1, hi, r);
    return r;
}

// Returns the root of a height-balanced BST holding a[0..n-1] (sorted).
node *sortedToBST(int a[], int n) {
    return build(a, 0, n - 1, NULL);
}
```

Listing 12.49: Sorted array to balanced BST

Complexity. $\Theta(n)$ time, height $\lfloor \lg n \rfloor$.

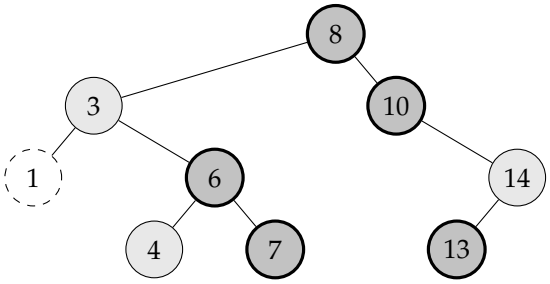
Common mistakes.

 Juevesano D. 2026 All rights reserved.

- Inserting the items one by one in sorted order (the result is a path).
- Not setting `parent`.

Solution to Problem 50.

IDEA. For `rangeCount`: if the item is $< lo$, everything on its left is smaller still, so go right only; if it is $> hi$, go left only; otherwise count it and both sides. For the floor, walk down from the root: if the item equals x we are done; if it is smaller than x it is a candidate and a better one can only be on the right; otherwise go left. The ceiling is the mirror image.



Counting the items in $[5, 13]$ *in* S_1*. Black-bordered shaded nodes are counted* $(8, 6, 7, 10, 13)$*; light nodes are visited but out of range* $(3, 4, 14)$*; the dashed node* 1 *is never visited because* $3 < 5$ *rules out its whole left subtree.*

```
// Number of items x with lo <= x <= hi. Skips the subtrees that cannot match.
int rangeCount(node *r, int lo, int hi) {
    if (r == NULL) return 0;
    // r too small: its whole left side is too small; r too large: its right side
    if (r->item < lo) return rangeCount(r->right, lo, hi);
    if (r->item > hi) return rangeCount(r->left, lo, hi);
    return 1 + rangeCount(r->left, lo, hi) + rangeCount(r->right, lo, hi);
}

// Largest item <= x. Returns 0 if there is none, otherwise 1 and sets *out.
int floorKey(node *root, int x, int *out) {
    int found = 0;
    node *cur = root;
    while (cur != NULL) {
        if (cur->item == x) {
            *out = x;
            return 1;
        }
        if (cur->item < x) {         // candidate; a better one may be right
            *out = cur->item;
            found = 1;
            cur = cur->right;
        } else {
            cur = cur->left;
        }
    }
    return found;
}
```

Juevesano D. 2026 All rights reserved.

```
29
30 // Smallest item >= x. Returns 0 if there is none, otherwise 1 and sets *out.
31 int ceilKey(node *root, int x, int *out) {
32     int found = 0;
33     node *cur = root;
34     while (cur != NULL) {
35         if (cur->item == x) {
36             *out = x;
37             return 1;
38         }
39         if (cur->item > x) {         // candidate; a better one may be left
40             *out = cur->item;
41             found = 1;
42             cur = cur->left;
43         } else {
44             cur = cur->right;
45         }
46     }
47     return found;
48 }
```

Listing 12.50: Range count, floor and ceiling

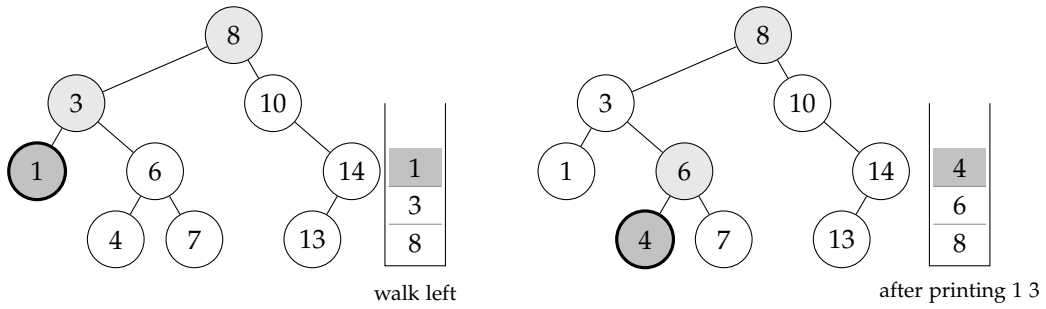
Complexity. Floor and ceiling $\Theta(h)$ time, $O(1)$ space.

Common mistakes.

- Visiting the whole tree ($\Theta(n)$) instead of pruning.
- Forgetting to keep the best candidate seen so far.

Solution to Problem 51.

Idea. Walk down the left side, saving every node on the stack. Pop one node, visit it, and then do the same for its right subtree. The stack holds exactly the ancestors whose left subtree is being processed.



The stack holds the nodes whose left subtree is being visited. Left: after walking left from 8 *the stack is* 8, 3, 1*. Right: after printing* 1, 3 *the walk goes into the right subtree of* 3 *and stops at* 4*; the stack is* 8, 6, 4*.*

```
1 // Prints the items in inorder without recursion.
2 void inorderIterative(node *root) {
3     nstack st;
```

 Juevesano D. 2026 All rights reserved.

```
    node *cur = root;

    while (cur != NULL || !st.empty()) {
        while (cur != NULL) {          // save the path down the left side
            st.push(cur);
            cur = cur->left;
        }
        cur = st.pop();
        cout << cur->item << " ";      // visit
        cur = cur->right;              // then do the right subtree
    }
}
```

Listing 12.51: Inorder traversal without recursion

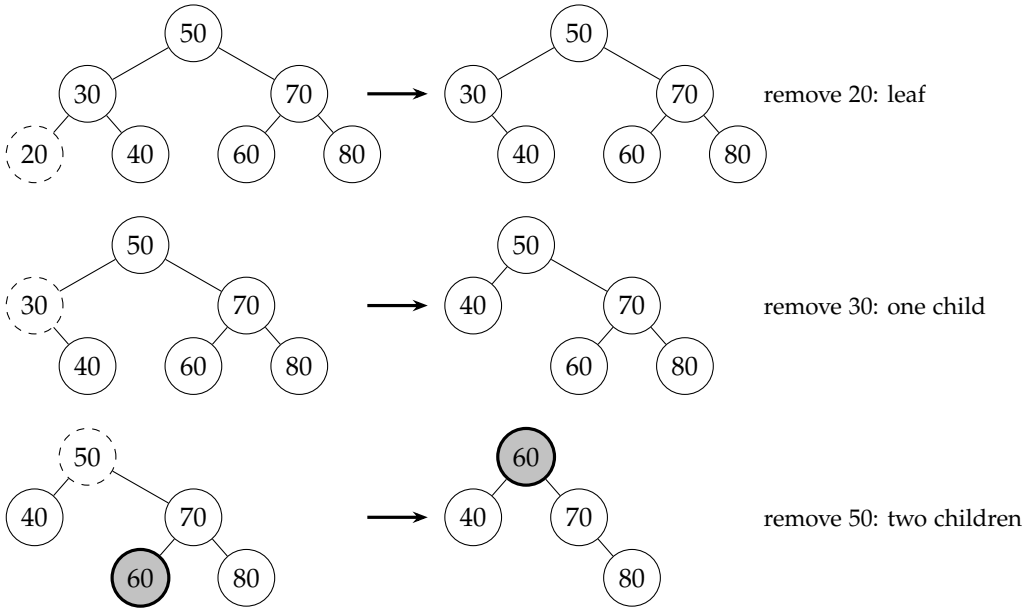
Complexity. $\Theta(n)$ time, $\Theta(h)$ space.

Common mistakes.

- Visiting a node when it is pushed instead of when it is popped.
- Forgetting to move to the right child after the visit.

Solution to Problem 52.

Idea. There are three cases. A node with no child is simply cut off. A node with one child is replaced by that child. A node with two children is not removed itself: copy the successor's item into it and remove the successor, which has no left child, so it falls into one of the first two cases. In every case the child's `parent` must be updated, and so must the `left/right` pointer of the parent (or `*root` when the removed node has no parent).



The three cases (tree $50, 30, 70, 20, 40, 60, 80$*). A leaf is simply cut off. A node with one child is bypassed by its parent. A node with two children takes the item of its successor (smallest of the right subtree) and the successor node, which has no left child, is removed instead.*

Juevesano D. 2026 All rights reserved.

```
// Removes one node with item x. Returns 1 if found, 0 otherwise.
int removeKey(node **root, int x) {
    // 1. find the node
    node *cur = *root;
    while (cur != NULL && cur->item != x) {
        if (x < cur->item) cur = cur->left;
        else cur = cur->right;
    }
    if (cur == NULL) return 0;

    // 2. two children: copy the successor's item here, remove the successor
    if (cur->left != NULL && cur->right != NULL) {
        node *s = cur->right;
        while (s->left != NULL) s = s->left;
        cur->item = s->item;
        cur = s;                        // s has no left child
    }

    // 3. cur has at most one child; that child takes its place
    node *child = cur->left;
    if (child == NULL) child = cur->right;
    if (child != NULL) child->parent = cur->parent;

    if (cur->parent == NULL) *root = child;
    else if (cur->parent->left == cur) cur->parent->left = child;
    else cur->parent->right = child;

    delete cur;
    return 1;
}
```

Listing 12.52: Remove a key from a BST

Complexity. $\Theta(h)$ time, $O(1)$ space.

Common mistakes.

- Updating the parent's child pointer but not the child's `parent`.
- Passing `node *root` so that removing the root cannot change it.
- Forgetting to `delete` the node.

 Juevesano D. 2026 All rights reserved.

Chapter A

Complexity Summary

Structure / operation	access i	insert front	insert back	erase front	erase back
Array list	1	n	1	n	1
Dynamic array (amortised)	1	n	1^*	n	1^*
SLL, head only	i	1	n	1	n
SLL, head and tail	i	1	1	1	n
DLL, head and tail	$\min(i, n-i)$	1	1	1	1
Circular array queue / deque	1	1^*	1^*	1^*	1^*

Table A.1: Costs of the list operations (Θ, worst case; * amortised). Insertion or removal *after a known node*: 1 in linked lists, n in arrays.

Task	Time	Extra space
linear search in a list	$\Theta(n)$	1
binary search in a sorted array	$\Theta(\log n)$	1
insert into a sorted array list	$\Theta(\log n)$ search $+\Theta(n)$ shift	1
reverse a list, in place	$\Theta(n)$	1
middle / k-th from end of an SLL	$\Theta(n)$, one pass	1
merge two sorted lists (relinking)	$\Theta(n+m)$	1
splice after a known node (with tail of the donor)	$\Theta(1)$	1
balanced brackets, postfix evaluation, infix→postfix	$\Theta(n)$	$\Theta(n)$
queue from two stacks	$\Theta(1)$ amortised	$\Theta(n)$
tree traversal (any order)	$\Theta(n)$	$\Theta(h)$ or $\Theta(w)$
BST search / insert / remove	$\Theta(h)$	1 (iterative)
BST with sorted insertions	$h = n-1$	
build a balanced BST from a sorted array	$\Theta(n)$	$\Theta(\log n)$

Table A.2: Summary of the algorithmic tasks of this volume.

Frequently needed sums..

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}, \qquad \sum_{i=0}^{k} 2^i = 2^{k+1} - 1, \qquad \sum_{i=0}^{\lfloor \lg n \rfloor} 2^i < 2n, \qquad \lceil \lg(n+1) \rceil - 1 = \lfloor \lg n \rfloor.$$

Juevesano D. 2026 All rights reserved.

Chapter B

Pitfall Checklist

(i) **Empty, one, two.** Run every function mentally on 0, 1 and 2 items.

(ii) **Save before you free or overwrite.** Read `cur->next` into a variable before `free(cur)` or before reassigning `cur->next`.

(iii) **Every pointer that names an end must follow the end.** `head`, `tail`, `size`, `mid`, `parent`: update all of them, in every case.

(iv) **A function that may change the head needs the address of the head** (`node **`) or must return the new head.

(v) **Shallow versus deep.** Copying a struct copies pointers, not nodes. Anything that must outlive its source must be deep copied.

(vi) **Do not walk until `NULL` when the structure changes under you**; walk a counted number of steps.

(vii) **Condition order.** Test the pointer before dereferencing it: `p != NULL && p->item == x`, `fast && fast->next`.

(viii) **Bounds.** Array list positions: legal insert positions are $0..size$, legal erase/access positions $0..size-1$. State whether positions are 0- or 1-based.

(ix) **Integer overflow** in sums, $n(n+1)/2$, products; midpoints use `lo+(hi-lo)/2`.

(x) **Half-open ranges** $[lo, hi)$ make binary search and range operations uniform.

(xi) **Stack and queue discipline.** Pop the right operand first; transfer between stacks reverses order; dequeue until the structure is empty means the queue size must be read once, before the loop modifies it.

(xii) **Tree conventions.** Height of the empty tree -1 (edges); equal keys go right; a BST test needs bounds from all ancestors; a path sum ends at a leaf.

(xiii) **Recursion depth equals the height**; a path-shaped tree of 10^5 nodes can overflow the call stack.

(xiv) **State the complexity** of your solution in a comment, and the extra space.

 Juevesano D. 2026 All rights reserved.

Chapter C

Discussion Notes: Binary Trees in C++

These are the notes taken during the class discussion, completed and checked. Every listing below was compiled and run, the outputs are those of the programs, and the insertion and removal versions were compared against each other on thousands of random inputs. Four small corrections were made to the original code; they are marked **(fix)** where they occur. The program text starts with the `node` class; the `#include` lines and `using namespace std;` are omitted as everywhere else.

C.1 The node class and the three traversals

```
// A binary tree node. The constructor lets us write  new node(x).
class node {
public:
    int item;
    node *left;
    node *right;
    node *parent;
    node(int x) {
        item = x;
        left = NULL;
        right = NULL;
        parent = NULL;
    }
};

// Root, Left, Right
void preOrder(node *r) {
    if (r != NULL) {
        cout << r->item << " ";
        preOrder(r->left);
        preOrder(r->right);
    }
}

// Left, Root, Right
void inOrder(node *r) {
    if (r != NULL) {
        inOrder(r->left);
        cout << r->item << " ";
        inOrder(r->right);
    }
}

```

Juevesano D. 2026 All rights reserved.

```
// Left, Right, Root
void postOrder(node *r) {
    if (r != NULL) {
        postOrder(r->left);
        postOrder(r->right);
        cout << r->item << " ";
    }
}

// Children are freed before their parent: a postorder walk.
void destroy(node *r) {
    if (r != NULL) {
        destroy(r->left);
        destroy(r->right);
        delete r;
    }
}
```

Listing C.1: The node class, the three depth-first traversals, and `destroy`.

The only difference between the traversals is *where* the `cout` sits: before both recursive calls (preorder), between them (inorder), after both (postorder). For the tree with root 1, children 2, 3 and 2's children 4, 5:

Preorder (root, left, right)	1 2 4 5 3
Inorder (left, root, right)	4 2 5 1 3
Postorder (left, right, root)	4 5 2 3 1

Inorder of a BST is sorted. Preorder is the order for *copying* a tree, postorder the order for *deleting* one (`destroy`: children before the parent). All three take $\Theta(n)$ time and $\Theta(h)$ stack space.

Fixes to the first snippet. A `;` after the closing brace of the class; `void`, not `Void`; `NULL`, not `null`; and `public:`, otherwise `main` cannot read the fields.

C.2 A queue of nodes and level order

Level order needs a queue. The lecture version is a linked queue of `node` pointers:

```
// A linked queue of tree nodes, as in the lecture.
class qnode {
public:
    node *data;
    qnode *next;
};

class Queue {
    qnode *front;
    qnode *rear;
public:
    Queue() {
        front = NULL;
        rear = NULL;
```

 Juevesano D. 2026 All rights reserved.

```
    }

    ~Queue() {                          // free what is still waiting
        while (!isEmpty()) dequeue();
    }

    bool isEmpty() {
        return front == NULL;
    }

    // add at the back
    void enqueue(node *n) {
        qnode *temp = new qnode;
        temp->data = n;
        temp->next = NULL;
        if (rear == NULL) {             // the queue was empty
            front = temp;
            rear = temp;
        } else {
            rear->next = temp;
            rear = temp;
        }
    }

    // remove from the front and return it
    node *dequeue() {
        if (isEmpty()) return NULL;
        qnode *temp = front;
        node *n = temp->data;
        front = front->next;
        if (front == NULL) rear = NULL;   // the queue became empty
        delete temp;
        return n;
    }
};
```

Listing C.2: A linked queue of tree nodes.

(fix) The destructor frees nodes still waiting: an insertion that returns early, in the middle of its search, would otherwise leak them.

Own `Queue`	STL `queue<node*>`
`q.enqueue(x)`	`q.push(x)`
`node *d = q.dequeue();`	`node *d = q.front(); q.pop();`
`q.isEmpty()`	`q.empty()`

```
// Level order with our own Queue: top to bottom, left to right.
void levelOrder(node *root) {
    if (root == NULL) return;
    Queue q;
    q.enqueue(root);
    while (!q.isEmpty()) {
```

Juevesano D. 2026 All rights reserved.

```
        node *cur = q.dequeue();
        cout << cur->item << " ";
        if (cur->left != NULL) q.enqueue(cur->left);
        if (cur->right != NULL) q.enqueue(cur->right);
    }
}

// Level order with the STL queue, one level per line.
void levelOrderByLine(node *root) {
    if (root == NULL) return;
    queue<node*> q;
    q.push(root);
    while (!q.empty()) {
        int count = q.size();          // nodes in the current level
        for (int i = 0; i < count; i++) {
            node *cur = q.front();
            q.pop();
            cout << cur->item << " ";
            if (cur->left) q.push(cur->left);
            if (cur->right) q.push(cur->right);
        }
        cout << endl;                  // end of this level
    }
}
```

Listing C.3: Level order, with the own queue and with the STL queue, one level per line.

Steps: enqueue the root; while the queue is not empty, dequeue a node, print it, and enqueue its left child and then its right child if they exist. The by-line version first reads `q.size()`, which is the number of nodes of the current level, and processes exactly that many before printing a newline.

Trace for the tree $1\,(2\,(4,5),\ 3\,(6,\varnothing))$:

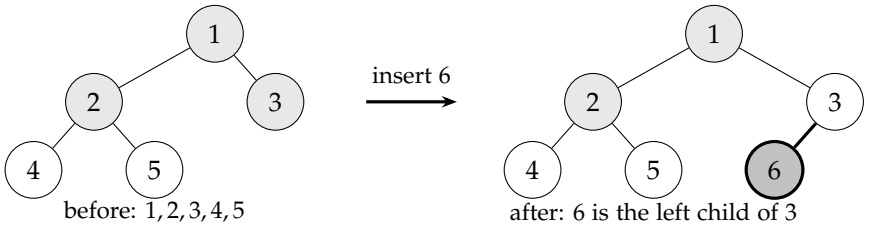
Step	Dequeue	Print	Enqueue	Queue after
start			1	1
1	1	1	2, 3	2 3
2	2	2	4, 5	3 4 5
3	3	3	6	4 5 6
4	4	4		5 6
5	5	5		6
6	6	6		(empty)

Output 1 2 3 4 5 6, and by line 1 / 2 3 / 4 5 6. Time $\Theta(n)$; the queue holds at most one level plus the children discovered so far, $O(w)$ for width w.

C.3 Insertion from left to right

The task. Attach the new node at the first position, in level order, that is missing a child. The tree stays *complete*.

 Juevesano D. 2026 All rights reserved.



Left-to-right insertion into the tree $1, \ldots, 5$*. Level order meets the first node with a free child slot:* 1, 2 *are full,* 3 *has no left child, so* 6 *goes there.*

```
// Version 1: return the root.          root = insertRoot(root, x);
node *insertRoot(node *root, int val) {
    node *newNode = new node(val);
    if (root == NULL) return newNode;      // empty tree
    Queue q;
    q.enqueue(root);
    while (!q.isEmpty()) {
        node *cur = q.dequeue();
        if (cur->left == NULL) {           // free spot on the left
            cur->left = newNode;
            newNode->parent = cur;
            return root;
        }
        q.enqueue(cur->left);
        if (cur->right == NULL) {          // free spot on the right
            cur->right = newNode;
            newNode->parent = cur;
            return root;
        }
        q.enqueue(cur->right);
    }
    return root;
}

// Version 2: r is the ADDRESS of the root pointer.   insertLevel(&root, x);
void insertLevel(node **r, int x) {
    if (*r != NULL) {
        node *n = new node(x);
        queue<node*> q;
        q.push(*r);
        do {
            node *d = q.front();
            q.pop();
            if (d->left == NULL) {
                d->left = n;
                n->parent = d;
                break;
            } else
                q.push(d->left);
            if (d->right == NULL) {
                d->right = n;
                n->parent = d;
```

Juevesano D. 2026 All rights reserved.

```
                break;
            } else
                q.push(d->right);
        } while (!q.empty());
    } else
        *r = new node(x);              // empty tree: change the caller's root
}

// Version 3: r is a REFERENCE to the root pointer.   insertRef(root, x);
void insertRef(node *&r, int x) {
    if (r == NULL) {
        r = new node(x);
        return;
    }
    node *n = new node(x);
    queue<node*> q;
    q.push(r);
    while (!q.empty()) {
        node *d = q.front();
        q.pop();
        if (d->left == NULL) {
            d->left = n;
            n->parent = d;
            return;
        }
        q.push(d->left);
        if (d->right == NULL) {
            d->right = n;
            n->parent = d;
            return;
        }
        q.push(d->right);
    }
}
```

Listing C.4: Level-order insertion, three ways to give the caller its (possibly new) root.

Steps: if the tree is empty the new node is the root; otherwise enqueue the root; dequeue a node and, if its left is empty, put the new node there and stop, else enqueue its left child; do the same on the right; repeat. Trace for inserting 6 into $1\,(2\,(4,5),\,3)$:

Step	Dequeue	Left? / Right?	Queue after
start			1
1	1	2 full, enqueue; 3 full, enqueue	2 3
2	2	4 full, enqueue; 5 full, enqueue	3 4 5
3	3	left is empty: **insert** 6 **here**	stop

If the tree is perfectly full, the new node becomes the left child of the leftmost leaf and starts a new level. Each insertion may scan the whole tree: $\Theta(n)$. A complete tree stored in an *array* does better: the next free slot is index n, and node i has its children at $2i+1$ and $2i+2$ (this is how heaps work).

 Juevesano D. 2026 All rights reserved.

C.3.1 Mistakes in a first attempt

The first draft of the insertion read (shown as typed):

```
void insert(node *r, int x)
if (r!=NULL)
queue<node*> q
q.push(r);
do {
 node *d = q.push();
cout<<d->item< " ";
if(d->left==NULL)
  d->left = n
   n->parent = d;
Break;
}
else
q.push(d->left)
}while(!q.empty())
```

(i) `node *d = q.push();` must be `node *d = q.front(); q.pop();` (`push` returns nothing).

(ii) The new node `n` is never created: `node *n = new node(x);`.

(iii) The *right* child is never checked, so every new node goes down the left side. Inserting $1, 2, 3$ gives the path $1 \to 2 \to 3$ instead of $1\,(2, 3)$.

(iv) Braces are needed around the body of the `if`, or the `break` runs every time.

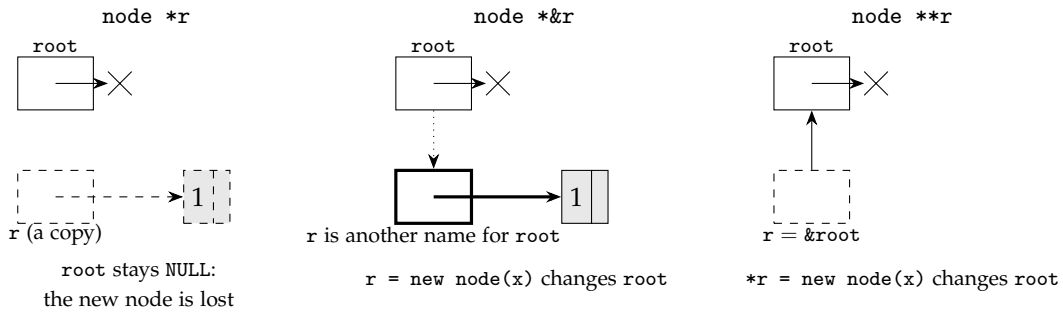
(v) `Break` $\to$ `break`; `< " " ` $\to$ `« " "`; missing semicolons and braces.

(vi) For the empty tree, `else r = new node(x);` only works if `r` is a reference or a double pointer (next section).

(vii) The `cout` line inside the search must be removed (Section C.5).

C.4 Changing the root: `*r`, `*&r`, `**r`

The only reason `insert` needs more than a plain pointer is the *empty tree*: it must change the root pointer itself.



The empty-tree case of `insert`. A plain pointer parameter is a copy; a reference is the caller's pointer under another name; a double pointer holds its address, so `*r` is the caller's pointer.

Juevesano D. 2026 All rights reserved.

	`node **r`	`node *&r`
Call	`insert(&root, x)`	`insert(root, x)`
Inside	`*r` every time	just `r`
Forgetting the `*`	compiles?, or bugs	not possible
May it be `NULL`?	yes (crash risk)	no

Rule. If the function only changes the nodes the pointer points to (`preOrder`, `levelOrder`, `countNodes`, `search`), `node *r` is enough. If it must change the pointer itself (set the root), use `node *&r`, `node **r`, or return the new root: `root = insert(root, x);`. With a double pointer write `*r` for the root pointer: `if (*r != NULL)`, `q.push(*r)`, `*r = new node(x)`.

Usual causes of a segmentation fault.

(i) `node *root;` left uninitialised (garbage); write `node *root = NULL;`.

(ii) `left` and `right` not set to `NULL` in a new node (no constructor, or only `item` set): garbage is pushed and dereferenced.

(iii) `r->left` instead of `(*r)->left` with a double pointer: `*r->left` means `*(r->left)`.

(iv) Calling with the wrong argument: with `node **r` the call is `insert(&root, x)`.

(v) A crash elsewhere: reading `root->item` of an empty tree, or a traversal without the `r != NULL` test.

C.5 Two printing bugs

`else cout « endl` inside a traversal. The `else` runs at *every* `NULL` child, not once at the end: six nodes have seven `NULL` children, so seven newlines are scattered through the output. Print one `endl` in `main`: `preOrder(root); cout « endl;`. Level order is a loop, so its `endl` may sit after the loop.

"0 0 0 1 0 1" before the traversals. A line `cout « d->item « " ";` left inside `insert` prints every node visited while *searching* for a free spot:

Inserting	Nodes visited	Printed
0	none (empty tree)	
1	0	0
2	0	0
3	0, 1	0 1
4	0, 1	0 1

which is 0 0 0 1 0 1. Delete that line. Duplicates are no problem: inserting $0, 0, 0, 1$ simply places them by position.

C.6 Counting nodes, leaves and internal nodes

```
1 // Recursive: count = 1 + count(left) + count(right); the empty tree is 0.
2 int countNodes(node *r) {
3     if (r == NULL) return 0;
4     return 1 + countNodes(r->left) + countNodes(r->right);
5 }
```

 Juevesano D. 2026 All rights reserved.

```
6
7  // The same with a queue: no recursion, so no stack overflow.
8  int countNodesIterative(node *r) {
9      if (r == NULL) return 0;
10     int count = 0;
11     queue<node*> q;
12     q.push(r);
13     while (!q.empty()) {
14         node *d = q.front();
15         q.pop();
16         count++;
17         if (d->left != NULL) q.push(d->left);
18         if (d->right != NULL) q.push(d->right);
19     }
20     return count;
21 }
22
23 // Leaves: nodes without children.
24 int countLeaves(node *r) {
25     if (r == NULL) return 0;
26     if (r->left == NULL && r->right == NULL) return 1;
27     return countLeaves(r->left) + countLeaves(r->right);
28 }
29
30 // Internal nodes: nodes with at least one child.
31 int countInternal(node *r) {
32     if (r == NULL) return 0;
33     if (r->left == NULL && r->right == NULL) return 0;
34     return 1 + countInternal(r->left) + countInternal(r->right);
35 }
```

Listing C.5: Counting by recursion, and once with a queue.

Trace on the tree $0\,(1\,(3,4),\,2)$: `countNodes(3)`$= 1+0+0 = 1$, `countNodes(4)`$= 1$, `countNodes(1)`$= 1+1+1 = 3$, `countNodes(2)`$= 1$, `countNodes(0)`$= 1+3+1 = 5$. So there are 5 nodes, 3 leaves $(3, 4, 2)$ and 2 internal nodes $(0, 1)$. All are $\Theta(n)$; the recursive ones use $\Theta(h)$ stack, the iterative one cannot overflow the stack but its queue holds a whole level.

C.6.1 Putting it together

```
1  int main() {
2      node *tree = NULL;                  // never leave the root uninitialised
3      for (int i = 0; i < 5; i++)
4          insertLevel(&tree, i);
5
6      levelOrder(tree);
7      cout << endl;
8      preOrder(tree);
9      cout << endl;
10     inOrder(tree);
11     cout << endl;
```

Juevesano D. 2026 All rights reserved.

```
    postOrder(tree);
    cout << endl;
    cout << countNodes(tree) << " " << countLeaves(tree) << " "
         << countInternal(tree) << endl;

    node *t2 = NULL;                    // the same tree with 1 to 6, by line
    for (int i = 1; i <= 6; i++)
        t2 = insertRoot(t2, i);
    levelOrderByLine(t2);

    destroy(tree);
    destroy(t2);
    return 0;
}
```

Listing C.6: A `main` for the programs above.

```
0 1 2 3 4
0 1 3 4 2
3 1 4 0 2
3 4 1 2 0
5 3 2
1
2 3
4 5 6
```

The tree built from $0, \ldots, 4$ is $0\,(1\,(3, 4),\, 2)$, giving level $0\,1\,2\,3\,4$, preorder $0\,1\,3\,4\,2$, inorder $3\,1\,4\,0\,2$, postorder $3\,4\,1\,2\,0$. The last three lines are the by-line level order of the tree built from $1, \ldots, 6$. (With the items $1, \ldots, 6$ in the first tree the traversals are level $1\,2\,3\,4\,5\,6$, pre $1\,2\,4\,5\,3\,6$, in $4\,2\,5\,1\,6\,3$, post $4\,5\,2\,6\,3\,1$.)

C.7 Recursive left-to-right insertion

A tree is either `NULL` or a root with two subtrees, so insertion can be recursive; the difficulty is choosing the subtree that keeps the tree complete. Three ways:

Method A (sizes). At each node let L and R be the node counts of the subtrees. If $L = R$, go left (a new level starts); if the left subtree is not perfect, keep filling it; otherwise go right. A tree with n nodes is perfect exactly when $n + 1$ is a power of two, i.e. `((n + 1) & n) == 0`.

Method B (depth). Try to place the node at depth 0, then 1, then 2, ...; at each depth left is tried before right, so the first empty spot found is the leftmost one.

Method C (one pointer). With only `(node *tree, int n)` a correct insertion is impossible; one extra piece of information, the depth `d`, makes it work with a single pointer.

```
// Method A: decide the direction from the sizes of the two subtrees.
// A tree with n nodes is perfect when n + 1 is a power of two.
bool isPerfect(int n) {                 // n = 0, 1, 3, 7, 15, ...
    return ((n + 1) & n) == 0;
}
```

 Juevesano D. 2026 All rights reserved.

```
6
7  void insertA(node **r, int x, node *p = NULL) {
8      if (*r == NULL) {                  // base case: the empty spot
9          *r = new node(x);
10         (*r)->parent = p;
11         return;
12     }
13     int L = countNodes((*r)->left);
14     int R = countNodes((*r)->right);
15     if (L == R || !isPerfect(L))
16         insertA(&(*r)->left, x, *r);   // equal: new level; else keep filling
17     else
18         insertA(&(*r)->right, x, *r);
19 }
20
21 // Method B: try depth 0, then 1, then 2, ... left before right.
22 bool insertAtDepth(node **r, int x, int d, node *p) {
23     if (*r == NULL) {
24         if (d == 0) {                  // an empty spot at the right depth
25             *r = new node(x);
26             (*r)->parent = p;
27             return true;
28         }
29         return false;
30     }
31     if (d == 0) return false;          // the spot is taken
32     return insertAtDepth(&(*r)->left, x, d - 1, *r) ||
33            insertAtDepth(&(*r)->right, x, d - 1, *r);
34 }
35
36 void insertB(node **r, int x) {
37     for (int d = 0; ; d++)
38         if (insertAtDepth(r, x, d, NULL))
39             break;
40 }
41
42 // Method C: a single pointer is enough once the depth d is passed along.
43 // d = 1 means: the children of this node are at the target level.
44 bool rinsert(node *tree, int n, int d) {
45     if (tree == NULL) return false;
46     if (d == 1) {
47         if (tree->left == NULL) {
48             tree->left = new node(n);
49             tree->left->parent = tree;
50             return true;
51         }
52         if (tree->right == NULL) {
53             tree->right = new node(n);
54             tree->right->parent = tree;
55             return true;
56         }
57         return false;
58     }
```

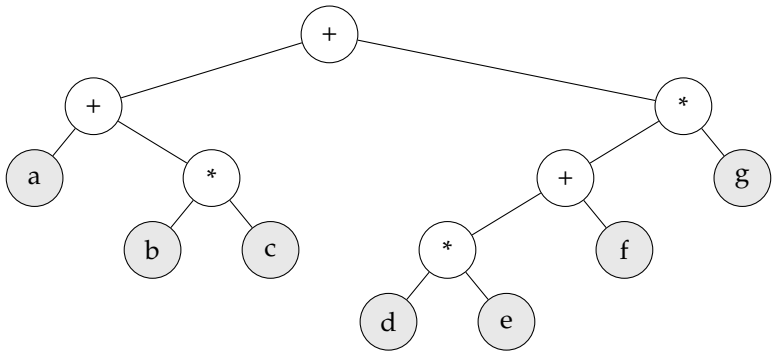
Juevesano D. 2026 All rights reserved.

```
59      return rinsert(tree->left, n, d - 1) ||
60             rinsert(tree->right, n, d - 1);
61  }
62
63  void insertC(node **tree, int x) {
64      if (*tree == NULL)
65          *tree = new node(x);
66      else
67          for (int d = 1; !rinsert(*tree, x, d); d++)
68              ;                           // try the next level down
69  }
```

Listing C.7: Three recursive insertions that build the same tree as the queue version.

Trace of Method B, inserting 5 into the tree $0, \ldots, 4$: depth 0, the root is taken (false); depth 1, 1 and 2 are taken (false); depth 2, under 1 the slots of 3 and 4 are taken, under 2 the left slot is empty: insert 5 there. All three methods built the same tree as the queue version (checked for sizes 1 to 60, including the parent pointers). Each insertion costs $\Theta(n)$.

C.8 Expression trees



operators: internal nodes; operands: leaves (shaded); the operator applied last is the root

The expression tree of $(a + b * c) + ((d * e + f) * g)$. Postorder gives the postfix form `abc*+de*f+g*+`*, inorder the infix form, preorder the prefix form.*

Operators are internal nodes, operands are leaves, and the operator applied *last* is the root. Reading the tree above:

Preorder (prefix, Polish)	`+ + a * b c * + * d e f g`
Inorder (infix, no parentheses)	`a + b * c + d * e + f * g`
Postorder (postfix, RPN)	`a b c * + d e * f + g * +`

Postorder built bottom up: $b * c \to$ `b c *`; $a + b * c \to$ `a b c * +`; $d * e \to$ `d e *`; $d * e + f \to$ `d e * f +`; $(d * e + f) * g \to$ `d e * f + g *`; the whole tree gives `a b c * + d e * f + g * +`.

```
1  // An expression-tree node holds one character: an operator or an operand.
2  class enode {
3  public:
4      char item;
```

 Juevesano D. 2026 All rights reserved.

```
    enode *left;
    enode *right;
    enode *parent;
    enode(char x) {
        item = x;
        left = NULL;
        right = NULL;
        parent = NULL;
    }
};

void preOrderE(enode *r) {             // prefix (Polish) form
    if (r != NULL) {
        cout << r->item << " ";
        preOrderE(r->left);
        preOrderE(r->right);
    }
}

void inOrderE(enode *r) {              // infix form, no parentheses
    if (r != NULL) {
        inOrderE(r->left);
        cout << r->item << " ";
        inOrderE(r->right);
    }
}

void postOrderE(enode *r) {            // postfix (RPN) form
    if (r != NULL) {
        postOrderE(r->left);
        postOrderE(r->right);
        cout << r->item << " ";
    }
}

// Infix WITH parentheses around every operator node.
void inOrderParen(enode *r) {
    if (r != NULL) {
        bool isOp = (r->left != NULL);
        if (isOp) cout << "(";
        inOrderParen(r->left);
        cout << r->item;
        inOrderParen(r->right);
        if (isOp) cout << ")";
    }
}

// Build from postfix with a stack: operands are pushed; an operator pops
// its RIGHT child first, then its LEFT child, and is pushed itself.
enode *buildFromPostfix(string s) {
    stack<enode*> st;
    for (int i = 0; i < (int)s.size(); i++) {
        char c = s[i];
```

Juevesano D. 2026 All rights reserved.

```
        enode *n = new enode(c);
        if (c == '+' || c == '-' || c == '*' || c == '/') {
            n->right = st.top();
            st.pop();
            n->left = st.top();
            st.pop();
            n->left->parent = n;
            n->right->parent = n;
        }
        st.push(n);
    }
    return st.top();                    // the root
}

// Postorder evaluation: value of the left, value of the right, then apply.
int evaluate(enode *r) {
    if (r->left == NULL && r->right == NULL)   // a leaf: one digit
        return r->item - '0';
    int L = evaluate(r->left);
    int R = evaluate(r->right);
    switch (r->item) {
        case '+': return L + R;
        case '-': return L - R;
        case '*': return L * R;
        case '/': return L / R;
    }
    return 0;
}

void destroyE(enode *r) {
    if (r != NULL) {
        destroyE(r->left);
        destroyE(r->right);
        delete r;
    }
}
```

Listing C.8: An expression tree: traversals, parentheses, building from postfix, evaluation.

Building from postfix uses a stack: an operand makes a node and is pushed; an operator makes a node, pops its *right* child first and then its *left*, and is pushed. Evaluation is a postorder walk. With $a = 1, b = 2, c = 3, d = 4, e = 5, f = 6, g = 7$: $b * c = 6$, $a + 6 = 7$, $d * e = 20$, $20 + f = 26$, $26 * g = 182$, and $7 + 182 = 189$.

```
int main() {
    // (a+b*c)+((d*e+f)*g)  in postfix:  a b c * + d e * f + g * +
    enode *t = buildFromPostfix("abc*+de*f+g*+");
    cout << "pre:   ";  preOrderE(t);   cout << endl;
    cout << "in:    ";  inOrderE(t);    cout << endl;
    cout << "post:  ";  postOrderE(t);  cout << endl;
    cout << "paren: ";  inOrderParen(t);  cout << endl;
    destroyE(t);

```

 Juevesano D. 2026 All rights reserved.

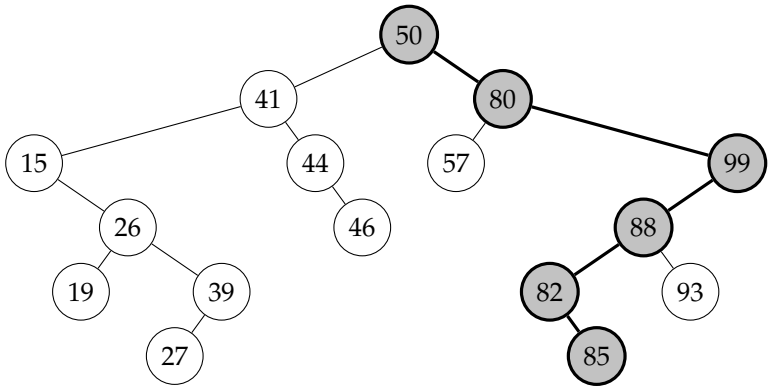
```
    // the same shape with a=1 b=2 c=3 d=4 e=5 f=6 g=7
    t = buildFromPostfix("123*+45*6+7*+");
    cout << "value: " << evaluate(t) << endl;
    destroyE(t);
    return 0;
}
```

Listing C.9: Building, printing and evaluating the expression tree.

```
pre:   + + a * b c * + * d e f g
in:    a + b * c + d * e + f * g
post:  a b c * + d e * f + g * +
paren: ((a+(b*c))+(((d*e)+f)*g))
value: 189
```

C.9 Binary search trees

The rule. At every node, the items of the left subtree are smaller and those of the right subtree are greater than or equal. Every operation uses this rule to choose a direction, and inorder prints the items sorted. The tree of the discussion is built by inserting $50, 41, 15, 26, 19, 39, 27, 44, 46, 80, 57, 99, 88, 82, 85, 93$ into an empty tree:



The 16-node BST of the discussion (insertion order $50, 41, 15, 26, 19, 39, 27, 44, 46, 80, 57, 99, 88, 82, 85, 93$*), height* 5*. The thick path is the search for* 85*:* $50 \to 80 \to 99 \to 88 \to 82 \to 85$*. Searching* 90 *follows* $50, 80, 99, 88, 93$ *and falls off the left of* 93*.*

```
// ---------- search ----------
// Returns the node holding x, or NULL.
node *search(node *r, int x) {
    if (r == NULL || r->item == x) return r;
    if (x < r->item) return search(r->left, x);
    return search(r->right, x);
}

// Returns true or false, recursively.
bool contains(node *r, int x) {
    if (r == NULL) return false;       // reached the bottom: not in the tree
    if (r->item == x) return true;     // found it
```

```
    if (x < r->item) return contains(r->left, x);   // "return" matters:
    return contains(r->right, x);                    // pass the answer up
}

// Returns true or false, with a loop.
bool containsIter(node *r, int x) {
    while (r != NULL) {
        if (r->item == x) return true;
        if (x < r->item) r = r->left;
        else r = r->right;
    }
    return false;
}

// ---------- minimum and maximum ----------
// The minimum is the leftmost node, the maximum the rightmost.
node *minimum(node *r) {
    if (r == NULL) return NULL;
    while (r->left != NULL) r = r->left;
    return r;
}

node *maximum(node *r) {
    if (r == NULL) return NULL;
    while (r->right != NULL) r = r->right;
    return r;
}

// The int versions crash on an empty tree: check in the caller.
int minValue(node *r) {
    while (r->left != NULL) r = r->left;
    return r->item;
}

int maxValueRec(node *r) {
    if (r->right == NULL) return r->item;
    return maxValueRec(r->right);
}

// ---------- insert: all four versions give the same tree ----------
// A) recursive, with the parent passed down.
void insert(node **r, int x, node *p = NULL) {
    if (*r == NULL) {
        *r = new node(x);
        (*r)->parent = p;
        return;
    }
    if (x < (*r)->item) insert(&(*r)->left, x, *r);
    else insert(&(*r)->right, x, *r);
}

// B) recursive, checking BEFORE going down.
void insertB(node **r, int x) {
```

 Juevesano D. 2026 All rights reserved.

```
    if (*r == NULL) {                   // empty tree: the new node is the root
        *r = new node(x);
        return;
    }
    node *cur = *r;
    if (x < cur->item) {
        if (cur->left == NULL) {        // check before going left
            cur->left = new node(x);
            cur->left->parent = cur;
        } else
            insertB(&cur->left, x);
    } else {
        if (cur->right == NULL) {       // check before going right
            cur->right = new node(x);
            cur->right->parent = cur;
        } else
            insertB(&cur->right, x);
    }
}

// C) iterative, with a trailing pointer p one step behind cur.
void insertC(node **r, int x) {
    node *n = new node(x);
    if (*r == NULL) {
        *r = n;
        return;
    }
    node *cur = *r;
    node *p = NULL;
    while (cur != NULL) {
        p = cur;
        if (x < cur->item) cur = cur->left;
        else cur = cur->right;
    }
    n->parent = p;
    if (x < p->item) p->left = n;
    else p->right = n;
}

// D) iterative, checking BEFORE going down (no trailing pointer needed).
void insertD(node **r, int x) {
    if (*r == NULL) {
        *r = new node(x);
        return;
    }
    node *cur = *r;
    while (true) {
        if (x < cur->item) {
            if (cur->left == NULL) {    // check before going left
                cur->left = new node(x);
                cur->left->parent = cur;
                return;
            }
```

```
            cur = cur->left;
        } else {
            if (cur->right == NULL) {  // check before going right
                cur->right = new node(x);
                cur->right->parent = cur;
                return;
            }
            cur = cur->right;
        }
    }
}

// ---------- remove: leaf, one child, two children ----------
// r is the address of the pointer that leads to the node being examined,
// so the pointer itself can be changed (even the root).
void remove(node **r, int x) {
    if (*r == NULL) return;                      // not found
    if (x < (*r)->item)
        remove(&(*r)->left, x);
    else if (x > (*r)->item)
        remove(&(*r)->right, x);
    else {                                       // found it
        node *d = *r;
        if (d->left == NULL && d->right == NULL) {       // case 1: a leaf
            *r = NULL;
            delete d;
        } else if (d->left == NULL) {                    // case 2: right only
            *r = d->right;
            (*r)->parent = d->parent;
            delete d;
        } else if (d->right == NULL) {                   // case 2: left only
            *r = d->left;
            (*r)->parent = d->parent;
            delete d;
        } else {                                         // case 3: two kids
            node *s = minimum(d->right);                 // the successor
            d->item = s->item;                           // copy it up
            remove(&d->right, s->item);                  // delete it below
        }
    }
}
```

Listing C.10: Search, minimum and maximum, four insertions, and removal.

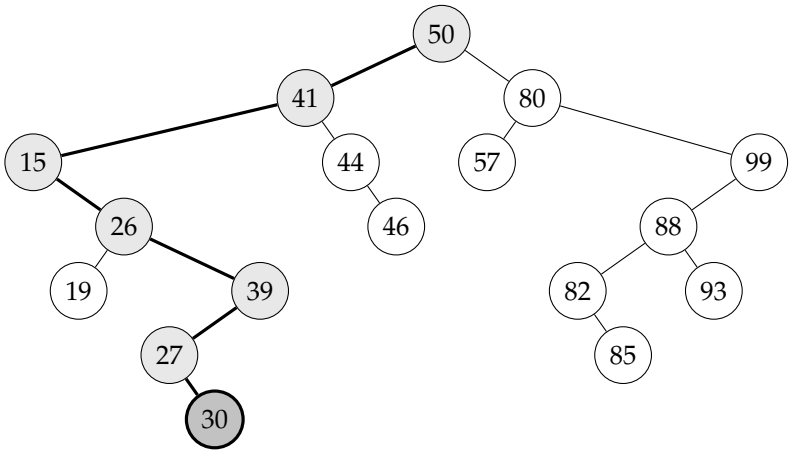
Search. Going down the path $50 \to 80 \to 99 \to 88 \to 82 \to 85$ finds 85 (the thick path above); searching 90 goes $50 \to 80 \to 99 \to 88 \to 93$ and then to the left of 93, which is `NULL`: not found. The `return` before every recursive call matters, otherwise the answer found deeper down never reaches the caller.

Minimum and maximum. The minimum is the leftmost node ($50 \to 41 \to 15$, so 15) and the maximum the rightmost ($50 \to 80 \to 99$, so 99). The `int` versions crash on an empty tree; test in the caller or return a sentinel. If you use the `int` minimum in `remove`, write the

 Juevesano D. 2026 All rights reserved.

two-children case as `int s = minValue(d->right); d->item = s; remove(&d->right, s);`.

Insertion. Go down as in search until `NULL`, then put the new node there; the *value* decides where it goes. The four versions differ only in style; all take $\Theta(h)$ and are called as `insert(&tree, x)`. Version B and D *check before going down*, so they need no trailing pointer; C keeps a pointer one step behind.



Inserting 30: $30 < 50$ *left*, < 41 *left*, > 15 *right*, > 26 *right*, < 39 *left*, > 27 *right; the right slot of* 27 *is empty, so* 30 *is attached there.*

Trace for inserting 30:

cur	Compare	Child checked	Action
50	$30 < 50$	left $= 41$	go left
41	$30 < 41$	left $= 15$	go left
15	$30 > 15$	right $= 26$	go right
26	$30 > 26$	right $= 39$	go right
39	$30 < 39$	left $= 27$	go left
27	$30 > 27$	right $=$ `NULL`	insert 30 here, parent $= 27$

Removal. Three cases: a *leaf* is deleted; a node with *one child* is bypassed (the child is connected to the deleted node's parent); a node with *two children* takes the item of its inorder successor, the minimum of its right subtree, and the successor, which has no left child, is deleted instead (case 1 or 2). `insert` and `remove` take `node **r` because they may change the root; `search`, `minimum` and `maximum` only read, so `node *r` suffices.

```
int main() {
    node *tree = NULL;
    int a[] = {50, 41, 15, 26, 19, 39, 27, 44, 46,
               80, 57, 99, 88, 82, 85, 93};
    for (int i = 0; i < 16; i++)
        insert(&tree, a[i]);

    cout << "Preorder:  ";  preOrder(tree);   cout << endl;
    cout << "Inorder:   ";  inOrder(tree);    cout << endl;
    cout << "Postorder: ";  postOrder(tree);  cout << endl;
```

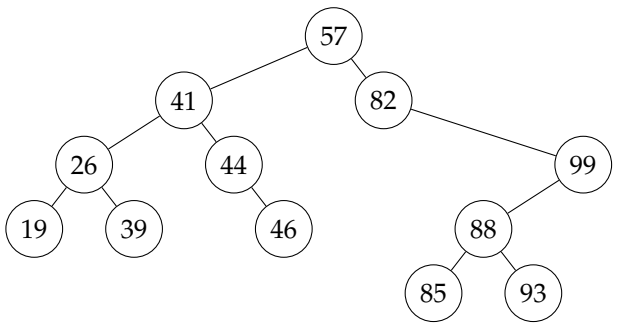
Juevesano D. 2026 All rights reserved.

```
11
12      cout << "Search 85: " << (search(tree, 85) ? "found" : "not found")
13           << endl;
14      cout << "Search 90: " << (search(tree, 90) ? "found" : "not found")
15           << endl;
16      cout << "Minimum: " << minimum(tree)->item << endl;
17      cout << "Maximum: " << maximum(tree)->item << endl;
18
19      remove(&tree, 27);
20      cout << "Remove 27 (leaf):         ";  preOrder(tree);  cout << endl;
21      remove(&tree, 15);
22      cout << "Remove 15 (one child):    ";  preOrder(tree);  cout << endl;
23      remove(&tree, 80);
24      cout << "Remove 80 (two children): ";  preOrder(tree);  cout << endl;
25      remove(&tree, 50);
26      cout << "Remove 50 (root):         ";  preOrder(tree);  cout << endl;
27      cout << "Inorder after removals: ";  inOrder(tree);  cout << endl;
28
29      destroy(tree);
30      return 0;
31  }
```

Listing C.11: The `main` that builds the tree and exercises every operation.

```
Preorder:  50 41 15 26 19 39 27 44 46 80 57 99 88 82 85 93
Inorder:   15 19 26 27 39 41 44 46 50 57 80 82 85 88 93 99
Postorder: 19 27 39 26 15 46 44 41 57 85 82 93 88 99 80 50
Search 85: found
Search 90: not found
Minimum: 15
Maximum: 99
Remove 27 (leaf):         50 41 15 26 19 39 44 46 80 57 99 88 82 85 93
Remove 15 (one child):    50 41 26 19 39 44 46 80 57 99 88 82 85 93
Remove 80 (two children): 50 41 26 19 39 44 46 82 57 99 88 85 93
Remove 50 (root):         57 41 26 19 39 44 46 82 99 88 85 93
Inorder after removals: 19 26 39 41 44 46 57 82 85 88 93 99
```

On the tree above: removing 27 (a leaf) sets the left of 39 to `NULL`; removing 15 (one child) moves 26 up to be the left child of 41, keeping 19 and 39; removing 80 (two children) takes the successor, the minimum of the right subtree ($99 \to 88 \to 82$), so 82 replaces 80 and 85 moves up to be the left child of 88; removing the root 50 takes the successor 57, which becomes the new root.



 Juevesano D. 2026 All rights reserved.

The tree after removing 27 *(leaf),* 15 *(one child),* 80 *(two children, replaced by its successor* 82*) and the root* 50 *(replaced by its successor* 57*).*

C.10 Insertion without a double pointer

The double pointer is needed only for the empty tree. Any other way of updating the root works:

```
// The double pointer is only needed for the EMPTY tree. Any other way of
// updating the root pointer works.

// 1. A reference to the pointer:           bstInsertRef(tree, x);
void bstInsertRef(node *&r, int x) {
    if (r == NULL) {                   // r IS the caller's pointer
        r = new node(x);
        return;
    }
    if (x < r->item) {
        if (r->left == NULL) {
            r->left = new node(x);
            r->left->parent = r;
        } else
            bstInsertRef(r->left, x);
    } else {
        if (r->right == NULL) {
            r->right = new node(x);
            r->right->parent = r;
        } else
            bstInsertRef(r->right, x);
    }
}

// 2. Return the root (the textbook form):    tree = bstInsertRet(tree, x);
node *bstInsertRet(node *r, int x) {
    if (r == NULL) return new node(x);
    if (x < r->item) {
        r->left = bstInsertRet(r->left, x);
        r->left->parent = r;
    } else {
        r->right = bstInsertRet(r->right, x);
        r->right->parent = r;
    }
    return r;
}

// 3. A single pointer; the caller handles the empty tree.
void bstRinsert(node *r, int x) {
    if (x < r->item) {
        if (r->left == NULL) {
            r->left = new node(x);
            r->left->parent = r;
        } else
```

Juevesano D. 2026 All rights reserved.

```
45              bstRinsert(r->left, x);
46      } else {
47          if (r->right == NULL) {
48              r->right = new node(x);
49              r->right->parent = r;
50          } else
51              bstRinsert(r->right, x);
52      }
53  }
54  //      if (tree == NULL) tree = new node(x);
55  //      else              bstRinsert(tree, x);
56
57  // 4. Hide the root inside a class.            BST t;  t.insert(50);
58  class BST {
59      node *root;
60  public:
61      BST() { root = NULL; }
62
63      void insert(int x) {
64          node *n = new node(x);
65          if (root == NULL) {
66              root = n;
67              return;
68          }
69          node *cur = root;
70          while (true) {
71              if (x < cur->item) {
72                  if (cur->left == NULL) {
73                      cur->left = n;
74                      n->parent = cur;
75                      return;
76                  }
77                  cur = cur->left;
78              } else {
79                  if (cur->right == NULL) {
80                      cur->right = n;
81                      n->parent = cur;
82                      return;
83                  }
84                  cur = cur->right;
85              }
86          }
87      }
88
89      void print() {
90          preOrder(root);
91          cout << endl;
92      }
93
94      ~BST() { destroy(root); }
95  };
96
97  // remove has the same issue; the "return the root" form is
```

 Juevesano D. 2026 All rights reserved.

```
//     tree = removeRet(tree, x);
node *removeRet(node *r, int x) {
    if (r == NULL) return NULL;
    if (x < r->item) {
        r->left = removeRet(r->left, x);
    } else if (x > r->item) {
        r->right = removeRet(r->right, x);
    } else {
        if (r->left == NULL || r->right == NULL) {      // 0 or 1 child
            node *child = (r->left != NULL) ? r->left : r->right;
            if (child != NULL) child->parent = r->parent;
            delete r;
            return child;                               // may be NULL
        }
        node *s = minimum(r->right);                    // two children
        r->item = s->item;
        r->right = removeRet(r->right, s->item);
    }
    return r;
}
```

Listing C.12: Four ways to insert without `node **`, and removal that returns the root.

```
int main() {
    int a[] = {50, 41, 15, 26, 19, 39, 27, 44, 46,
               80, 57, 99, 88, 82, 85, 93};

    node *t1 = NULL, *t2 = NULL, *t3 = NULL;
    for (int i = 0; i < 16; i++) {
        bstInsertRef(t1, a[i]);
        t2 = bstInsertRet(t2, a[i]);
        if (t3 == NULL) t3 = new node(a[i]);
        else bstRinsert(t3, a[i]);
    }
    BST t4;
    for (int i = 0; i < 16; i++) t4.insert(a[i]);

    preOrder(t1);  cout << endl;
    preOrder(t2);  cout << endl;
    preOrder(t3);  cout << endl;
    t4.print();

    int gone[] = {27, 15, 80, 50};     // the removals of the previous program
    for (int i = 0; i < 4; i++) t2 = removeRet(t2, gone[i]);
    preOrder(t2);  cout << endl;

    destroy(t1);  destroy(t2);  destroy(t3);
    return 0;
}
```

Listing C.13: All versions build the same tree.

Juevesano D. 2026 All rights reserved.

```
50 41 15 26 19 39 27 44 46 80 57 99 88 82 85 93
50 41 15 26 19 39 27 44 46 80 57 99 88 82 85 93
50 41 15 26 19 39 27 44 46 80 57 99 88 82 85 93
50 41 15 26 19 39 27 44 46 80 57 99 88 82 85 93
57 41 26 19 39 44 46 82 99 88 85 93
```

The four lines are the preorders of the trees built by the four versions (identical); the last line is the preorder after the four removals of the previous program, done with `removeRet`.

Method	Call	Notes
`node **r`	`insert(&tree, x)`	C style, `*r` everywhere
`node *&r`	`bstInsertRef(tree, x)`	cleanest in C++
return the root	`tree = bstInsertRet(tree, x)`	shortest; do not forget `tree =`
check in `main`	`if/else` in `main`	every caller must remember
class	`t.insert(x)`	best for larger programs

(A *global* root variable also works, but only one tree can exist and the course asks you to avoid global variables.)

C.11 Time complexity

Every BST operation follows *one* path from the root downwards, so its cost depends on the height h, not on n.

Operation	Best	Average (random keys)	Worst
search, minimum, maximum	$O(1)$	$O(\log n)$	$O(n)$
insert	$O(1)$	$O(\log n)$	$O(n)$
remove	$O(1)$	$O(\log n)$	$O(n)$
preorder, inorder, postorder	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

- All BST operations are $O(h)$. Removal costs $O(h)$ to find the node plus $O(h)$ to find the successor, still $O(h)$.
- Best $O(1)$: the value is at the root, or the root has no left (right) child where we need to go.
- Traversals visit every node once: always $\Theta(n)$.
- Balanced ($h \approx \lg n$): inserting $4, 2, 6, 1, 3, 5, 7$ gives $h = 2$ and a search takes at most 3 steps. Skewed ($h = n - 1$): inserting $1, 2, \ldots, 7$ in sorted order gives a path with $h = 6$ and a search takes up to 7 steps, as in a linked list. Inserting sorted data is the worst case.
- The tree of the discussion has $n = 16$ and $h = 5$ (paths $50, 80, 99, 88, 82, 85$ and $50, 41, 15, 26, 39, 27$); the best possible height for 16 nodes is 4, so it is close to balanced: at most 6 comparisons per operation.
- Space: recursive versions use $\Theta(h)$ stack; loops (minimum, maximum, iterative search and insert) use $\Theta(1)$.
- Left-to-right (complete tree) insertion costs $\Theta(n)$ per insert, because it may scan the whole tree.

 Juevesano D. 2026 All rights reserved.

Bibliography

[1] P. Morin, *Open Data Structures*, open content text (arrays, linked lists, binary trees, binary search trees).

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

[3] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., Pearson, 2013.

[4] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed., Addison-Wesley, 1997.

[5] R. W. Floyd, Nondeterministic algorithms, *J. ACM* 14 (1967), 636–644.

Juevesano D. 2026 All rights reserved.